

Лабораторна робота № 4-6

Мета: Ознайомитися з процесом створення клієнт-серверної архітектури. Навчитися створювати RESTful API для взаємодії між клієнтом і сервером. Розвинути вміння проєктувати та реалізовувати ендпойнти для типових CRUD-операцій.

ЗАВДАННЯ

ЗАВДАННЯ 1 Створити ASP.NET Core Minimal API

- Створити 3–4 групи маршрутів відповідно до Додатку А (наприклад: /todos, /users, /categories).
- Використати in-memory “сховище” (простий список List в коді) для зберігання даних.
- Реалізувати CRUD для кожної групи маршрутів
- Виконати валідацію даних під час створення нових об’єктів (наприклад, перевірка порожніх полів, мінімальної довжини рядка, формату email тощо).
- Для кожної групи створити окремий набір ендпойнтів у коді.
- Винести ендпойнти в окремі файли (наприклад: Endpoints/ToDoEndpoints.cs, Endpoints/UserEndpoints.cs) і підключити їх у Program.cs.
- Організувати структуру проєкту за рекомендаціями:
- Додати Swagger (OpenAPI) для зручного тестування API через браузер:
- Перевірити роботу API через Swagger або Postman.

ЗАВДАННЯ 2 Створити ASP.NET Core Web API

- Створити 3–4 контролера відповідно до Додатку А (наприклад: TodoController, UserController, CategoryController).
- Створити відповідні моделі даних. Використати Enum, як поле класу.
- Використати in-memory “сховище” (простий список List в коді) для зберігання даних.
- Реалізувати CRUD для кожної моделі в контролерах:
- Виконати валідацію даних під час створення нових об’єктів (наприклад, перевірка порожніх полів, мінімальної довжини рядка, формату email тощо). Для однієї моделі (самої меншої за к-тю полів) виконати валідацію через DataAnnotations. Для інших моделей виконати

					ЛР.ОК24.ПІ231.03.04-06			
Змін.	Аркуш	№ докум.	Підпис	Дата				
Розроб.		Башук О. Ю.			Створення RESTful API	Лім.	Арк.	Акрушів
Перевір.		Жереб Д. В.				Н	1	102
						Х П К		
Н. Контр.								
Затверд.								

- валідація через FluentValidation. Для одного з полів використати Regular Expressions
- Організувати структуру проєкту за рекомендаціями
- Перевірити роботу API через Swagger або Postman.

ЗАВДАННЯ 3 Спроектувати архітектури Controller – Service – Repository

- Розробити логічну структуру Web API, що складається з трьох рівнів:
- Створити UML Component Diagram, на якій відобразити: три основні компоненти: Controllers, Services, Repositories; зв'язки між ними: Controller → Service → Repository; залежності (інтерфейси IService, IRepository).
- Описати кожен рівень (2–3 речення): його роль у системі; що саме він реалізує; приклади методів або обов'язків.

ЗАВДАННЯ 4 Додати MongoDB та реалізація CRUD через Repository/Service. Підключити MongoDB замість in-memory списків і реалізувати CRUD-операції на рівні архітектури Controller–Service–Repository.

- Підключити MongoDB:
- Оновити моделі: додати атрибути [BsonId], BsonRepresentation(BsonType.ObjectId)]; підготувати моделі документів для MongoDB.
- Створити Repository-шар: створити інтерфейси IRepository з методами: GetAllAsync(), GetByIdAsync(id), CreateAsync(entity), UpdateAsync(entity), DeleteAsync(id); реалізувати їх за допомогою MongoDB драйвера.
- Створити Service-шар: створити інтерфейси IService і класи, які використовують репозиторії; додати базову бізнес-логіку (наприклад, перевірку існування об'єкта перед оновленням чи видаленням).
- Оновити контролери: прибрати роботу зі списками List<T>; підключити сервіси через конструктор (Dependency Injection), можна перейти відразу до завдання 5, де це виконується CRUD-операції мають працювати з MongoDB.

ЗАВДАННЯ 5 Впровадити Dependency Injection (DI) для всіх шарів і налаштувати AutoMapper для автоматичного мапінгу моделей.

- Налаштувати Dependency Injection
- Додати AutoMapper

ЗАВДАННЯ 6 Додати автентифікацію та авторизацію користувачів

- Додати аутентифікацію через JWT-токен
- Створити контролер для перевірки доступу

					ЛР.ОК24.ПІ231.03.04-06	Арк.
						2
Змін.	Арк.	№ докум.	Підпис	Дата		

- Реалізувати механізм оновлення токена (Refresh Token)
- Додати авторизацію користувачів із ролями

Хід роботи:

Завдання 1

Розроблено проект за допомогою C# ASP.Net Core.

Проект реалізує CRUD операції для власного API на тему: програма для керування підписками, надано три види ресурсів:

PeopleItems – модель, яка представляє користувача з властивостями: Id – унікальний ідентифікатор, Name – ім'я користувача та Email – електронна адреса користувача,

SubscriptionItems – модель, яка представляє об'єкт підписки з властивостями: Id – унікальний ідентифікатор, OwnerId – Ідентифікатор власника підписки та Service – сервіс на якому зроблено підписку,

MessageItems – модель, яка представляє об'єкт повідомлення про підписку з властивостями: Id – унікальний ідентифікатор, Title – повідомлення, OwnerId – Ідентифікатор власника підписки та Service – сервіс на якому зроблено підписку, SubId – ідентифікатор підписки.

Лістинг:

Створено каталог Models із такими файлами:

People.cs:

```
namespace CRUDAPI.Models
```

```
{
```

```
    /// <summary>
```

```
    /// Моедль що представляє користувачів програми для управління підписками
```

```
    /// </summary>
```

```
    public class PeopleItems
```

```
    {
```

```
        /// <summary>
```

```
        /// Унікальний ідентифікатор
```

```
        /// </summary>
```

```
        public int Id { get; set; }
```

```
        /// <summary>
```

```
        /// Ім'я або логін користувача
```

```
        /// </summary>
```

```
        public string Name { get; set; }
```

```
        /// <summary>
```

```
        /// Поштова адреса користувача
```

```
        /// </summary>
```

```
        public string Email { get; set; }
```

```
    }
```

```
}
```

Subscription.cs:

					ЛР.ОК24.ПІ231.03.04-06	Арк.
						3
Змін.	Арк.	№ докум.	Підпис	Дата		

```

namespace CRUDAPI.Models
{
    /// <summary>
    /// Модель, яка описує підписку, яку може мати користувач
    /// </summary>
    public class SubscriptionItems
    {
        /// <summary>
        /// Унікальний ідентифікатор підписки
        /// </summary>
        public int Id { get; set; }
        /// <summary>
        /// Ідентифікатор власника підписки
        /// </summary>
        public int OwnerId { get; set; }
        /// <summary>
        /// Сервіс для кого виконана підписка
        /// </summary>
        public string Service { get; set; }
    }
}
Message.cs:
namespace CRUDAPI.Models
{
    /// <summary>
    /// Модель реалізує сповіщення, кі отримує користувач про підписку
    /// </summary>
    public class MessageItems
    {
        /// <summary>
        /// Унікальний ідентифікатор повідомлення
        /// </summary>
        public int Id { get; set; }
        /// <summary>
        /// Заголовок повідомлення
        /// </summary>
        public string Title { get; set; }
        /// <summary>
        /// Кому надсилається повідомлення
        /// </summary>
        public int OwnerId { get; set; }
        /// <summary>
        /// Ідентифікатор підписки
        /// </summary>
        public int SubId { get; set; }
    }
}
З цього каталогі також розміщено файл для валідації даних:
using CRUDAPI.Data;
using System.Text.RegularExpressions;
namespace CRUDAPI.Models
{

```

					ЛР.ОК24.ПІ231.03.04-06	Арк.
						4
Змін.	Арк.	№ докум.	Підпис	Дата		

```

public static class Valid
{
    const string compareField = @"^[w-]+@[w-]+\.\w+$";
    public static bool StringField(string field, int minLen, string nameField, out string err
or)
    {
        if (string.IsNullOrEmpty(field))
        {
            error = $"Текстове поле {nameField} пусте";
            return false;
        }
        else if (field.Length < minLen) {
            error = $"Занадто коротке поле {nameField}";
            return false;
        }
        error = string.Empty;
        return true;
    }
    public static bool Email(string email, out string error)
    {
        if (!Regex.IsMatch(email, compareField))
        {
            error = "Невірна поштова адреса";
            return false;
        }
        error = string.Empty;
        return true;
    }
    public static bool CheckOwner(int id)
    {
        var owner = Elements.peopleItems.FirstOrDefault(A => A.Id == id);
        if (owner is null)
            return false;
        return true;
    }
    public static bool Check_Subscribe_Owner(int Subid, int OwnerId, out string error)
    {
        var subscription = Elements.subsriptionItems.FirstOrDefault(A => A.Id == Subi
d);
        if (subscription is null)
        {
            error = "Не знайдено вказаної підписки";
            return false;
        }
        else if (subscription.OwnerId != OwnerId)
        {
            error = "Неправильний власник підписки";
            return false;
        }
        error = string.Empty;
        return true;
    }
}

```

```

    }
}

```

Створенно каталог Data, в якому містяться списки із елементів класів моделей:
using CRUDAPI.Models;

```

namespace CRUDAPI.Data

```

```

{

```

```

    public static class Elements

```

```

    {

```

```

        public static List<PeopleItems> peopleItems = new List<PeopleItems>()

```

```

        {

```

```

            new PeopleItems() {Id = 1, Name = "Kate", Email = "Kate@Email.com" },

```

```

            new PeopleItems() {Id = 2, Name = "John", Email = "John@Email.com" },

```

```

            new PeopleItems() {Id = 3, Name = "Jane", Email = "Jane@Email.com" },

```

```

            new PeopleItems() {Id = 4, Name = "Steve", Email = "Steve@Email.com" },

```

```

            new PeopleItems() {Id = 5, Name = "Alex", Email = "Alex@Email.com" }

```

```

        };

```

```

        public static List<SubscriptionItems> subsriptionItems = new List<SubscriptionItems>()

```

```

        {

```

```

            new SubscriptionItems(){Id = 1, OwnerId = 1, Service = "Netflix"},

```

```

            new SubscriptionItems(){Id = 2, OwnerId = 1, Service = "Xbox"},

```

```

            new SubscriptionItems(){Id = 3, OwnerId = 1, Service = "Amazon"},

```

```

            new SubscriptionItems(){Id = 4, OwnerId = 2, Service = "Steam"},

```

```

            new SubscriptionItems(){Id = 5, OwnerId = 2, Service = "Google"},

```

```

            new SubscriptionItems(){Id = 6, OwnerId = 2, Service = "Amazon"},

```

```

            new SubscriptionItems(){Id = 7, OwnerId = 3, Service = "YouTube"},

```

```

            new SubscriptionItems(){Id = 8, OwnerId = 3, Service = "Netflix"},

```

```

            new SubscriptionItems(){Id = 9, OwnerId = 4, Service = "YouTube"},

```

```

            new SubscriptionItems(){Id = 10, OwnerId = 4, Service = "Netflix"},

```

```

            new SubscriptionItems(){Id = 11, OwnerId = 4, Service = "Megogo"},

```

```

            new SubscriptionItems(){Id = 12, OwnerId = 4, Service = "AppleFilms"},

```

```

            new SubscriptionItems(){Id = 13, OwnerId = 5, Service = "AppleSerice"},

```

```

            new SubscriptionItems(){Id = 14, OwnerId = 5, Service = "Xbox"},

```

```

            new SubscriptionItems(){Id = 15, OwnerId = 5, Service = "AppleFilms"},

```

```

        };

```

```

        public static List<MessageItems> messageItems = new List<MessageItems>()

```

```

        {

```

```

            new MessageItems(){Id = 1,Title = "До оплати три дні", OwnerId = 1, SubId = 1

```

```

        },

```

```

            new MessageItems(){Id = 2,Title = "До оплати два дні", OwnerId = 1, SubId = 1

```

```

        },

```

```

            new MessageItems(){Id = 3,Title = "До оплати один день", OwnerId = 1, SubId

```

```

            = 1 },

```

```

            new MessageItems(){Id = 4,Title = "Підписка прострочена", OwnerId = 2, SubI

```

```

            d = 5 },

```

					ЛР.ОК24.ПІ231.03.04-06	Арк.
						6
Змін.	Арк.	№ докум.	Підпис	Дата		

```

        new MessageItems(){Id = 5,Title = "Підписку скасовано", OwnerId = 3, SubId
= 7 },
        new MessageItems(){Id = 6,Title = "Підписку просрочена", OwnerId = 4, SubI
d = 9 },
        new MessageItems(){Id = 7,Title = "Підпску продовжено", OwnerId = 4, SubId
= 9 },
        new MessageItems(){Id = 8,Title = "Підписка прострочена", OwnerId = 4, SubI
d = 11 },
        new MessageItems(){Id = 9,Title = "Підписку скасовано", OwnerId = 4, SubId
= 11 },
        new MessageItems(){Id = 10,Title = "До оплати підписки тиждень", OwnerId =
5, SubId = 15 },
    };
}
}

```

Створено каталог EndPoints із класами для реалізації endpoints моделей:

PeopleEndpoints.cs:

```

using CRUDAPI.Models;
using CRUDAPI.Data;
namespace CRUDAPI.EndPoints
{
    public static class PeopleEndpoints
    {
        ///<summary>
        ///Реєструє всі маршрути до ресурсу People
        ///</summary>
        ///<param name="app">Екземпляр<see cref="WebApplication"/>.</param>
        public static void mapPeople(this WebApplication app)
        {
            app.MapGet("/peoples/{id:int}", (int id) =>
            {
                var peopleItem = Elements.peopleItems.FirstOrDefault(A => A.Id == id);
                return peopleItem is null ? Results.NotFound() : Results.Ok(peopleItem);
            })
            .WithSummary("Отримати корисувача за Id")
            .Produces<PeopleItems>(StatusCodes.Status200OK)
            .Produces(StatusCodes.Status404NotFound)
            .WithName("GETPeople")
            .WithDescription("Отримати користувача за Id")
            .WithOpenApi();
            app.MapGet("/peoples", (string? name, string? email) =>
            {
                IEnumerable<PeopleItems> items = Elements.peopleItems;
                if (name is not null)

```

					ЛР.ОК24.ПІ231.03.04-06	Арк.
Змін.	Арк.	№ докум.	Підпис	Дата		7

```

        items = items.Where(A => A.Name == name);
        if (email is not null)
            items = items.Where(A => A.Email == email);
        return items.Count() != 0 ? Results.Ok(items) : Results.NotFound();
    })
    .WithSummary("Отримати всіх користувачів або конкретизувати за допомого
ю фільтрів")
    .Produces<PeopleItems>(StatusCodes.Status200OK)
    .Produces(StatusCodes.Status404NotFound)
    .WithName("GETPeoples")
    .WithDescription("Отримання всіх користувачів або конкретного за фільтром")
    .WithOpenApi();
app.MapPost("/peoples", (PeopleItems peopleItem) =>
{
    if (!Valid.StringField(peopleItem.Name, 5, "Name", out var error1))
        return Results.BadRequest(error1);
    if (!Valid.Email(peopleItem.Email, out var error2))
        return Results.BadRequest(error2);
    peopleItem.Id = Elements.peopleItems.Max(A => A.Id) + 1;
    Elements.peopleItems.Add(peopleItem);
    return Results.Created($"/peoples/{peopleItem.Id}", peopleItem);
})
    .WithSummary("Надіслати конкретного користувача в список")
    .Produces<PeopleItems>(StatusCodes.Status201Created)
    .Produces(StatusCodes.Status400BadRequest)
    .WithName("POSTPeople")
    .WithDescription("Відправити конкретного користувача")
    .WithOpenApi();
app.MapPut("/peoples/{id:int}", (int id, PeopleItems updatePeople) =>
{
    var people = Elements.peopleItems.Find(A => A.Id == id);
    if (people is null)
        return Results.NotFound();
    if (!Valid.StringField(updatePeople.Name, 5, "Name", out var error1))
        return Results.BadRequest(error1);
    if (!Valid.Email(updatePeople.Email, out var error2))
        return Results.BadRequest(error2);
    people.Name = updatePeople.Name;
    people.Email = updatePeople.Email;
    return Results.Ok(people);
})
    .WithSummary("Оновити повністю конкретного користувача")
    .Produces<PeopleItems>(StatusCodes.Status200OK)
    .Produces(StatusCodes.Status404NotFound)
    .Produces(StatusCodes.Status400BadRequest)
    .WithName("PUTPeople")
    .WithDescription("Оновити повністю конкретного користувача")
    .WithOpenApi();
app.MapPatch("/peoples/{id:int}", (int id, PeopleItems updatePeople) =>
{
    var people = Elements.peopleItems.Find(A => A.Id == id);
    if (people is null)
        return Results.NotFound();

```



```

        if (updatePeople.Name is not null)
        {
            if (!Valid.StringField(updatePeople.Name, 5, "Name", out var error1))
                return Results.BadRequest(error1);
            people.Name = updatePeople.Name;
        }
        if (updatePeople.Email is not null)
        {
            if (!Valid.Email(updatePeople.Email, out var error2))
                return Results.BadRequest(error2);
            people.Email = updatePeople.Email;
        }
        return Results.Ok(people);
    })
    .WithSummary("Оновити частково конкретного користувача")
    .Produces<PeopleItems>(StatusCodes.Status200OK)
    .Produces(StatusCodes.Status404NotFound)
    .Produces(StatusCodes.Status400BadRequest)
    .WithName("PATCHPeople")
    .WithDescription("Оновити частково конкретного користувача")
    .WithOpenApi();
app.MapDelete("/peoples/{id:int}", (int id) =>
{
    var people = Elements.peopleItems.FirstOrDefault(A => A.Id == id);
    if (people is null)
        return Results.NotFound();
    Elements.peopleItems.Remove(people);
    return Results.NoContent();
})
    .WithSummary("Видалити конкретного користувача")
    .Produces(StatusCodes.Status204NoContent)
    .Produces(StatusCodes.Status404NotFound)
    .WithName("DELETEPeople")
    .WithDescription("Видалити конкретного користувача")
    .WithOpenApi();
}
}
}

```

SubscriptionsEndpoints.cs:

using CRUDAPI.Models;

using CRUDAPI.Data;

namespace CRUDAPI.EndPoints

{

public static class SubscriptionsEndpoints

{

///<summary>

///Регіструє всі маршрути до ресурсу Subscriptions

///</summary>

///<param name="app">Екземпляр<see cref="WebApplication"/>.</param>

public static void mapSubscriptions(this WebApplication app)

{

app.MapGet("/subs/{id:int}", (int id) =>

					ЛР.ОК24.ПІ231.03.04-06	Арк.
						9
Змін.	Арк.	№ докум.	Підпис	Дата		

```

{
    var subItem = Elements.subscripptionItems.FirstOrDefault(A => A.Id == id);
    return subItem is null ? Results.NotFound() : Results.Ok(subItem);
})
.WithSummary("Отримати підписку")
.Produces<SubscriptionItems>(StatusCodes.Status200OK)
.Produces(StatusCodes.Status404NotFound)
.WithName("GETSub")
.WithDescription("Отримати конкретну підписку за Id")
.WithOpenApi();
app.MapGet("/subs", (int? ownerId, string? service) =>
{
    IEnumerable<SubscriptionItems> items = Elements.subscripptionItems;
    if (ownerId is not null)
        items = items.Where(A => A.OwnerId == ownerId);
    if (service is not null)
        items = items.Where(A => A.Service == service);

    return items.Count() != 0 ? Results.Ok(items) : Results.NotFound();
})
.WithSummary("Отримати всі підписки або конкретні")
.Produces<SubscriptionItems>(StatusCodes.Status200OK)
.Produces(StatusCodes.Status404NotFound)
.WithName("GETSubs")
.WithDescription("Отримати всі підписки або конкретні за допомогою філь
трів")

.WithOpenApi();
app.MapPost("/subs", (SubscriptionItems subscription) =>
{
    if (!Valid.StringField(subscription.Service, 3, "Service", out var error))
        return Results.BadRequest(error);
    if (!Valid.CheckOwner(subscription.OwnerId))
        return Results.BadRequest("Немає вказаного 'ownerId'");
    subscription.Id = Elements.subscripptionItems.Max(A => A.Id) + 1;
    Elements.subscripptionItems.Add(subscription);
    return Results.Created($"{"/subs/{subscription.Id}", subscription);
})
.WithSummary("Відправити підписку")
.Produces<SubscriptionItems>(StatusCodes.Status201Created)
.Produces(StatusCodes.Status400BadRequest)
.WithName("POSTSub")
.WithDescription("Відправити конкретну підписку")
.WithOpenApi();
app.MapPut("/subs/{id:int}", (int id, SubscriptionItems updateSub) =>
{
    var subscription = Elements.subscripptionItems.Find(A => A.Id == id);
    if (subscription is null)
        return Results.NotFound();
    if (!Valid.CheckOwner(subscription.OwnerId))
        return Results.BadRequest("Немає вказаного 'ownerId'");
    if (!Valid.StringField(updateSub.Service, 3, "Service", out var error))
        return Results.BadRequest(error);

```

```

        subscription.OwnerId = updateSub.OwnerId;
        subscription.Service = updateSub.Service;
        return Results.Ok(subscription);
    })
    .WithSummary("Оновити повністю підписку")
    .Produces<SubscriptionItems>(StatusCodes.Status200OK)
    .Produces(StatusCodes.Status404NotFound)
    .Produces(StatusCodes.Status400BadRequest)
    .WithName("PUTSub")
    .WithDescription("Оновити повністю конкретну підписку за Id")
    .WithOpenApi();

app.MapPatch("/subs/{id:int}", (int id, SubscriptionItems updateSub) =>
{
    var subscription = Elements.subsriptionItems.Find(A => A.Id == id);
    if (subscription is null)
        return Results.NotFound();
    if (!Valid.StringField(updateSub.Service, 3, "Service", out var error))
        return Results.BadRequest(error);
    if (updateSub.OwnerId != 0)
    {
        if (!Valid.CheckOwner(subscription.OwnerId))
            return Results.BadRequest("Немає вказаного 'ownerId'");
        subscription.OwnerId = updateSub.OwnerId;
    }
    if(updateSub.Service is not null)
        subscription.Service = updateSub.Service;
    return Results.Ok(subscription);
})
.WithSummary("Оновити частково підписку")
.Produces<SubscriptionItems>(StatusCodes.Status200OK)
.Produces(StatusCodes.Status404NotFound)
.Produces(StatusCodes.Status400BadRequest)
.WithName("PATCHSub")
.WithDescription("Оновити частково конкретну підписку за Id")
.WithOpenApi();
app.MapDelete("/subs/{id:int}", (int id) =>
{
    var sub = Elements.subsriptionItems.FirstOrDefault(A => A.Id == id);
    if (sub is null)
        return Results.NotFound();
    Elements.subsriptionItems.Remove(sub);
    return Results.NoContent();
})
.WithSummary("Видалити підписку")
.Produces(StatusCodes.Status204NoContent)
.Produces(StatusCodes.Status404NotFound)
.WithName("DELETESub")
.WithDescription("Видалити конкретну підписку за Id")
.WithOpenApi();
}
}

```

```

}
MessageEndpoints.cs:
using CRUDAPI.Data;
using CRUDAPI.Models;
namespace CRUDAPI.EndPoints
{
    public static class MessageEndpoints
    {
        ///<summary>
        ///Рєєстрє вєі маршрути до ресурсу Subscriptions
        ///</summary>
        ///<param name="app">Екземпляр<see cref="WebApplication"/>.</param>
        public static void mapMessages(this WebApplication app)
        {
            app.MapGet("/messages/{id:int}", (int id) =>
            {
                var mItem = Elements.messageItems.FirstOrDefault(A => A.Id == id);
                return mItem is null ? Results.NotFound() : Results.Ok(mItem);
            })
            .WithSummary("Отримати повідомлення")
            .Produces<MessageItems>(StatusCodes.Status200OK)
            .Produces(StatusCodes.Status404NotFound)
            .WithName("GETmessage")
            .WithDescription("Отримати конкретне повідомлення за Id")
            .WithOpenApi();
            app.MapGet("/messages", (int? ownerId, int? subId, string? title) =>
            {
                IEnumerable<MessageItems> items = Elements.messageItems;
                if (ownerId is not null)
                    items = items.Where(A => A.OwnerId == ownerId);
                if (subId is not null)
                    items = items.Where(A => A.SubId == subId);
                if (title is not null)
                    items = items.Where(A => A.Title == title);
                return items.Count() != 0 ? Results.Ok(items) : Results.NotFound();
            })
            .WithSummary("Отримати вєі повідомлення або конкретні")
            .Produces<MessageItems>(StatusCodes.Status200OK)
            .Produces(StatusCodes.Status404NotFound)
            .WithName("GETmessages")
            .WithDescription("Отримати вєі повідомлення або використати фїльри для
конкретизації")
            .WithOpenApi();
            app.MapPost("/messages", (MessageItems message) =>
            {
                if (!Valid.StringField(message.Title, 10, "Title", out var error))
                    return Results.BadRequest(error);
                if (!Valid.Check_Subscribe_Owner(message.SubId, message.OwnerId
, out string error1))
                    return Results.BadRequest(error1);
                message.Id = Elements.messageItems.Max(A => A.Id) + 1;
                Elements.messageItems.Add(message);
            })
        }
    }
}

```

```

        return Results.Created($"/messages/{message.Id}", message);
    })
    .WithSummary("Зафіксувати повідомлення в колекції")
    .Produces<MessageItems>(StatusCodes.Status201Created)
    .Produces(StatusCodes.Status404NotFound)
    .WithName("POSTmessage")
    .WithDescription("Відправити повідомлення в колекцію")
    .WithOpenApi();
app.MapPut("/messages/{id:int}", (int id, MessageItems updateM) =>
{
    var m = Elements.messageItems.Find(A => A.Id == id);
    if (m is null)
        return Results.NotFound();
    if (!Valid.Check_Subscribe_Owner(updateM.SubId, updateM.OwnerId, out string error1))
        return Results.BadRequest(error1);
    if (!Valid.StringField(updateM.Title, 10, "Title", out var error))
        return Results.BadRequest(error);
    m.Title = updateM.Title;
    m.OwnerId = updateM.OwnerId;
    m.SubId = updateM.SubId;
    return Results.Ok(m);
})
.WithSummary("Оновити повністю повідомлення")
.Produces<MessageItems>(StatusCodes.Status200OK)
.Produces(StatusCodes.Status404NotFound)
.Produces(StatusCodes.Status400BadRequest)
.WithName("PUTmessage")
.WithDescription("Оновити повністю повідомлення в колекції за Id")
.WithOpenApi();
app.MapPatch("/messages/{id:int}", (int id, MessageItems updateM) =>
{
    var m = Elements.messageItems.Find(A => A.Id == id);
    if (m is null)
        return Results.NotFound();
    int OId = updateM.OwnerId != 0? updateM.OwnerId : m.OwnerId;
    int SId = updateM.SubId != 0? updateM.SubId : m.SubId;
    if (!Valid.Check_Subscribe_Owner(SId, OId, out string error1))
        return Results.BadRequest(error1);
    if (updateM.OwnerId != 0)
        m.OwnerId = updateM.OwnerId;
    if(updateM.SubId != 0)
        m.SubId = updateM.SubId;
    if (updateM.Title is not null)
    {
        if (!Valid.StringField(updateM.Title, 10, "Title", out var error))
            return Results.BadRequest(error);
        m.Title = updateM.Title;
    }
    return Results.Ok(m);
})
.WithSummary("Оновити частково повідомлення")

```

```

        .Produces<MessageItems>(StatusCodes.Status200OK)
        .Produces(StatusCodes.Status404NotFound)
        .Produces(StatusCodes.Status400BadRequest)
        .WithName("PATCHmessage")
        .WithDescription("Оновити частково повідомлення в колекції за Id")
        .WithOpenApi();
app.MapDelete("/messages/{id:int}", (int id) =>
{
    var m = Elements.messageItems.FirstOrDefault(A => A.Id == id);
    if (m is null)
        return Results.NotFound();
    Elements.messageItems.Remove(m);
    return Results.NoContent();
})
    .WithSummary("Видалити повідомлення")
    .Produces(StatusCodes.Status204NoContent)
    .Produces(StatusCodes.Status404NotFound)
    .WithName("DELETEmessage")
    .WithDescription("Видалити повідомлення з колекції за Id")
    .WithOpenApi();
    }
}
}
}
}
Підключенно в program.cs всі endpoints:
using CRUDAPI.EndPoints;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    var xmlFile = $"{System.Reflection.Assembly.GetExecutingAssembly().GetName().Name}.xml";
    var xmlPath = Path.Combine(AppContext.BaseDirectory, xmlFile);
    options.IncludeXmlComments(xmlPath);
});
var app = builder.Build();
app.mapPeople();
app.mapSubscriptions();
app.mapMessages();
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.Run();
Протестовано роботу програму за допомогою swagger:
Запит1

```



Рисунок 1 – Отримання користувача за Id

Запит 2



Рисунок 2 – Отримання користувача за відсутнім Id

Запит 3

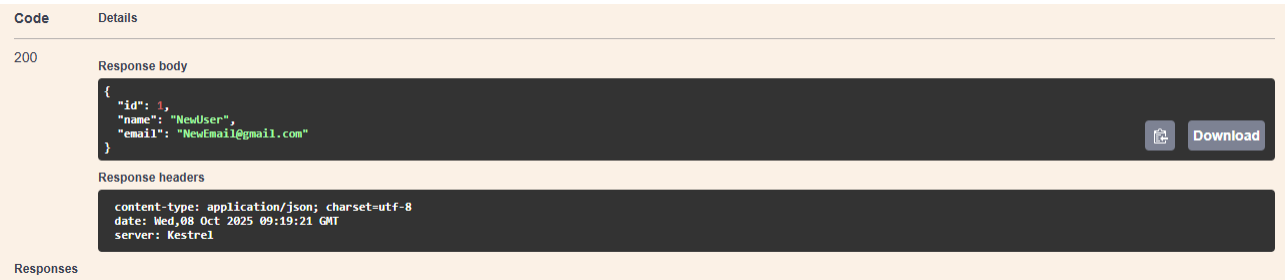


Рисунок 3 – Коректне оновлення користувача

Запит 4

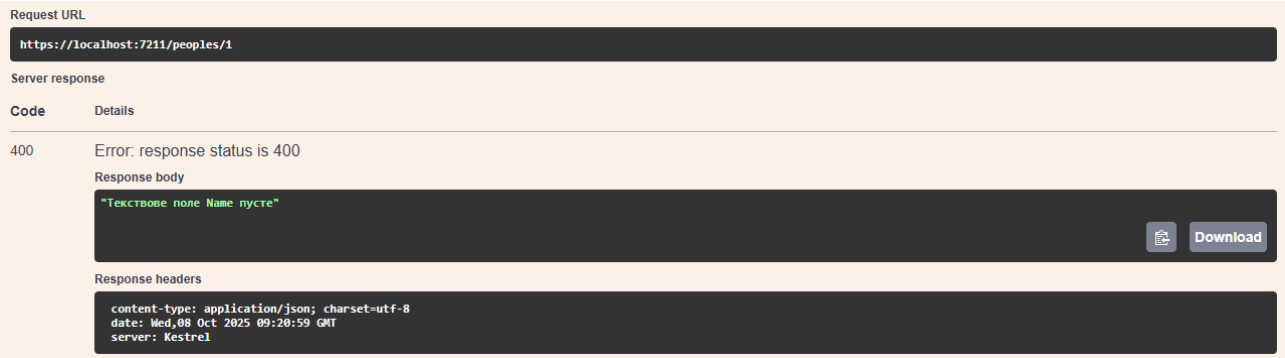


Рисунок 4 – Помилка під час оновлення

Запит 5

Request URL

https://localhost:7211/peoples/1

Server response

Code	Details
400	Error: response status is 400

Response body

"Невірна поштова адреса"

Download

Response headers

content-type: application/json; charset=utf-8
date: Wed,08 Oct 2025 09:22:11 GMT
server: Kestrel

Рисунок 5 – Помилка валідації – невірна електронна адреса

Запит 6

Request URL

https://localhost:7211/peoples/3

Server response

Code	Details
200	

Response body

{
 "id": 3,
 "name": "Jane",
 "email": "NewEmail@gmail.com"
}

Download

Response headers

content-type: application/json; charset=utf-8
date: Wed,08 Oct 2025 09:24:25 GMT
server: Kestrel

Рисунок 6 – Часткове оновлення користувача

Запит 7

Request URL

https://localhost:7211/peoples/1

Server response

Code	Details
204	

Response headers

date: Wed,08 Oct 2025 09:25:49 GMT
server: Kestrel

Рисунок 7 – Видалення користувача

Запит 8

Request URL

https://localhost:7211/peoples

Server response

Code	Details
200	

Response body

[
 {
 "id": 2,
 "name": "John",
 "email": "John@gmail.com"
 },
 {
 "id": 3,
 "name": "Jane",
 "email": "NewEmail@gmail.com"
 },
 {
 "id": 4,
 "name": "Steve",
 "email": "Steve@gmail.com"
 },
 {
 "id": 5,
 "name": "Alex",
 "email": "Alex@gmail.com"
 }
]

Download

Response headers

content-type: application/json; charset=utf-8
date: Wed,08 Oct 2025 09:26:36 GMT
server: Kestrel

Рисунок 8 – Отримано список всіх користувачів

Запит 9

					ЛР.ОК24.ПІ231.03.04-06	Арк.
						16
Змін.	Арк.	№ докум.	Підпис	Дата		

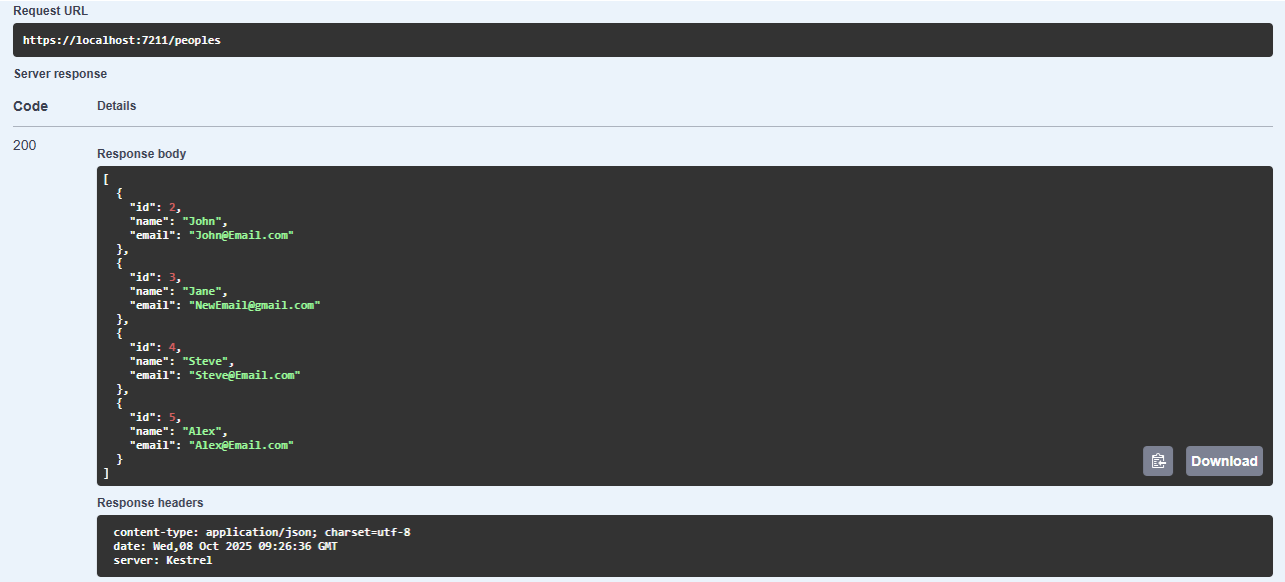


Рисунок 9 – Отримано всіх користувачів

Запит 10



Рисунок 10 – Отримано конкретного користувача

Запит 11



Рисунок 11 – Користувача не знайдено

Запит 12



Рисунок 12 – Створення нового користувача

Запит 13

Code	Details
400	Error: response status is 400 Response body "Невірна поштова адреса" Response headers content-type: application/json; charset=utf-8 date: Wed,08 Oct 2025 09:38:10 GMT server: Kestrel

Рисунок 13 – Спроба додавання користувача з невалідною адресою

Запит 14

Code	Details
200	Response body { "id": 10, "ownerId": 4, "service": "Netflix" } Response headers content-type: application/json; charset=utf-8 date: Wed,08 Oct 2025 09:40:19 GMT server: Kestrel

Рисунок 14 – Отримання конкретної підписки

Запит 15

Code	Details
404	Error: response status is 404 Response headers content-length: 0 date: Wed,08 Oct 2025 09:41:48 GMT server: Kestrel

Рисунок 15 – спроба отримати неіснуючу підписку

Запит 16

Request URL	
https://localhost:7211/subs/1	
Server response	
Code	Details
200	Response body { "id": 1, "ownerId": 1, "service": "NewService" } Response headers content-type: application/json; charset=utf-8 date: Wed,08 Oct 2025 09:47:47 GMT server: Kestrel

Рисунок 16 - Оновлення підписки

Запит 17

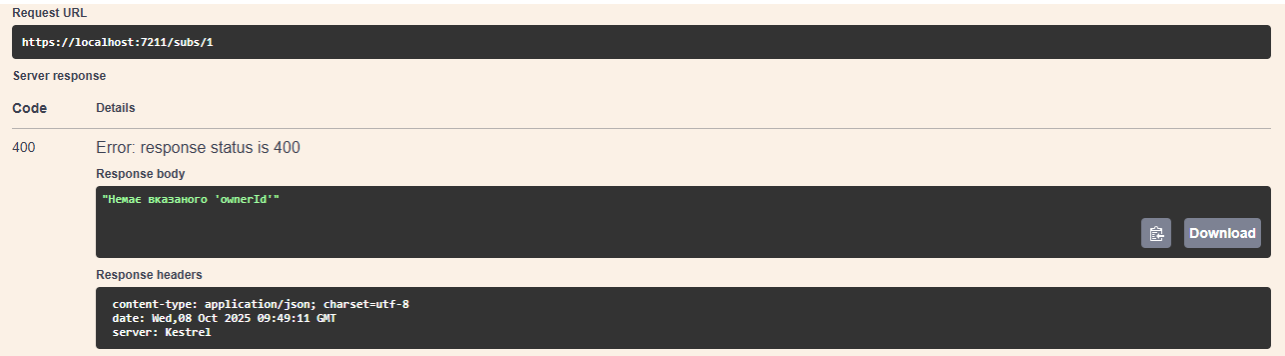


Рисунок 17 – Оновлення підписки з Id користувача, якого не існує

Запит 18

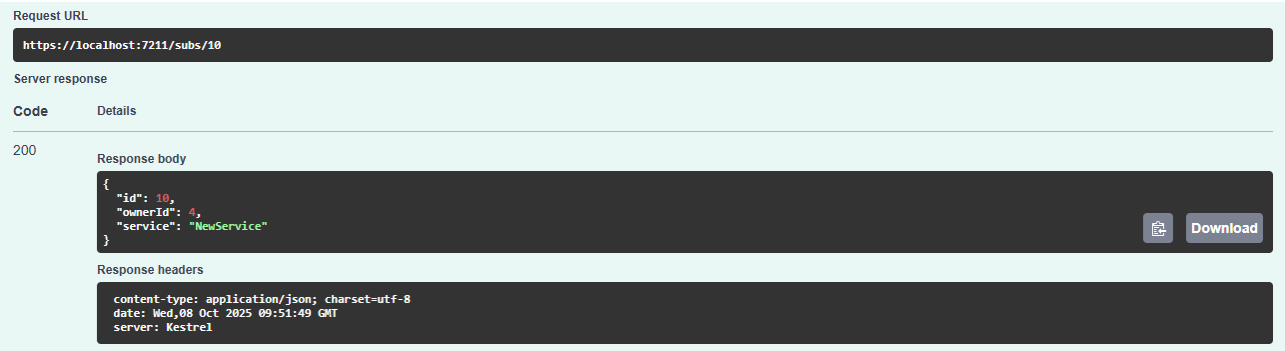


Рисунок 18 – Часткове оновлення користувача

Запит 19



Рисунок 19 – Спроба оновлення поля підписки на пусте

Запит 20



Рисунок 20 – Видалення підписки

Запит 21

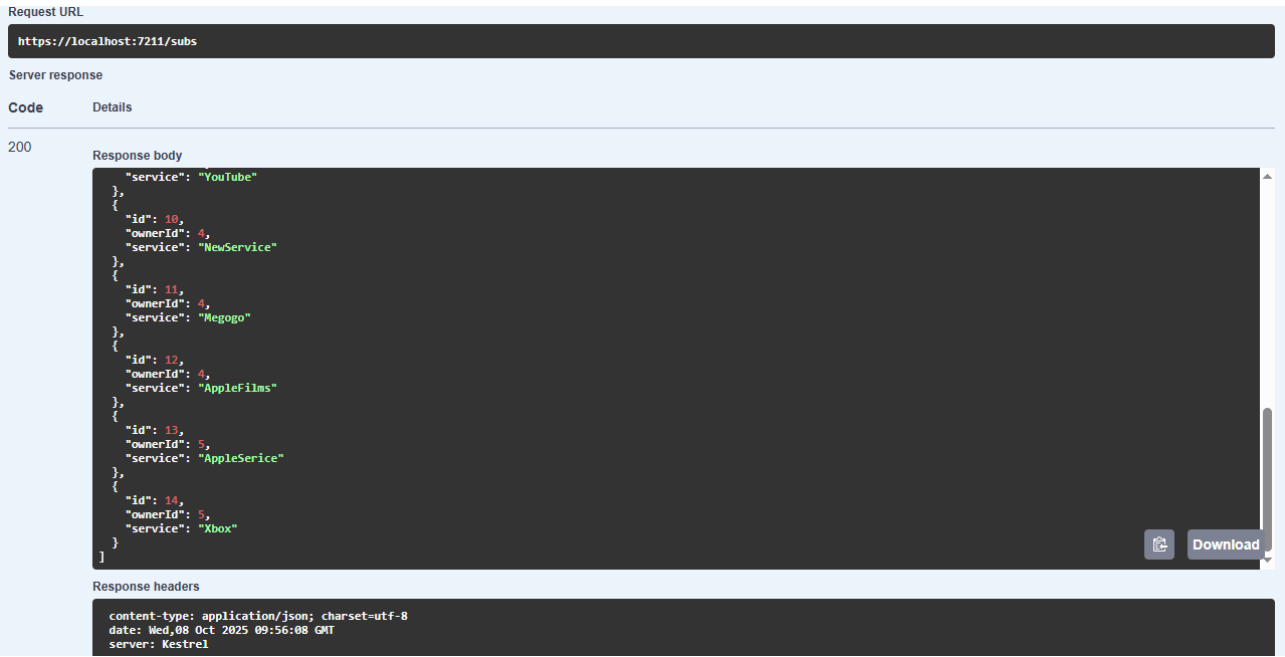


Рисунок 21 – Отримання списку всіх підписок

Запит 22



Рисунок 22 – Отримання підписки за вказаними полями

Запит 23



Рисунок 23 – Створення нової підписки

Запит 24



Рисунок 24 – Спроба додавання нової підписки з неіснуючм користувачем

Запит 25



Рисунок 25 – Спроба створення підписки з невалідним сервісом

Запит 26



Рисунок 26 – Отримання конкретного повідомлення

Запит 27



Рисунок 27 – Спроба отримання неіснуючої підписки

Запит 28

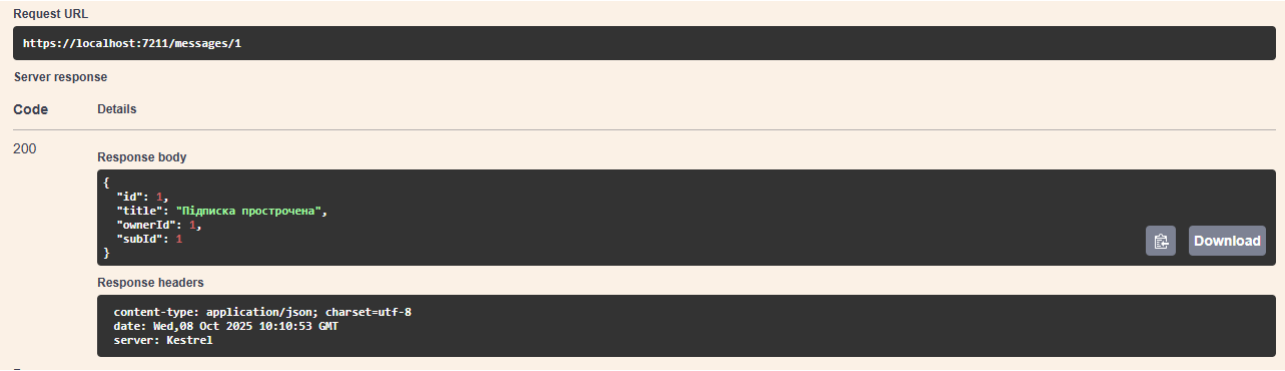


Рисунок 28 – Оновлення вмісту повідомлення

Запит 29

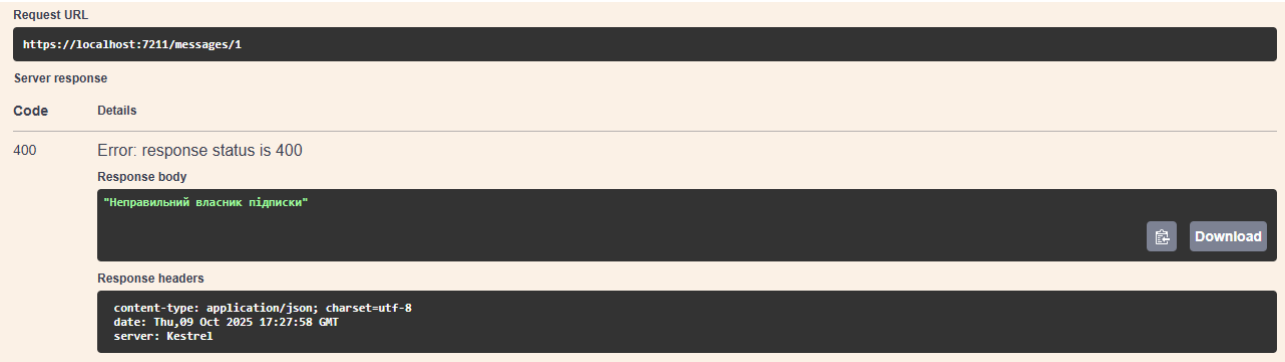


Рисунок 29 – Спроба відправит підписнику нагадування не про те повідомлення

Запит 30



Рисунок 30 – Оновлення підписки в користувача

Запит 31



Рисунок 31 – Спроба оновлення невірної підписки користувача

Запит 32

Request URL	
https://localhost:7211/messages/10	
Server response	
Code	Details
204	Response headers
date: Thu,09 Oct 2025 17:39:59 GMT server: Kestrel	
Responses	

Рисунок 32 – Видалення повідомленн про підписку

Запит 33

Request URL	
https://localhost:7211/messages	
Server response	
Code	Details
200	Response body
<pre>{ "ownerId": 3, "subId": 7 }, { "id": 6, "title": "Підписку просрочена", "ownerId": 4, "subId": 9 }, { "id": 7, "title": "Підписку продовжено", "ownerId": 4, "subId": 9 }, { "id": 8, "title": "Підписка прострочена", "ownerId": 4, "subId": 11 }, { "id": 9, "title": "Підписку скасовано", "ownerId": 4, "subId": 11 } }</pre>	
Response headers	
content-type: application/json; charset=utf-8 date: Thu,09 Oct 2025 17:40:49 GMT server: Kestrel	

Рисунок 33 -Отримання списку усіх повідомлень

Запит 34

Request URL	
https://localhost:7211/messages?ownerId=1	
Server response	
Code	Details
200	Response body
<pre>{ "id": 2, "title": "До оплати два дні", "ownerId": 1, "subId": 1 }, { "id": 3, "title": "До оплати один день", "ownerId": 1, "subId": 1 } }</pre>	
Response headers	
content-type: application/json; charset=utf-8 date: Thu,09 Oct 2025 17:43:09 GMT server: Kestrel	

Рисунок 34 – Отримання всіх повідомлень до користувача

Запит 35

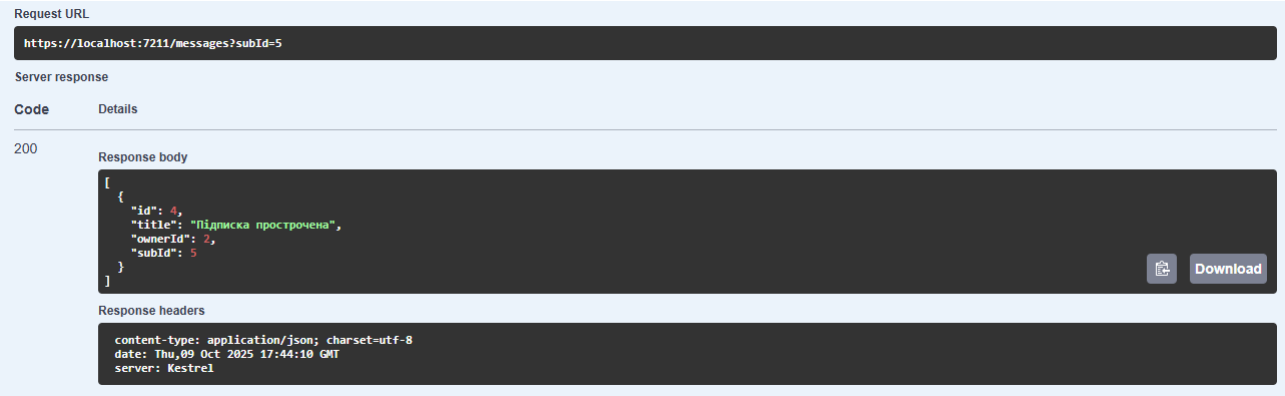


Рисунок 35 – Отримання всіх повідомлень про конкретну підписку

Запит 36



Рисунок 36 – Спроба відправити пусте повідомлення

Запит 37



Рисунок 37 – Додано повідомлення для користувача та його підписки

Запит 38

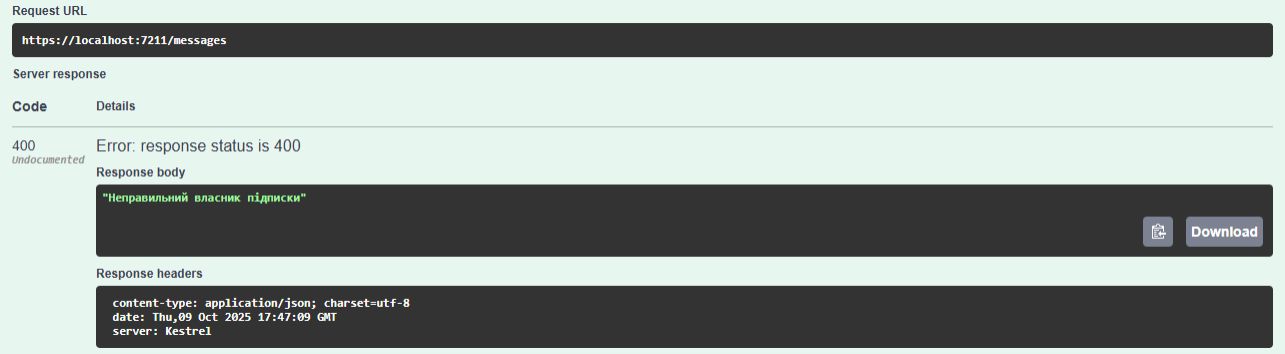


Рисунок 38 – Спроба відправити повідомлення користувачу не про його підписку

Завдання 2

Розроблено проект за допомогою C# ASP.Net Core WebAPI.

Проект реалізує CRUD операції для власного API на тему: програма для керування підписками, надано три види ресурсів:

PeopleItems – модель, яка представляє користувача з властивостями: Id – унікальний ідентифікатор, Name – ім'я користувача та Email – електронна адреса користувача,

SubscriptionItems – модель, яка представляє об'єкт підписки з властивостями: Id – унікальний ідентифікатор, OwnerId – Ідентифікатор власника підписки та Service – сервіс на якому зроблено підписку, Status – перелік, для відображення станів підписки,

MessageItems – модель, яка представляє об'єкт повідомлення про підписку з властивостями: Id – унікальний ідентифікатор, Title – повідомлення, OwnerId – Ідентифікатор власника підписки та Service – сервіс на якому зроблено підписку, SubId – ідентифікатор підписки.

Реалізовано валідацію в класові PeopleItems за допомогою Data.Anotation, для класів SubscriptionItems, MessageItems за допомогою FluentValidator та винесено логіку в окремий каталог Validator.

Лістинг:

People.cs:

```
using System.ComponentModel.DataAnnotations;
```

```
namespace LW_4_2.Models
```

```
{
```

```
    /// <summary>
```

```
    /// Моедль що представляє користувачів програми для управління підписками
```

```
    /// </summary>
```

```
    public class PeopleItems
```

```
    {
```

```
        /// <summary>
```

```
        /// Унікальний ідентифікатор
```

```
        /// </summary>
```

```
        public int Id { get; set; }
```

```
        /// <summary>
```

```
        /// Ім'я або логін користувача
```

```
        /// </summary>
```

```
        [Required(ErrorMessage = "Поле 'Name' Обов'язкове для заповнення")]
```

					ЛР.ОК24.ПІ231.03.04-06	Арк.
						25
Змін.	Арк.	№ докум.	Підпис	Дата		

```
[RegularExpression(@"^[A-Z][a-z]{5,10}[0-9]{0,5}$", ErrorMessage = "'Name' не відповідає вимогам:" +
    "Перші велика англійська літера, далі від 5 до 10 малих англійських символі
в та цифри за бажанням")]
```

```
public string? Name { get; set; }
```

```
/// <summary>
```

```
/// Поштова адреса користувача
```

```
/// </summary>
```

```
[Required(ErrorMessage = "Поле 'Email' є обов'язковим")]
```

```
[EmailAddress(ErrorMessage = "Не коректне значення 'Email'")]
```

```
public string? Email { get; set; }
```

```
}
```

```
}
```

```
Subscription.cs:
```

```
namespace LW_4_2.Models
```

```
{
```

```
/// <summary>
```

```
/// Перелік статусів підписки
```

```
/// </summary>
```

```
public enum SubStatus
```

```
{
```

```
/// <summary>
```

```
/// Статус очікування - Expectation - 0
```

```
/// </summary>
```

```
Expectation,
```

```
/// <summary>
```

```
/// Статус активна - Active - 1
```

```
/// </summary>
```

```
Active,
```

```
/// <summary>
```

```
/// Статус прострочена - Overdue - 2
```

```
/// </summary>
```

```
Overdue,
```

```
/// <summary>
```

```
/// Статус Архівована - Archived - 3
```

					ЛР.ОК24.ПІ231.03.04-06	Арк.
Змін.	Арк.	№ докум.	Підпис	Дата		26

```

        /// </summary>
        Archived
    }

    /// <summary>
    /// Модель, яка описує підписку, яку може мати користувач
    /// </summary>
    public class SubscriptionItems
    {
        /// <summary>
        /// Унікальний ідентифікатор підписки
        /// </summary>
        public int? Id { get; set; }

        /// <summary>
        /// Ідентифікатор власника підписки
        /// </summary>
        public int? OwnerId { get; set; }

        /// <summary>
        /// Сервіс для якого виконана підписка
        /// </summary>
        public string? Service { get; set; }

        /// <summary>
        /// Статус в якому перебуває підписка користувача
        /// </summary>
        public SubStatus? Status { get; set; }
    }
}

Message.cs:
namespace LW_4_2.Models
{
    /// <summary>
    /// Модель реалізує сповіщення, кі отримує користувач про підписку
    /// </summary>
    public class MessageItems
    {
        /// <summary>

```

					ЛР.ОК24.ПІ231.03.04-06	Арк.
Змін.	Арк.	№ докум.	Підпис	Дата		27

```

        /// Унікальний ідентифікатор повідомлення
        /// </summary>
        public int Id { get; set; }
        /// <summary>
        /// Заголовок повідомлення
        /// </summary>
        public string? Title { get; set; }
        /// <summary>
        /// Кому надсилається повідомлення
        /// </summary>
        public int? OwnerId { get; set; }
        /// <summary>
        /// Ідентифікатор підписки
        /// </summary>
        public int? SubId { get; set; }
    }
}

PeopleData.cs:
using LW_4_2.Models;
namespace LW_4_2.Data
{
    public static class PeopleData
    {
        public static List<PeopleItems> peopleItems = new List<PeopleItems>()
        {
            new PeopleItems() { Id = 1, Name = "Kate", Email = "Kate@Email.com" },
            new PeopleItems() { Id = 2, Name = "John", Email = "John@Email.com" },
            new PeopleItems() { Id = 3, Name = "Jane", Email = "Jane@Email.com" },
            new PeopleItems() { Id = 4, Name = "Steve", Email = "Steve@Email.com" },
            new PeopleItems() { Id = 5, Name = "Alex", Email = "Alex@Email.com" }
        };
    }
}

SubData.cs:
using LW_4_2.Models;

```

```

namespace LW_4_2.Data
{
    public static class SubData
    {
        public static List<SubscriptionItems> subsripptionItems = new List<SubscriptionItems>()
        {
            new SubscriptionItems() { Id = 1, OwnerId = 1, Service = "Netflix", Status = SubStatus.Active },
            new SubscriptionItems() { Id = 2, OwnerId = 1, Service = "Xbox", Status = SubStatus.Active },
            new SubscriptionItems() { Id = 3, OwnerId = 1, Service = "Amazon", Status = SubStatus.Active },
            new SubscriptionItems() { Id = 4, OwnerId = 2, Service = "Steam", Status = SubStatus.Active },
            new SubscriptionItems() { Id = 5, OwnerId = 2, Service = "Google", Status = SubStatus.Overdue },
            new SubscriptionItems() { Id = 6, OwnerId = 2, Service = "Amazon", Status = SubStatus.Active },
            new SubscriptionItems() { Id = 7, OwnerId = 3, Service = "YouTube", Status = SubStatus.Archived },
            new SubscriptionItems() { Id = 8, OwnerId = 3, Service = "Netflix", Status = SubStatus.Active },
            new SubscriptionItems() { Id = 9, OwnerId = 4, Service = "YouTube", Status = SubStatus.Active },
            new SubscriptionItems() { Id = 10, OwnerId = 4, Service = "Netflix", Status = SubStatus.Active },
            new SubscriptionItems() { Id = 11, OwnerId = 4, Service = "Megogo", Status = SubStatus.Archived },
            new SubscriptionItems() { Id = 12, OwnerId = 4, Service = "AppleFilms", Status = SubStatus.Active },
            new SubscriptionItems() { Id = 13, OwnerId = 5, Service = "AppleService", Status = SubStatus.Active },
        }
    }
}

```

```

        new SubscriptionItems() { Id = 14, OwnerId = 5, Service = "Xbox", Status =
SubStatus.Active},
        new SubscriptionItems() { Id = 15, OwnerId = 5, Service = "AppleFilms", Stat
us = SubStatus.Active},
    };
}
}

```

MessageData.cs:

```

using LW_4_2.Models;
namespace LW_4_2.Data
{
    public static class MessageData
    {
        public static List<MessageItems> messageItems = new List<MessageItems>()
        {
            new MessageItems() { Id = 1, Title = "До оплати три дні", OwnerId = 1, SubId = 1
},
            new MessageItems() { Id = 2, Title = "До оплати два дні", OwnerId = 1, SubId = 1
},
            new MessageItems() { Id = 3, Title = "До оплати один день", OwnerId = 1, SubId
= 1 },
            new MessageItems() { Id = 4, Title = "Підписка прострочена", OwnerId = 2, SubI
d = 5 },
            new MessageItems() { Id = 5, Title = "Підписку скасовано", OwnerId = 3, SubId
= 7 },
            new MessageItems() { Id = 6, Title = "Підписку просрочена", OwnerId = 4, SubId
= 9 },
            new MessageItems() { Id = 7, Title = "Підпску продовжено", OwnerId = 4, SubId
= 9 },
            new MessageItems() { Id = 8, Title = "Підписка прострочена", OwnerId = 4, SubI
d = 11 },
            new MessageItems() { Id = 9, Title = "Підписку скасовано", OwnerId = 4, SubId
= 11 },

```

					ЛР.ОК24.ПІ231.03.04-06	Арк.
						30
Змін.	Арк.	№ докум.	Підпис	Дата		

```

        new MessageItems() { Id = 10, Title = "До оплати підписки тиждень", OwnerId =
5, SubId = 15 },
        };
    }
}
MessageValidator.cs:
using FluentValidation;
using LW_4_2.Models;
using LW_4_2.Data;
namespace LW_4_2.Validator
{
    public class MessageValidator: AbstractValidator<MessageItems>
    {
        public MessageValidator()
        {
            RuleFor(x => x.Title)
                .NotEmpty().WithMessage("Поле 'Title' не може бути порожнім")
                .Length(5, 100).WithMessage("Поле 'Title' має бути від трьох до ста символі
в");

            RuleFor(x => x.SubId)
                .NotNull().WithMessage("Поле 'SubId' не може бути порожнім")
                .Must(SubId => SubData.subscripptionItems.Any(s => s.Id == SubId))
                .WithMessage("Не знайдено 'SubId' із таким індексом");

            RuleFor(x => x.OwnerId)
                .NotEmpty().WithMessage("Поле 'OwnerId' не може бути порожнім")
                .Must(ownerId => PeopleData.peopleItems.Any(u => u.Id == ownerId))
                .WithMessage("Немає людини із таким значенням 'Id'")
                .Must((mes, ownerId) =>
                {
                    var sub = SubData.subscripptionItems.FirstOrDefault(s => s.Id == mes.SubId)

;

                    if (sub is null)
                        return false;

                    return sub.OwnerId == ownerId;
                });
        }
    }
}

```

```

        }).WithMessage("Відсутня підписка за вказаним 'SubId' або для неї не існує користувача");
    }
}

SubscriptionValidtor.cs:
using LW_4_2.Models;
using LW_4_2.Data;
using FluentValidation;
namespace LW_4_2.Validator
{
    public class SubscriptionValidator: AbstractValidator<SubscriptionItems>
    {
        public SubscriptionValidator()
        {
            RuleFor(x => x.OwnerId)
                .NotEmpty().WithMessage("Поле 'OwnerId' не може бути порожнім")
                .Must(ownerID => SubData.subscripptionItems.Any(s => s.OwnerId == ownerID))
                .WithMessage("Не знайдено користувача за вказаним 'OwnerId'");

            RuleFor(x => x.Service)
                .NotEmpty().WithMessage("Поле 'Service' не може бути порожнім")
                .Length(3, 50).WithMessage("Поле 'Service' має бути в межах від 3 до 50 символів");

            RuleFor(x => x.Status)
                .NotEmpty().WithMessage("Поле 'Status' не можу бути порожнім")
                .IsInEnum().WithMessage("Поле 'Status' має бути вибране в заданих межах");
        }
    }
}

PeopleController.cs:
using LW_4_2.Data;

```



```

using LW_4_2.Models;
using Microsoft.AspNetCore.Mvc;
using System.Text.Json;
namespace LW_4_2.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class PeopleController : ControllerBase
    {
        [HttpGet]
        public ActionResult<IEnumerable<PeopleItems>> GetAll(
            [FromQuery] string? Name, [FromQuery] string? Email)
        {
            IEnumerable<PeopleItems> peoples = PeopleData.peopleItems;
            if(Name is not null)
                peoples = peoples.Where(x => x.Name == Name);
            if(Email is not null)
                peoples = peoples.Where(x => x.Email == Email);
            if (!peoples.Any())
                return NotFound("Не знайдено елементів");
            return Ok(peoples);
        }
        [HttpGet("{id}")]
        public ActionResult<PeopleItems> GetById(int id)
        {
            var people = PeopleData.peopleItems.FirstOrDefault(x => x.Id == id);
            if (people is null)
                return NotFound($"Не знайдено людину із вказаним 'Id': {id}");
            return Ok(people);
        }
        [HttpPost]
        public ActionResult<PeopleItems> Create(PeopleItems item)
        {
            item.Id = PeopleData.peopleItems.Max(x => x.Id) + 1;
            PeopleData.peopleItems.Add(item);
        }
    }
}

```

```

        return CreatedAtAction(nameof(GetById), new { Id = item.Id }, item);
    }
    [HttpPut("{id}")]
    public IActionResult Update(int id, PeopleItems upPeople)
    {
        var people = PeopleData.peopleItems.FirstOrDefault(p => p.Id == id);
        if (people is null)
            return NotFound($"Не знайдено людини із Id {id}");
        people.Name = upPeople.Name;
        people.Email = upPeople.Email;
        return NoContent();
    }
    [HttpPatch("{id}")]
    public IActionResult UpdatePart(int id, [FromBody] JsonElement upPeople)
    {
        ModelState.ClearValidationState(nameof(upPeople));
        var people = PeopleData.peopleItems.FirstOrDefault(x => x.Id == id);
        if(people is null)
            return NotFound($"Не знайдено людини за вказаним Id {id}");
        var temp = new PeopleItems
        {
            Id = id,
            Email = people.Email,
            Name = people.Name
        }
        if(upPeople.TryGetProperty("name",out var name))
            temp.Name = name.GetString();
        if(upPeople.TryGetProperty("email",out var email))
            temp.Email = email.GetString();
        if (!TryValidateModel(temp))
            return BadRequest(ModelState);
        people.Name = temp.Name;
        people.Email = temp.Email;
        return NoContent();
    }
}

```

```

[HttpDelete("{id}")]
public IActionResult DeletePart(int id)
{
    var people = PeopleData.peopleItems.FirstOrDefault(p => p.Id == id);
    if (people is null)
        return NotFound($"Не знайдено людину із вказаним Id {id}");
    PeopleData.peopleItems.Remove(people);
    return NoContent();
}
}
}

```

SubController.cs:

```

using LW_4_2.Data;
using LW_4_2.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.ApplicationParts;
using System.Text.Json;
namespace LW_4_2.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class SubController : ControllerBase
    {
        [HttpGet]
        public ActionResult<IEnumerable<SubscriptionItems>> GetAll(
            [FromQuery] int? ownerId, [FromQuery] string? service, [FromQuery] SubStatus
? subStatus)
        {
            IEnumerable<SubscriptionItems> subItems = SubData.subscripptionItems;
            if (ownerId is not null)
                subItems = subItems.Where(x => x.OwnerId == ownerId);
            if (service is not null)
                subItems = subItems.Where(x => x.Service == service);
            if (subStatus is not null)
                subItems = subItems.Where(x => x.Status == subStatus);

```

```

        if (subItems.Count() == 0)
            return NotFound("Не знайдено елементів за запитом");
        return Ok(subItems);
    }
    [HttpGet("{id}")]
    public ActionResult<SubscriptionItems> GetById(int id)
    {
        var sub = SubData.subscripptionItems.FirstOrDefault(x => x.Id == id);
        if (sub is null)
            return NotFound($"Не знайдено підписки з Id {id}");
        return Ok(sub);
    }
    [HttpPost]
    public ActionResult<SubscriptionItems> Create(SubscriptionItems item)
    {
        item.Id = SubData.subscripptionItems.Max(x => x.Id) + 1;
        SubData.subscripptionItems.Add(item);
        return CreatedAtAction(nameof(GetById), new { Id = item.Id }, item);
    }
    [HttpPut("{id}")]
    public IActionResult Update(int id, SubscriptionItems item)
    {
        var sub = SubData.subscripptionItems.FirstOrDefault(x => x.Id == id);
        if (sub is null)
            return NotFound($"Не знайдено підписки з Id {id}");
        sub.OwnerId = item.OwnerId;
        sub.Service = item.Service;
        sub.Status = item.Status;
        return NoContent();
    }
    [HttpPatch("{id}")]
    public IActionResult UpdatePart(int id, JsonElement item)
    {
        var sub = SubData.subscripptionItems.FirstOrDefault(x => x.Id == id);
        if (sub is null)

```

```

        return NotFound($"Не знайдено підписки з Id {id}");
var temp = new SubscriptionItems
{
    Id = sub.Id,
    OwnerId = sub.OwnerId,
    Service = sub.Service,
    Status = sub.Status
};
if(item.TryGetProperty("ownerId",out var upId))
    temp.OwnerId = upId.GetInt32();
if (item.TryGetProperty("status",out var upStatus))
    temp.Status = (SubStatus)upStatus.GetInt32();
if (item.TryGetProperty("service",out var upServ))
    temp.Service = upServ.GetString();
TryValidateModel(temp);
if (!ModelState.IsValid)
    return BadRequest(ModelState);
sub.OwnerId = temp.OwnerId;
sub.Service = temp.Service;
sub.Status = temp.Status;
return NoContent();
}
[HttpDelete("{id}")]
public IActionResult Delete(int id)
{
    var sub = SubData.subsripptionItems.FirstOrDefault(x => x.Id == id);
    if (sub is null)
        return NotFound($"Не знайдено підписки з Id {id}");
    SubData.subsripptionItems.Remove(sub);
    return NoContent();
}
}

MessageController.cs:
using LW_4_2.Data;

```

```

using LW_4_2.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.ModelBinding;
using System.Text.Json;
namespace LW_4_2.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class MessageController: ControllerBase
    {
        [HttpGet]
        public ActionResult<IEnumerable<MessageItems>> GetAll(
            [FromQuery] int? ownerId,
            [FromQuery] int? subId,
            [FromQuery] string? title)
        {
            IEnumerable<MessageItems> messages = MessageData.messageItems;
            if(ownerId is not null)
                messages = messages.Where(x => x.OwnerId == ownerId);
            if (subId is not null)
                messages = messages.Where(x => x.SubId == subId);
            if (title is not null)
                messages = messages.Where(x => x.Title == title);
            if (!messages.Any())
                return NotFound("Не знайдено елементів за вказаним запитом");
            return Ok(messages);
        }
        [HttpGet("{id}")]
        public ActionResult<MessageItems> GetById(int id)
        {
            var message = MessageData.messageItems.FirstOrDefault(x => x.Id == id);
            if (message is null)
                return NotFound($"Повідомлення із вказаним Id {id} не знайдено");
            return Ok(message);
        }
    }
}

```

```

[HttpPost]
public ActionResult<MessageItems> Create(MessageItems message)
{
    message.Id = MessageData.messageItems.Max(x => x.Id) + 1;
    MessageData.messageItems.Add(message);
    return CreatedAtAction(nameof(GetById), new { Id = message.Id }, message);
}

[HttpPut("{id}")]
public IActionResult Update(int id, MessageItems message)
{
    var mes = MessageData.messageItems.FirstOrDefault(x => x.Id == id);
    if (mes is null)
        return NotFound($"Повідомлення із вказаним Id {id} не знайдено");
    mes.OwnerId = message.OwnerId;
    mes.SubId = message.SubId;
    mes.Title = message.Title;
    return NoContent();
}

[HttpPatch("{id}")]
public IActionResult UpdatePart(int id, JsonElement message)
{
    var mes = MessageData.messageItems.FirstOrDefault(x => x.Id == id);
    if (mes is null)
        return NotFound($"Повідомлення із вказаним Id {id} не знайдено");
    var temp = new MessageItems()
    {
        Id = id,
        OwnerId = mes.OwnerId,
        SubId = mes.SubId,
        Title = mes.Title,
    };
    if(message.TryGetProperty("ownerId",out var upO))
        temp.OwnerId = upO.GetInt32();
    if(message.TryGetProperty("subId", out var upS))
        temp.SubId = upS.GetInt32();
}

```

```

        if (message.TryGetProperty("title", out var upT))
            temp.Title = upT.GetString();
        TryValidateModel(temp);
        if (!ModelState.IsValid)
            return BadRequest(ModelState);
        mes.OwnerId = temp.OwnerId;
        mes.SubId = temp.SubId;
        mes.Title = temp.Title;
        return NoContent();
    }
    [HttpDelete("{id}")]
    public IActionResult Delete(int id)
    {
        var mes = MessageData.messageItems.FirstOrDefault(x => x.Id == id);
        if (mes is null)
            return NotFound($"Повідомлення із вказаним Id {id} не знайдено");
        MessageData.messageItems.Remove(mes);
        return NoContent();
    }
}

```

Program.cs:

```

using FluentValidation;
using FluentValidation.AspNetCore;
using LW_4_2.Models;
using LW_4_2.Validator;
var builder = WebApplication.CreateBuilder(args);
// Add services to the container.
builder.Services.AddControllers().AddFluentValidation();
// Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbu
ckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(oprions =>
{

```



```
var xmlFile = $"{System.Reflection.Assembly.GetExecutingAssembly().GetName().Name}.xml";

var xmlPath = Path.Combine(AppContext.BaseDirectory, xmlFile);

options.IncludeXmlComments(xmlPath);

});

builder.Services.AddValidatorsFromAssemblyContaining<SubscriptionValidator>();
builder.Services.AddValidatorsFromAssemblyContaining<MessageValidator>();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseAuthorization();

app.MapControllers();

app.Run();
```

Протестовано роботу програми за допомогою swagger:

Запит 1

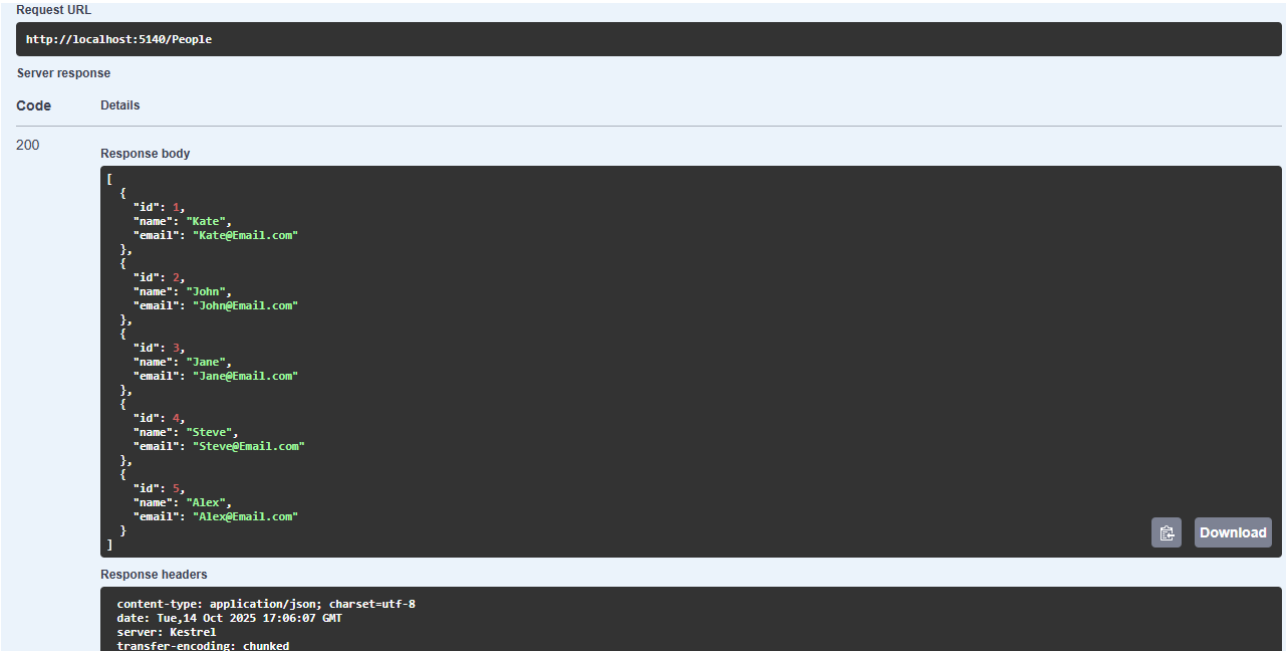


Рисунок 39 – Отримання всіх користувачів

Запит 2

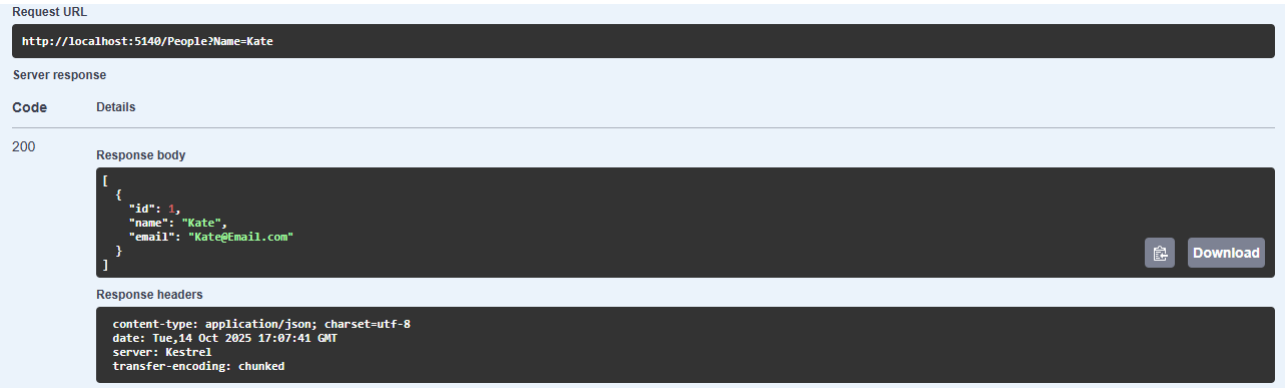


Рисунок 40 – Отримання користувачів за одним параметром

Запит 3



Рисунок 41 -Отримання користувачів за двома параметрами

Запит 4



Рисунок 42 -Спроба отримати користувачів за хибним параметром запити

Запит 5



Рисунок 43 – Отримання користувача за Id

Запит 6

Request URL

http://localhost:5140/People/10

Server response

Code

Details

404

Undocumented

Error: Not Found

Response body

Не знайдено людину із вказаним "Id": 10

Download

Response headers

content-type: text/plain; charset=utf-8
date: Tue,14 Oct 2025 17:12:33 GMT
server: Kestrel
transfer-encoding: chunked

Рисунок 44 – Спроба отримати неіснуючого користувача

Запит 7

Request URL

http://localhost:5140/People

Server response

Code

Details

201

Undocumented

Response body

{
 "id": 6,
 "name": "Timcat1",
 "email": "Tim@Email.com"
}

Download

Response headers

content-type: application/json; charset=utf-8
date: Tue,14 Oct 2025 17:14:53 GMT
location: http://localhost:5140/People/6
server: Kestrel
transfer-encoding: chunked

Responses

Рисунок 45 – Додавання нового користувача

Запит 8

Request URL

http://localhost:5140/People

Server response

Code

Details

400

Undocumented

Error: Bad Request

Response body

{
 "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
 "title": "One or more validation errors occurred.",
 "status": 400,
 "errors": {
 "Name": [
 "'Name' не відповідає вимогам:Перші велика англійська літера, далі від 5 до 10 малих англійських символів та цифри за бажанням"
]
 },
 "traceId": "00-8e01a42057ea453b4f73a55f752a3a90-366f7e4d94013a79-00"
}

Download

Response headers

content-type: application/problem+json; charset=utf-8
date: Tue,14 Oct 2025 17:16:16 GMT
server: Kestrel
transfer-encoding: chunked

Рисунок 46 – Спроба додавання користуваа із некоректним логінок

Запит 9



Рисунок 47 – Спроа додавання користувача із некоректними полями

Запит 10



Рисунок 48 – Спроба відправлення пустого поля

Запит 11



Рисунок 49 – Оновлення інформації про першого користувача

Запит 12



Рисунок 50 – Спроба оновлення з використання некоректного логіну

Запит 13



Рисунок 51 – Спроба оновленн на пусті поля

Запит 14



Рисунок 52 – Часткове оновлення користувача

Запит 15



Рисунок 53 – Спроба оновлення на пусте поле

Запит 16



Рисунок 54 – Видалення користувача

Запит 17

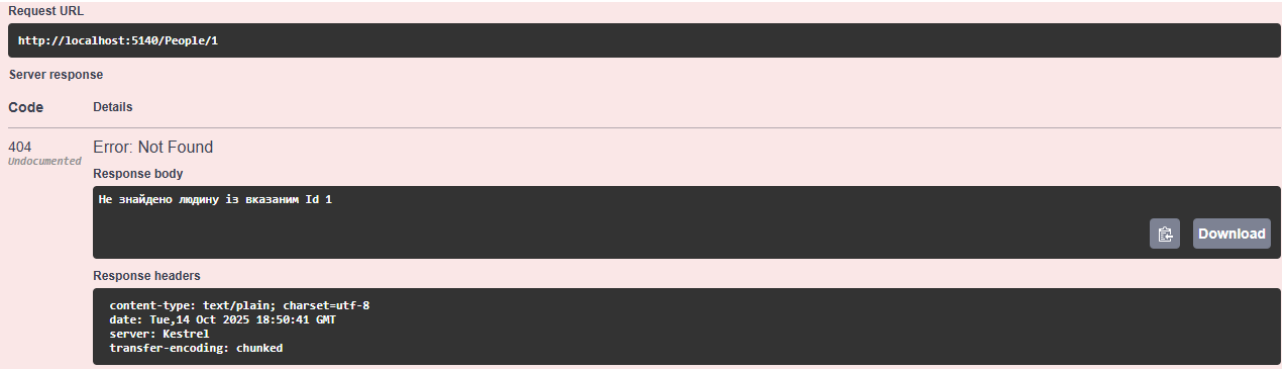


Рисунок 55 – Спроба видалення користувача

Запит 18

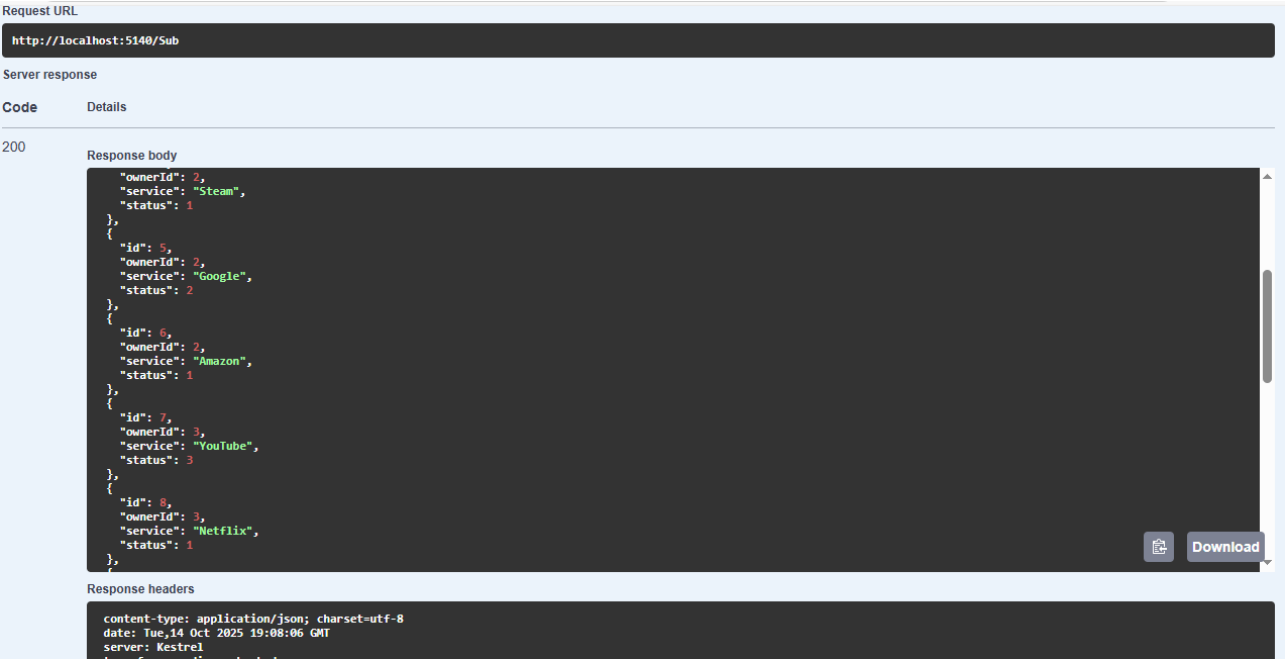


Рисунок 56 – Отримано список всіх підписок

Запит 19

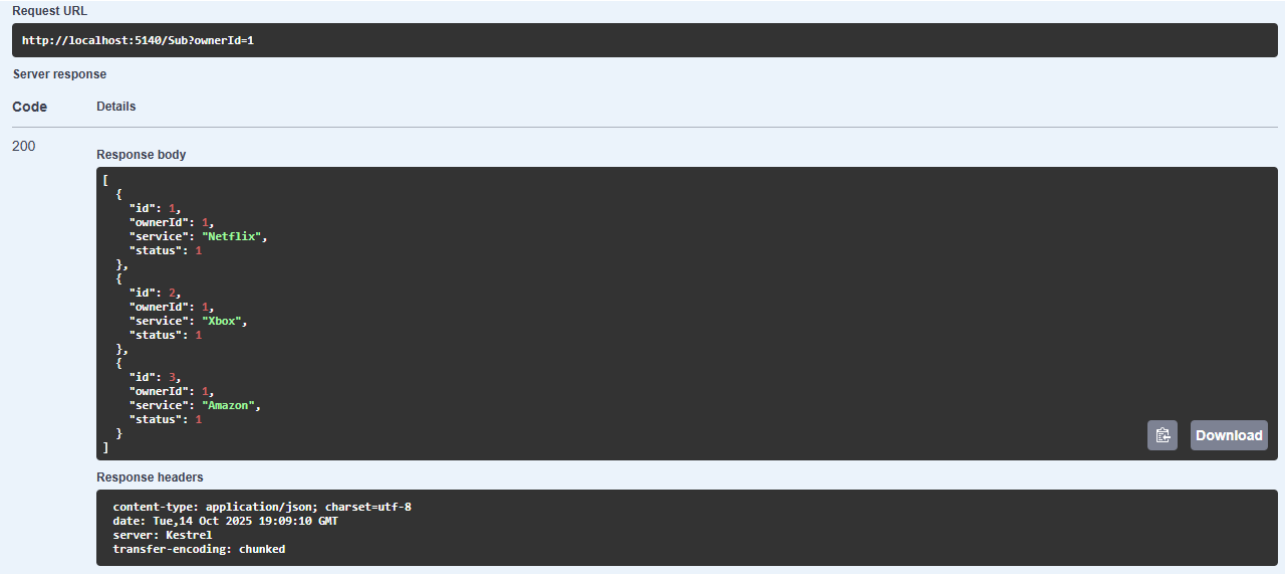


Рисунок 57 – Отримання підписок з використанням параметра

Запит 20



Рисунок 58 – Отримання підписок за деількома параметрами

Запит 21

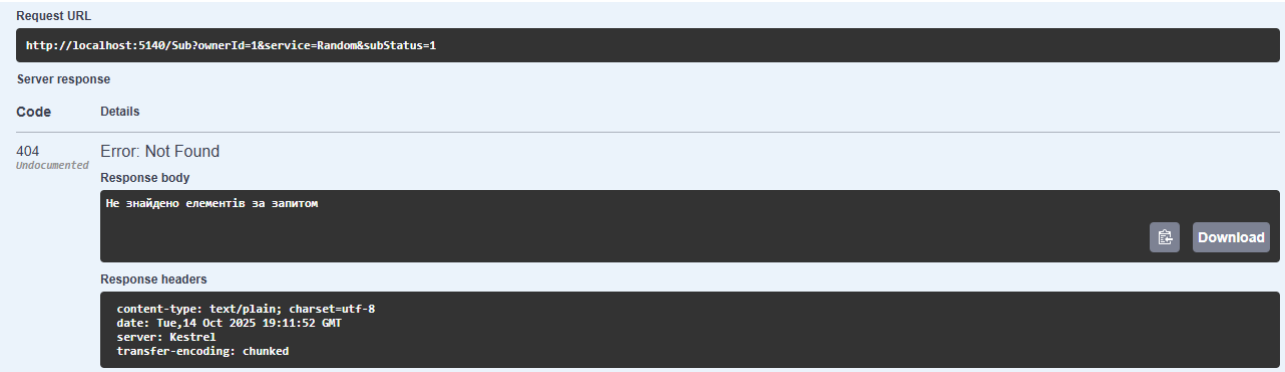


Рисунок 59 – Спроба отримати підписки за відсутнім параметром

Запит 22



Рисунок 60 – Отримання підписки за Id

Запит 23



Рисунок 61 – Спроба отримання неіснуючої підписки за Id
Запит 24



Рисунок 62 – Додавання нової підписки
Запит 25



Рисунок 64 – Спроба додавання підписки із неіснуючим користувачем
Запит 26



Рисунок 65 – Ще одна спроба додати невірну інформацію про підписку

Запит 27

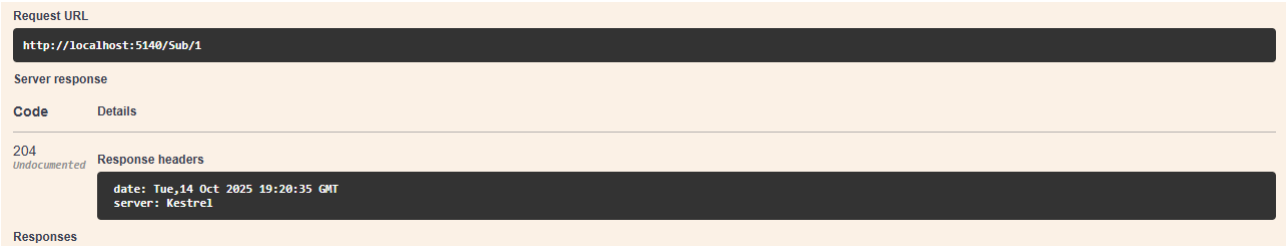


Рисунок 66 – Оновлення інформації про підписку

Запит 28



Рисунок 67 – Спроба некоректного оновлення підписки

Запит 29



Рисунок 68 – Часткове оновлення власника підписки

Запит 30

Request URL

http://localhost:5140/Sub/1

Server response

Code	Details
400	Error: Bad Request

Undocumented

Response body

```
{  "OwnerId": [    "Не знайдено користувача за вказаним 'OwnerId'"  ]}
```

Download

Response headers

```
content-type: application/json; charset=utf-8date: Tue, 14 Oct 2025 19:24:41 GMTserver: Kestreltransfer-encoding: chunked
```

Рисунок 69 – Спроба часткового оновлення підписки невірним користувачем

Запит 31

Request URL

http://localhost:5140/Sub/11

Server response

Code	Details
204	

Undocumented

Response headers

```
date: Tue, 14 Oct 2025 19:25:39 GMTserver: Kestrel
```

Рисунок 70 – Видалення підписки за Id

Запит 32

Request URL

http://localhost:5140/Sub/111

Server response

Code	Details
404	Error: Not Found

Undocumented

Response body

Не знайдено підписки з Id 111

Download

Response headers

```
content-type: text/plain; charset=utf-8date: Tue, 14 Oct 2025 19:26:33 GMTserver: Kestreltransfer-encoding: chunked
```

Рисунок 71 – Спроба видалення неіснуючої підписки

Запит 33

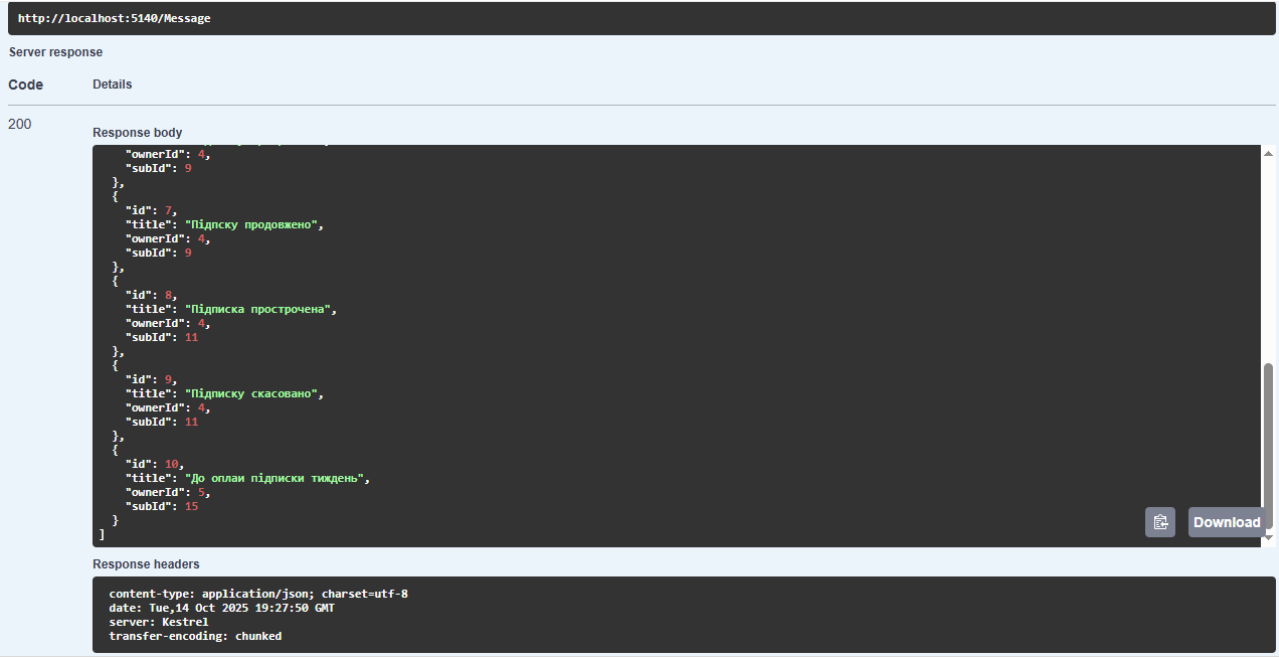


Рисунок 72 – Отримання списку всіх повідомлень

Запит 34

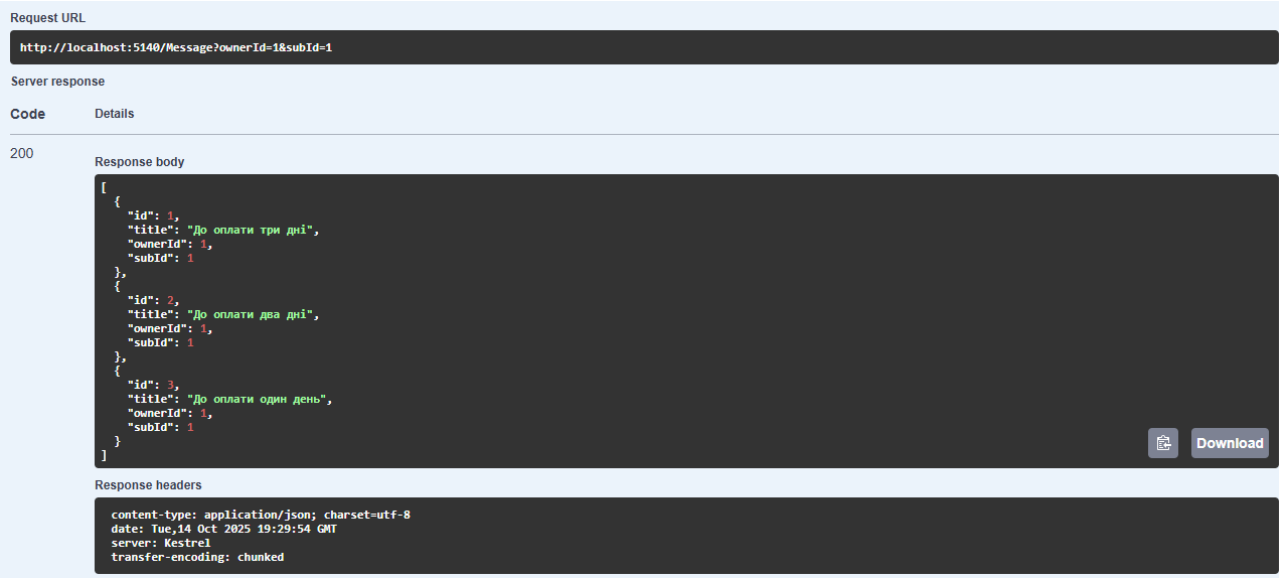


Рисунок 73 – Отримання повідомлень за вказаними параметрами

Запит 35



Рисунок 74 – Спроба отримати неіснуюче повідомлення

Запит 36

Request URL

http://localhost:5140/Message/1

Server response

Code	Details
200	Response body <pre>{ "id": 1, "title": "До оплати три дні", "ownerId": 1, "subId": 1 }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 14 Oct 2025 19:32:09 GMT server: Kestrel transfer-encoding: chunked</pre>

Рисунок 75 – Отримання повідомлення за Id

Запит 37

Request URL

http://localhost:5140/Message/111

Server response

Code	Details
404 <i>Undocumented</i>	Error: Not Found Response body Повідомлення із вказаним Id 111 не знайдено Response headers <pre>content-type: text/plain; charset=utf-8 date: Tue, 14 Oct 2025 19:33:00 GMT server: Kestrel transfer-encoding: chunked</pre>

Рисунок 76 – Спроба отримання неіснуючого повідомлення за Id

Запит 38

Request URL

http://localhost:5140/Message

Server response

Code	Details
201 <i>Undocumented</i>	Response body <pre>{ "id": 10, "title": "Підписка прострочена", "ownerId": 1, "subId": 3 }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 14 Oct 2025 19:34:40 GMT location: http://localhost:5140/Message/10 server: Kestrel transfer-encoding: chunked</pre>

Рисунок 77 – Додавання нової підписки

Запит 39

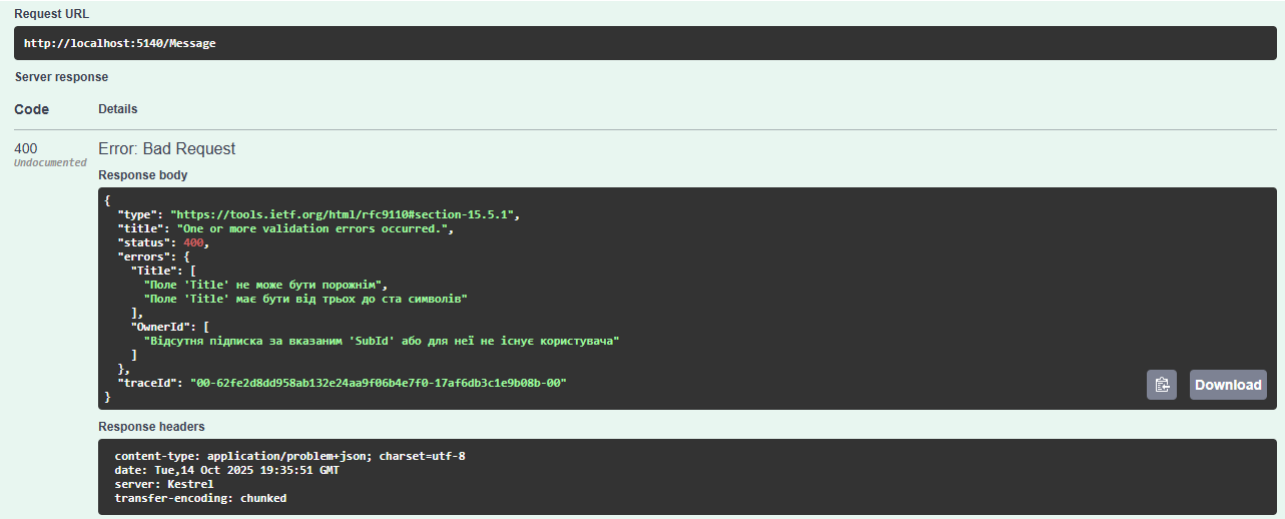


Рисунок 78 – Спроба додавання повідомлення про підписку не тому користувачеві та з порожніс текстом

Запит 40



Рисунок 79 – Коректне оновлення повідомлення

Запит 41



Рисунок 80 – Спроба оновити повідомлення, присвоївши підписку не ому користувачеві та з порожнім підписом

Запит 42



Рисунок 81 – Коректне часткове оновлення

Запит 43

Request URL

http://localhost:5140/Message/1

Server response

Code	Details
400 Undocumented	Error: Bad Request Response body <pre>{ "Title": ["Поле 'Title' не може бути порожнім", "Поле 'Title' має бути від трьох до ста символів"] }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 14 Oct 2025 19:45:28 GMT server: Kestrel transfer-encoding: chunked</pre>

Рисунок 82 – Спроба часткового оновлення повідомлення пустим текстом

Запит 44

Request URL

http://localhost:5140/Message/1

Server response

Code	Details
204 Undocumented	Response headers <pre>date: Tue, 14 Oct 2025 19:47:06 GMT server: Kestrel</pre>

Рисунок 83 – Видалення повідомлення

Запит 45

Request URL

http://localhost:5140/Message/1

Server response

Code	Details
404 Undocumented	Error: Not Found Response body <pre>Повідомлення із вказаним Id 1 не знайдено</pre> Download Response headers <pre>content-type: text/plain; charset=utf-8 date: Tue, 14 Oct 2025 19:47:30 GMT server: Kestrel transfer-encoding: chunked</pre>

Рисунок 84 – Спроба видалення повідомлення ще раз

Завдання 3

Створено діаграму компонентів для програми по відображенню підписок користувача. Діаграма складається Controller Layer – відповідає за прийняття та відправлення http – запитів, валідацію вхідної інформації, Service Layer – відповідає за бізнес логіку, Repository Layer – відповідає за логіку звернення до клієнта БД, DB Layer – відповідає за створення клієнта БД із потрібним вмістом.

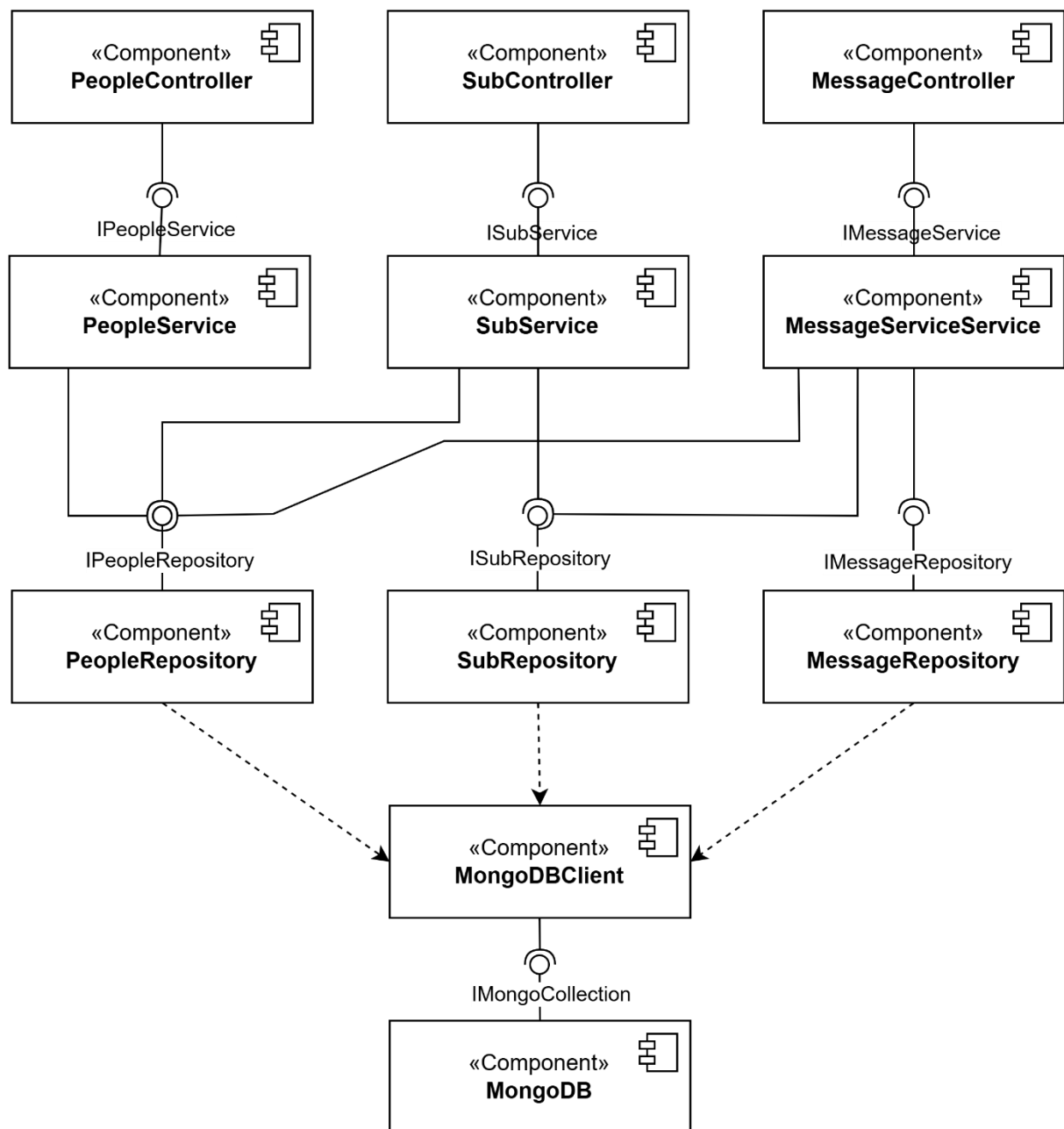


Рисунок 85 – Component Diagram

Controller Layer – надає клієнтові методи для звернення до ресурсів за вказаними ендпоінтами. Він складається із трьох компонентів(контролер для користувачів, підписок, повідомлень), кожний із яких надає можливість виконання CRUD – операцій, приклади методів Create – створення сутності, Get – отримання, Update – Оновлення, Delete – Видалення.

Service Layer – виконує бізнес логіку для запитів/об’єктів від контролерів. Також складається із трьох компонентів (сервіс для користувачів, підписок, повідомлень), реалізує відповідний для нього інтерфейс із набором методів, які має виконати сервіс. Наприклад має реалізувати такі методи: CreateAsync – створення, GetAsync – отримання, UpdateAsync – Оновлення, DeleteAsync – Видалення.

Repository Layer – виконує логіку звернення до клієнта БД для отримання елементів БД чи оновлення. Також складається із трьох компонентів (репозиторій для користувачів, підписок, повідомлень), реалізує відповідний для нього інтерфейс із набором методів, які має виконати репозиторій. Наприклад має реалізувати такі методи: CreateAsync – створення, GetAsync – отримання, UpdateAsync – Оновлення, DeleteAsync – Видалення.

DB Layer – реалізує логіку клієнта для отримання колекції/таблиці елементів із БД. Наприклад реалізує метод GetCollection, за яким можна отримати колекцію потрібних елементів.

Завдання 4

Вдосконалено проект із другого завдання, використовуючи компонентну діаграму із третього завдання. В проекті створено підключення до MongoDB, оновлено моделі, створено шар репозиторіїв, які реалізують відповідні до них інтерфейси, та використовують MongoClient, створено шар сервісів, які реалізують відповідні їм інтерфейси, та які використовують репозиторії, також оновлено шар контролерів, які використовують сервіси

Виконано CRUD операції із використанням MongoDB:

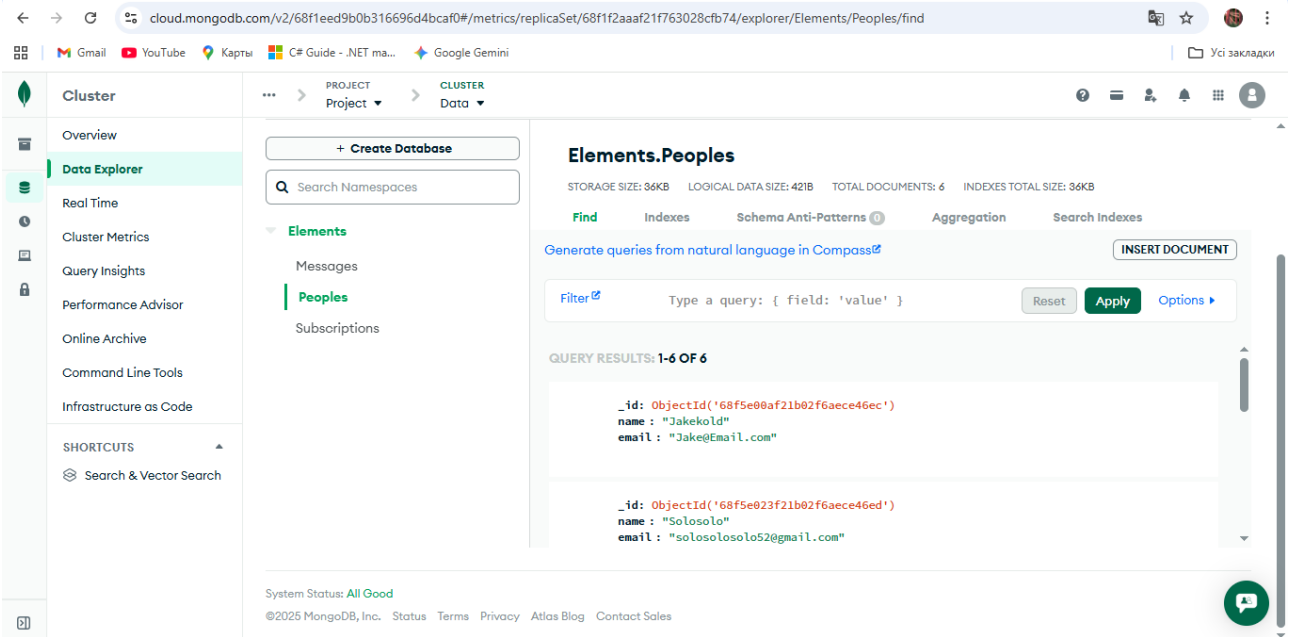


Рисунок 86 – Початковий вміст БД

http://localhost:5129/People

Server response

Code

Details

200

Response body

```
[
  {
    "id": "68f5e00ef21b02f6aace46ec",
    "name": "Jakekold",
    "email": "Jake@Email.com"
  },
  {
    "id": "68f5e023f21b02f6aace46ed",
    "name": "Solosolo",
    "email": "solosolosolo52@gmail.com"
  },
  {
    "id": "68f5e054f21b02f6aace46ee",
    "name": "Susanteam",
    "email": "Susan@Email.com"
  },
  {
    "id": "68f5e06bf21b02f6aace46ef",
    "name": "Mikefreer",
    "email": "Mike@Email.com"
  },
  {
    "id": "68ff7822a866d7d7495a11c",
    "name": "Chorpovnil",
    "email": "Chort@Email.com"
  },
  {
    "id": "690113f4b26a5f31097a0fad",

```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,30 Oct 2025 14:53:06 GMT
server: Kestrel
transfer-encoding: chunked
```

Рисунок 87 – Отримання всіх користувачів

Curl

```
curl -X 'GET' \
'http://localhost:5129/People?name=Susanteam' \
-H 'accept: text/plain'
```

Request URL

http://localhost:5129/People?name=Susanteam

Server response

Code

Details

200

Response body

```
[
  {
    "id": "68f5e054f21b02f6aace46ee",
    "name": "Susanteam",
    "email": "Susan@Email.com"
  }
]
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,30 Oct 2025 14:54:57 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Рисунок 88 – Отримно користувача з використанням фільтру

Request URL

http://localhost:5129/People/68f5e054f21b02f6aace46ee

Server response

Code

Details

200

Response body

```
{
  "id": "68f5e054f21b02f6aace46ee",
  "name": "Susanteam",
  "email": "Susan@Email.com"
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,30 Oct 2025 14:58:08 GMT
server: Kestrel
transfer-encoding: chunked
```

Рисунок 89 – Отримано користувача за Id

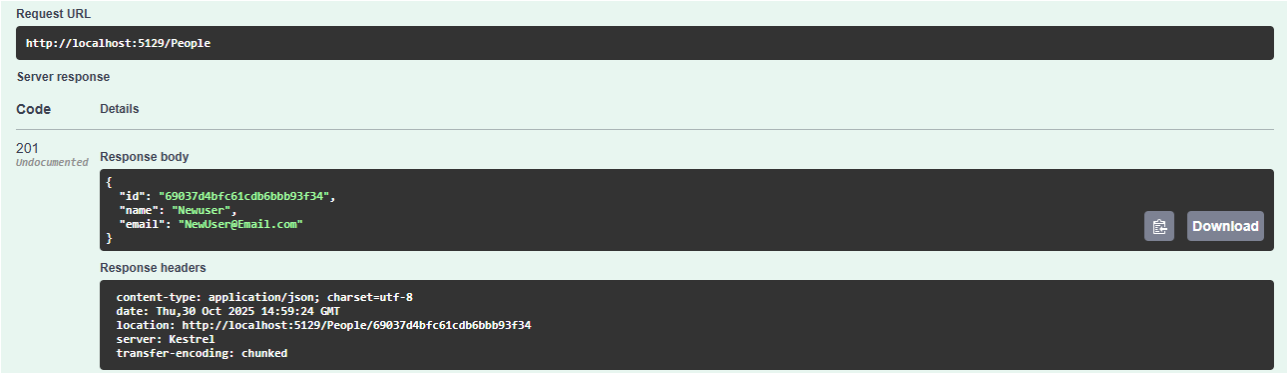


Рисунок 90 – Створено нового користувача

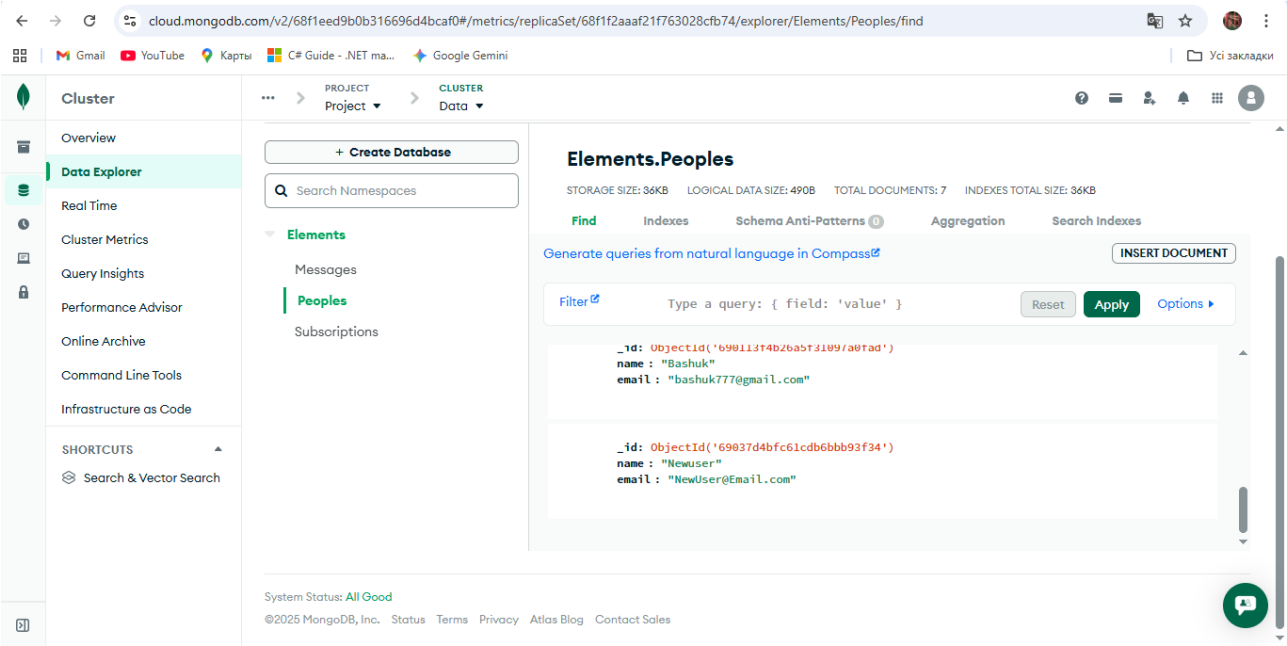


Рисунок 91 – Вміст бази даних після додавання нового користувача



Рисунок 92 – Повне оновлення користувача

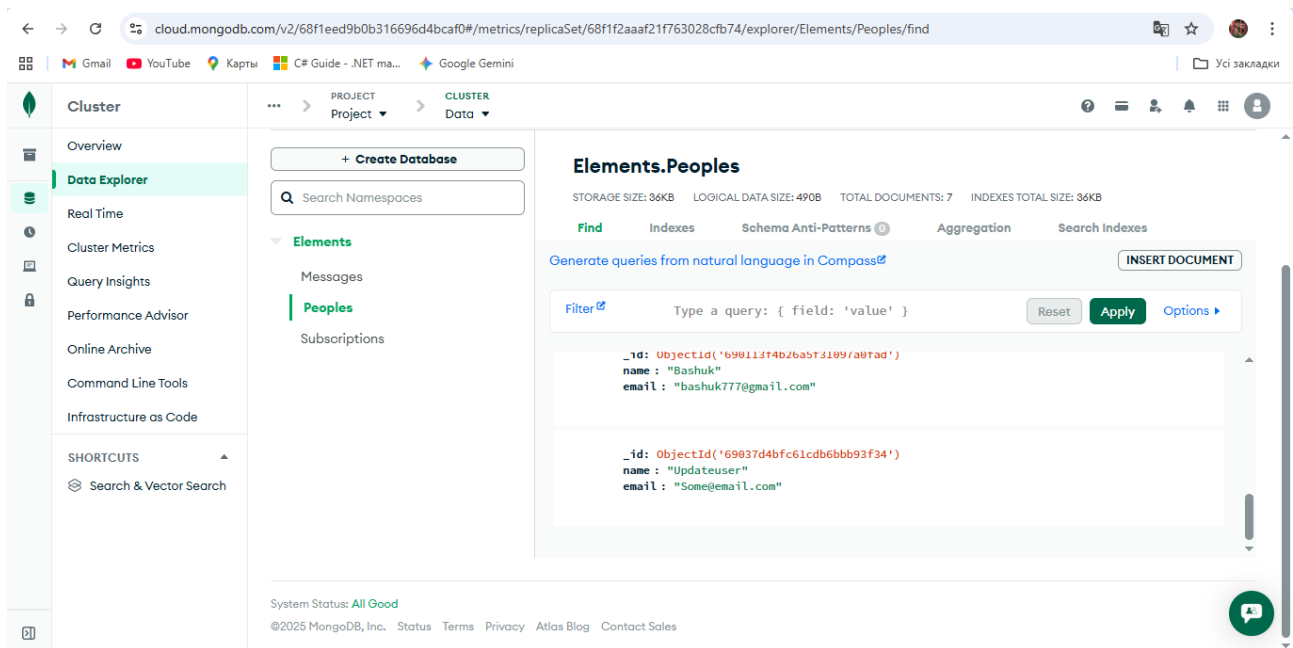


Рисунок 93 – Вміст після повного оновлення

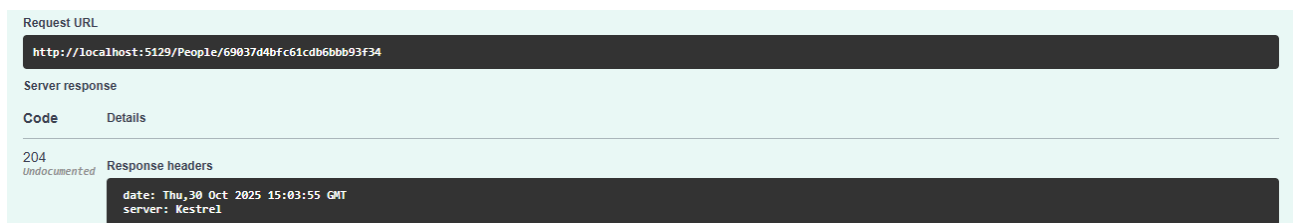


Рисунок 94 – Часткове оновлення користувача

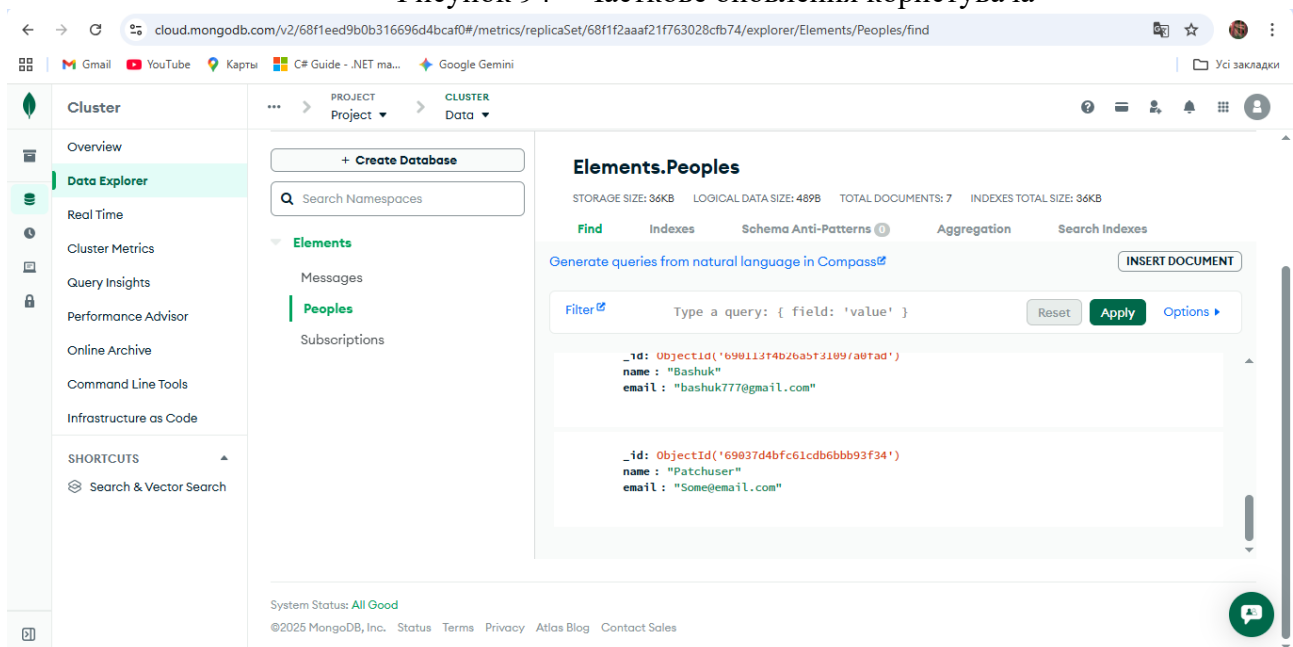


Рисунок 95 – Вміст бази даних після часткового оновлення

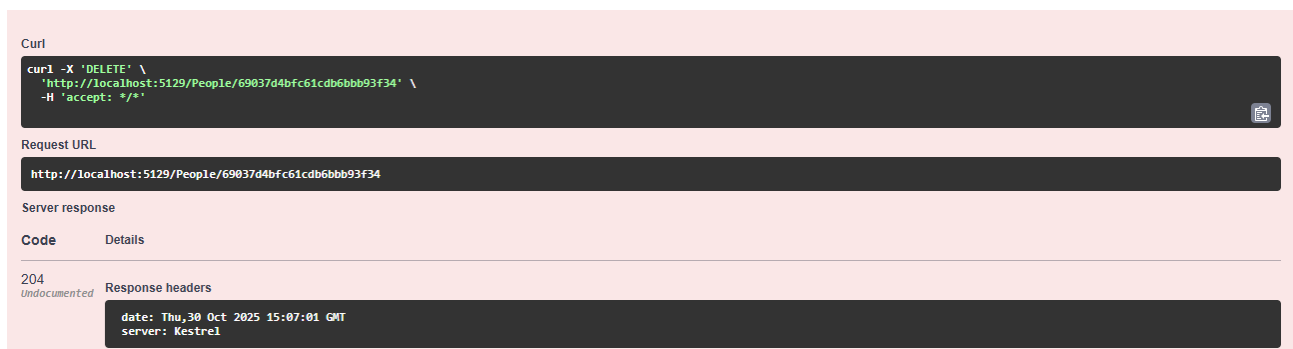


Рисунок 96 – Видалення користувача

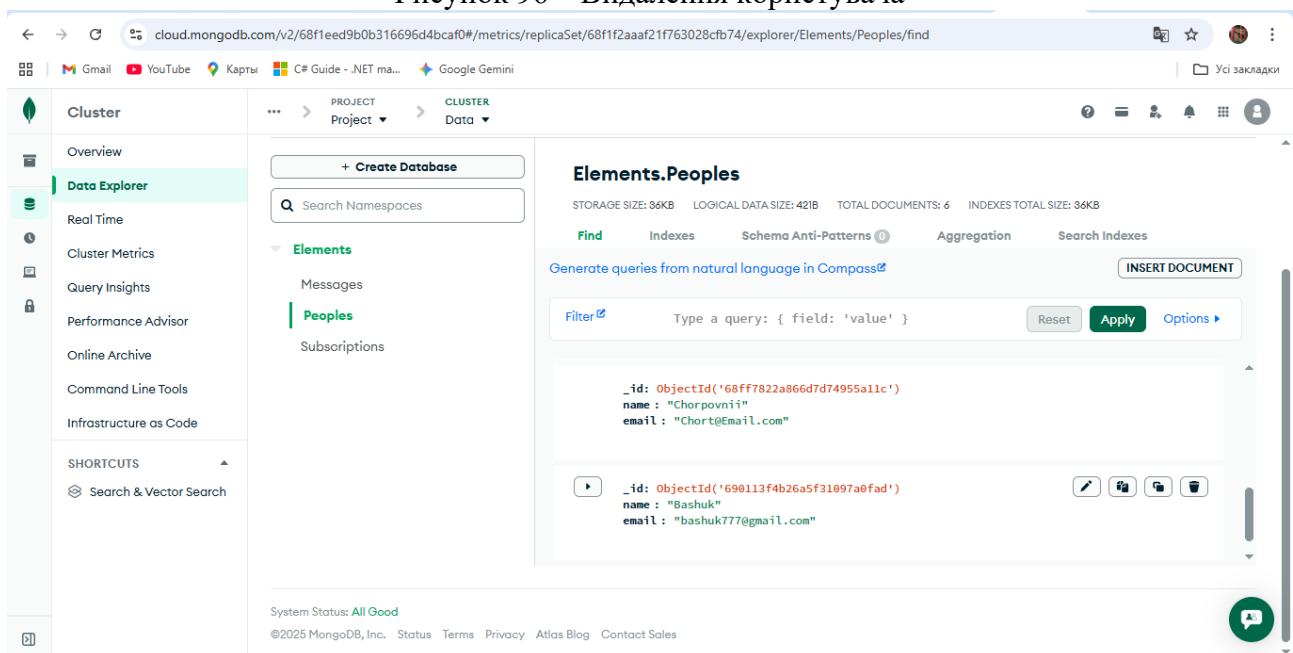


Рисунок 97 – Вміст бази даних після видалення

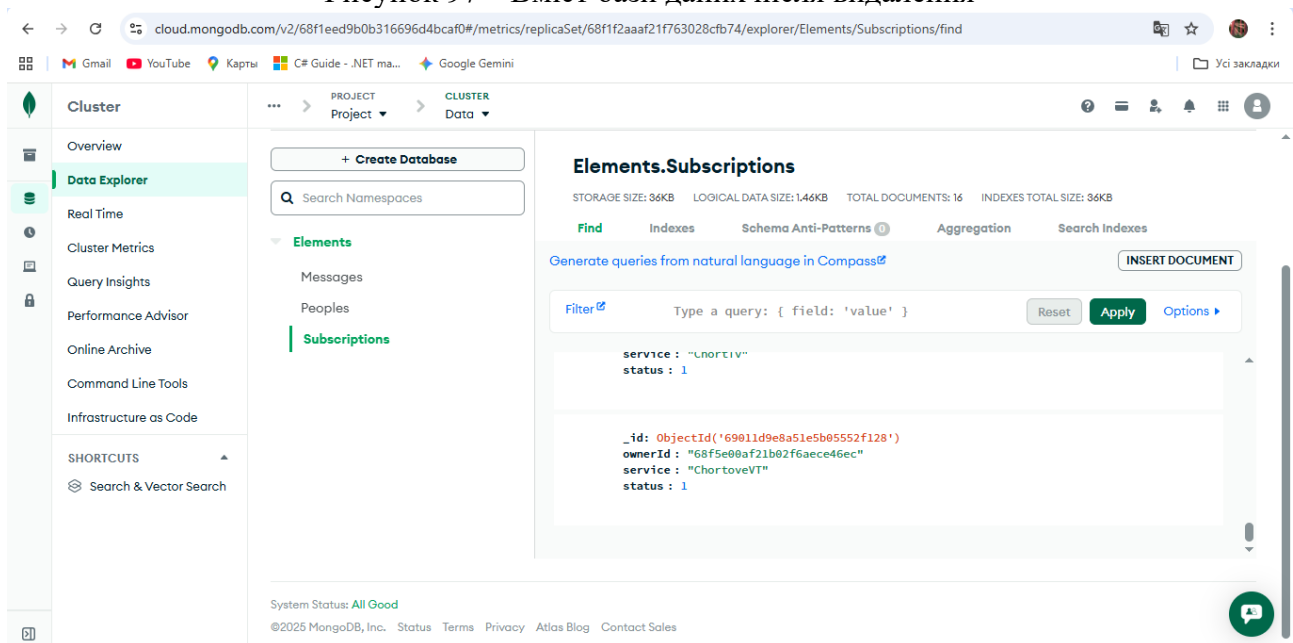


Рисунок 98 – Вміст колекції підписок

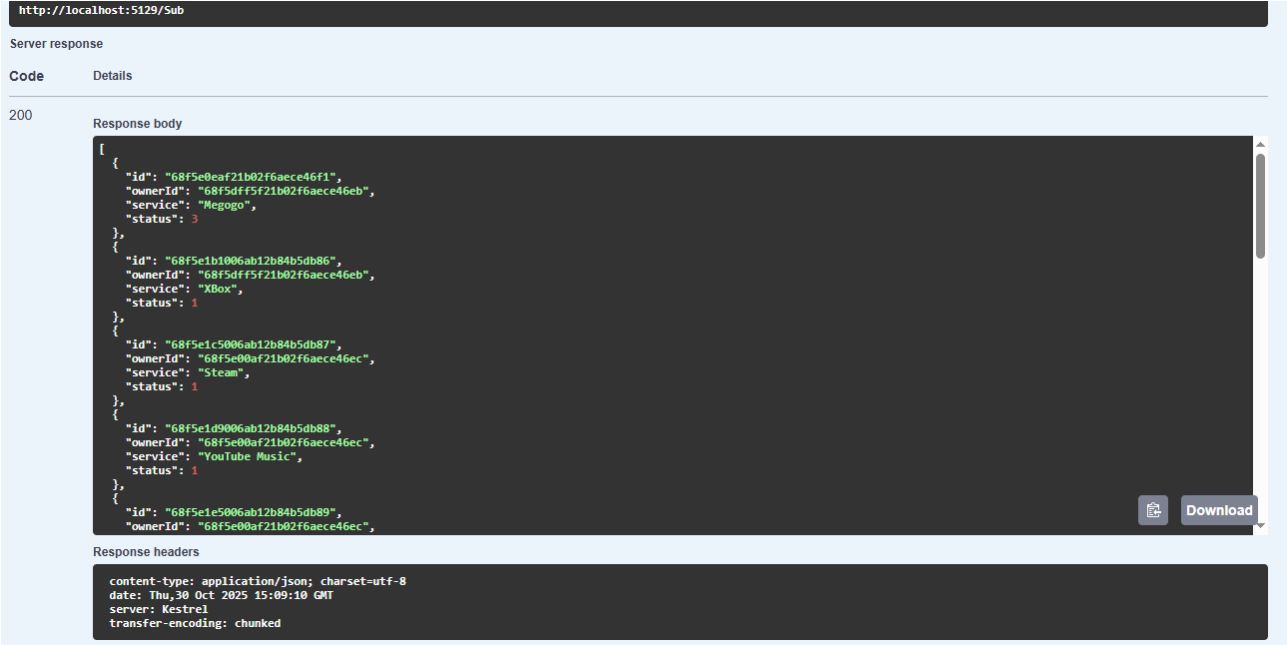


Рисунок 99 – Отримано всі підписки користувачів



Рисунок 100 – Отримано підписки за вказаними фільтрами



Рисунок 101 – Отримано підписку за конкретним Id

Request URL

http://localhost:5129/Sub

Server response

Code

Details

201

Undocumented

Response body

```
{
  "id": "6903802bfc61cdb6bbb93f35",
  "ownerId": "68f5e054f21b02f6aeca46ee",
  "service": "Amazon",
  "status": 1
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu, 30 Oct 2025 15:11:39 GMT
location: http://localhost:5129/Sub/6903802bfc61cdb6bbb93f35
server: Kestrel
transfer-encoding: chunked
```

Рисунок 102 – Створено нову підписку для користувача

cloud.mongodb.com/v2/68f1eed9b0b316696d4bcaf0#/metrics/replicaSet/68f1f2aaaf21f763028cfb74/explorer/Elements/Subscriptions/find

Cluster

Overview

Data Explorer

Real Time

Cluster Metrics

Query Insights

Performance Advisor

Online Archive

Command Line Tools

Infrastructure as Code

SHORTCUTS

Search & Vector Search

PROJECT

Project

CLUSTER

Data

+ Create Database

Search Namespaces

Elements

Messages

Peoples

Subscriptions

Elements.Subscriptions

STORAGE SIZE: 36KB LOGICAL DATA SIZE: 1.55KB TOTAL DOCUMENTS: 17 INDEXES TOTAL SIZE: 36KB

Find Indexes Schema Anti-Patterns Aggregation Search Indexes

Generate queries from natural language in Compass

INSERT DOCUMENT

Filter Type a query: { field: 'value' } Reset Apply Options

```
service : "LhortoveVI"
status : 1

_id: ObjectId('6903802bfc61cdb6bbb93f35')
ownerId : "68f5e054f21b02f6aeca46ee"
service : "Amazon"
status : 1
```

System Status: All Good ©2025 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

Рисунок 103 – Вміст бази даних після створення підписки

Request URL

http://localhost:5129/Sub/6903802bfc61cdb6bbb93f35

Server response

Code

Details

204

Undocumented

Response headers

```
date: Thu, 30 Oct 2025 15:19:30 GMT
server: Kestrel
```

Рисунок 104 – Повне оновлення підписки

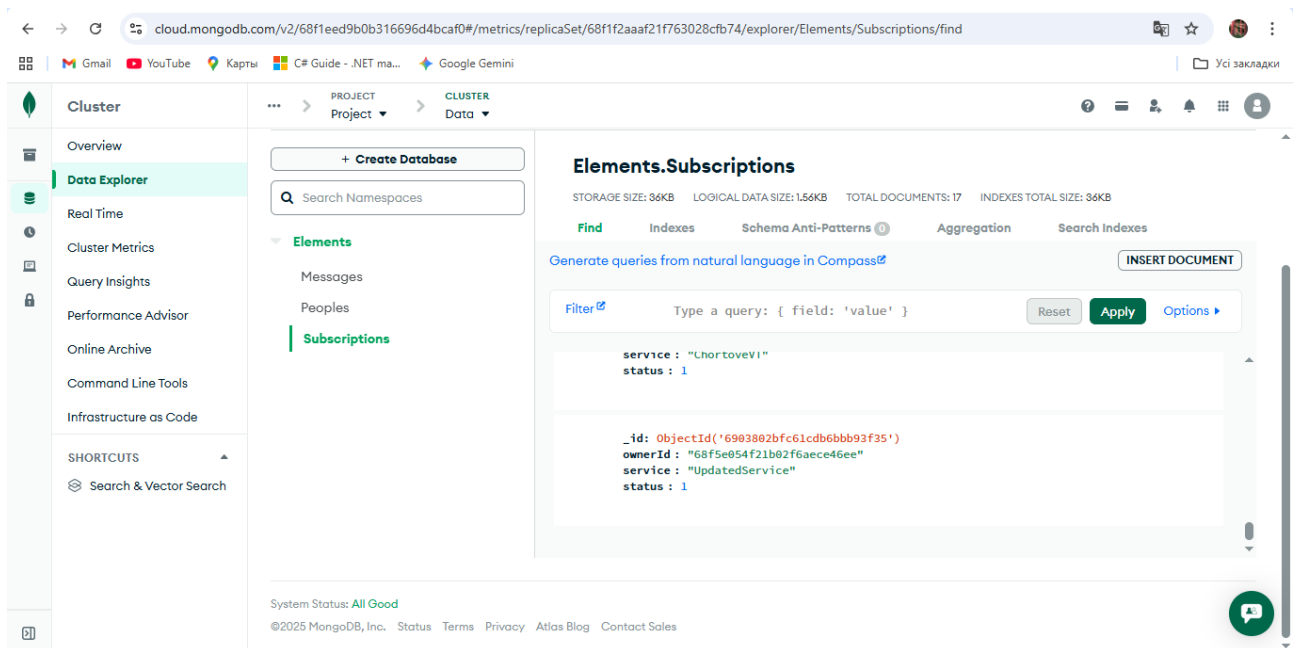


Рисунок 105 – Вміст колекції після повного оновлення

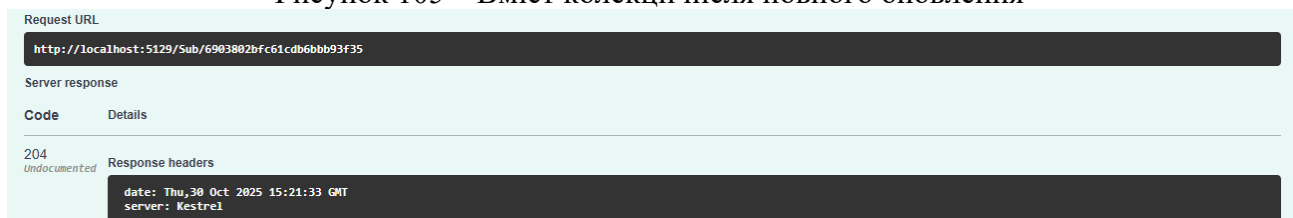


Рисунок 106 – Часткове оновлення підписки

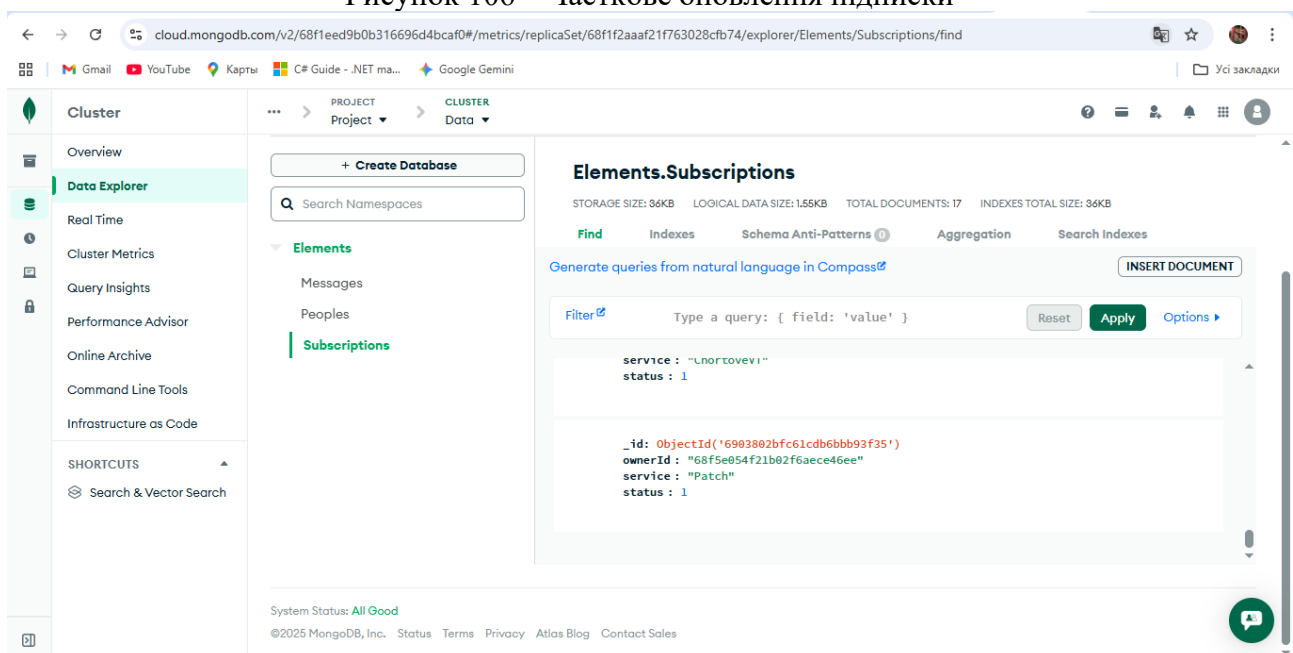


Рисунок 107 – Вміст колекції після часткового оновлення

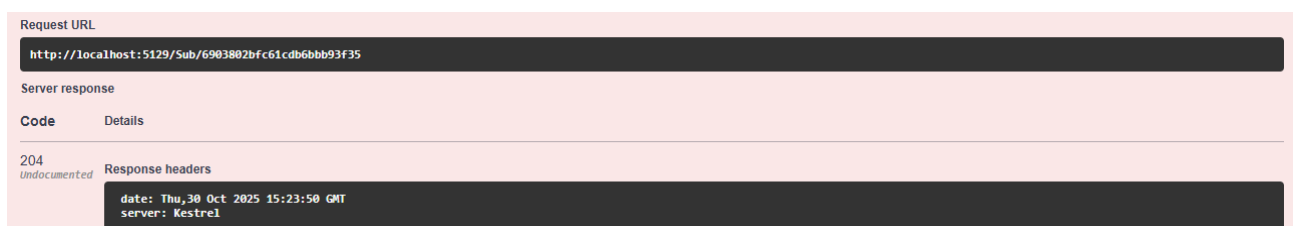


Рисунок 108 – Видалення підписки

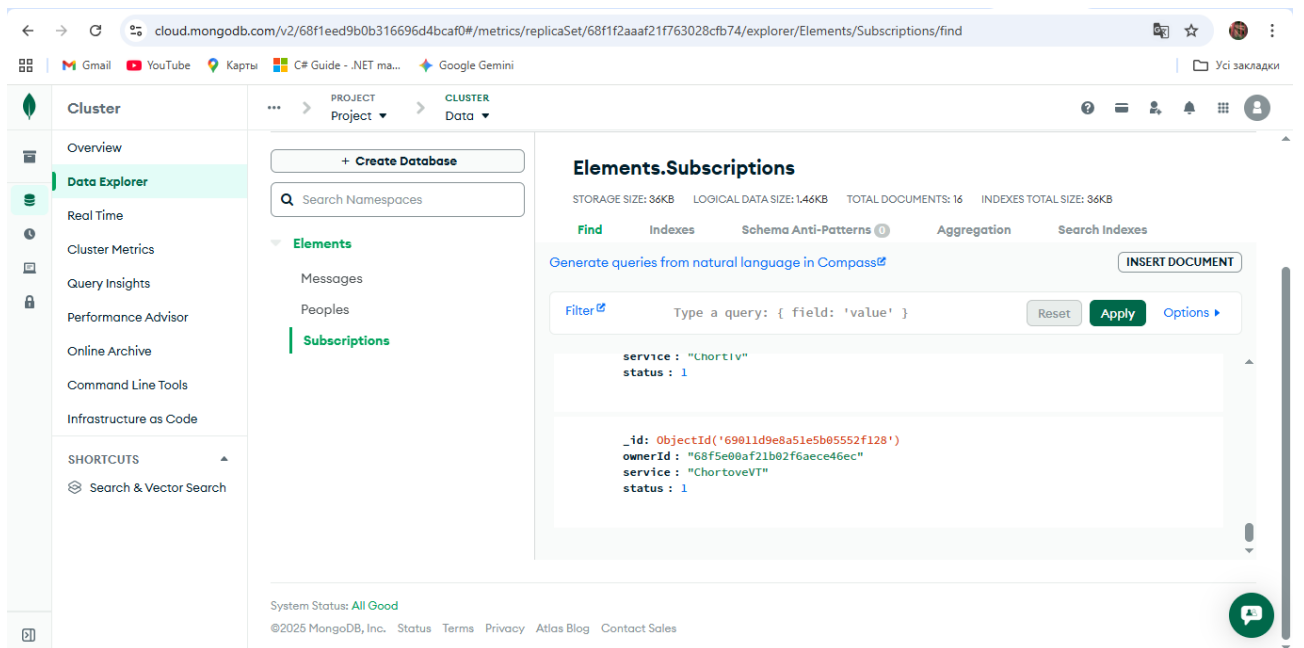


Рисунок 109 – Вміст колекції після видалення підписки

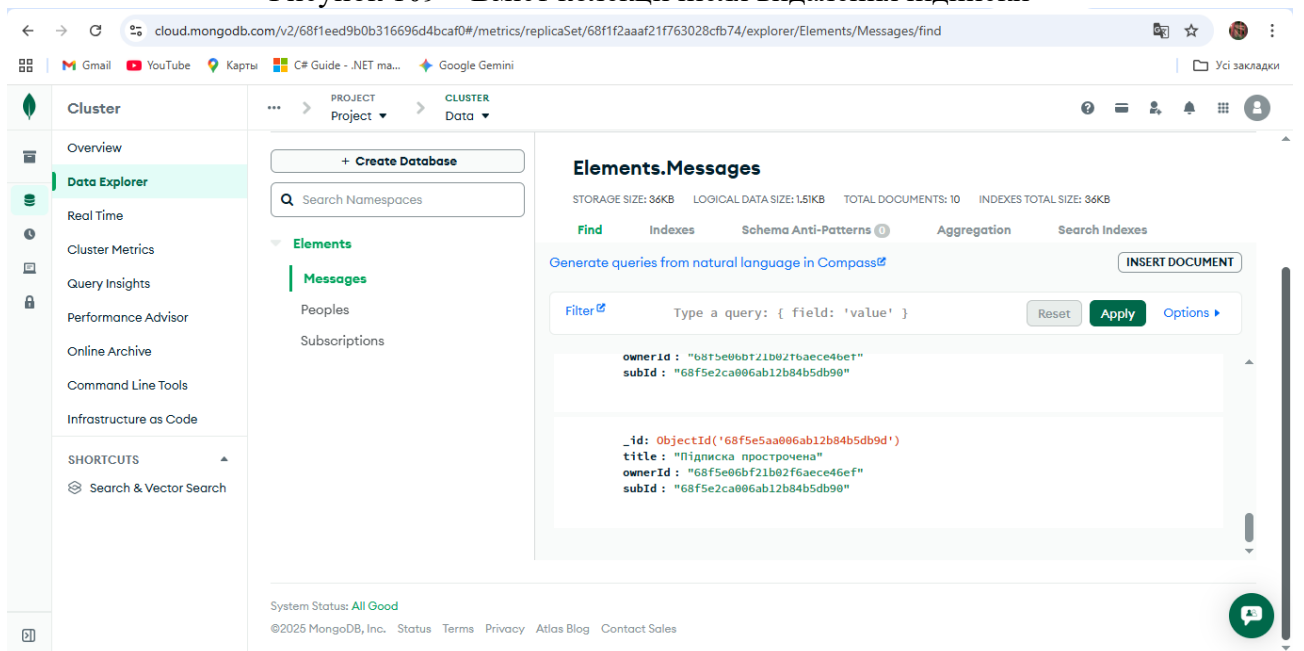


Рисунок 110 – Вміст колекції повідомлень

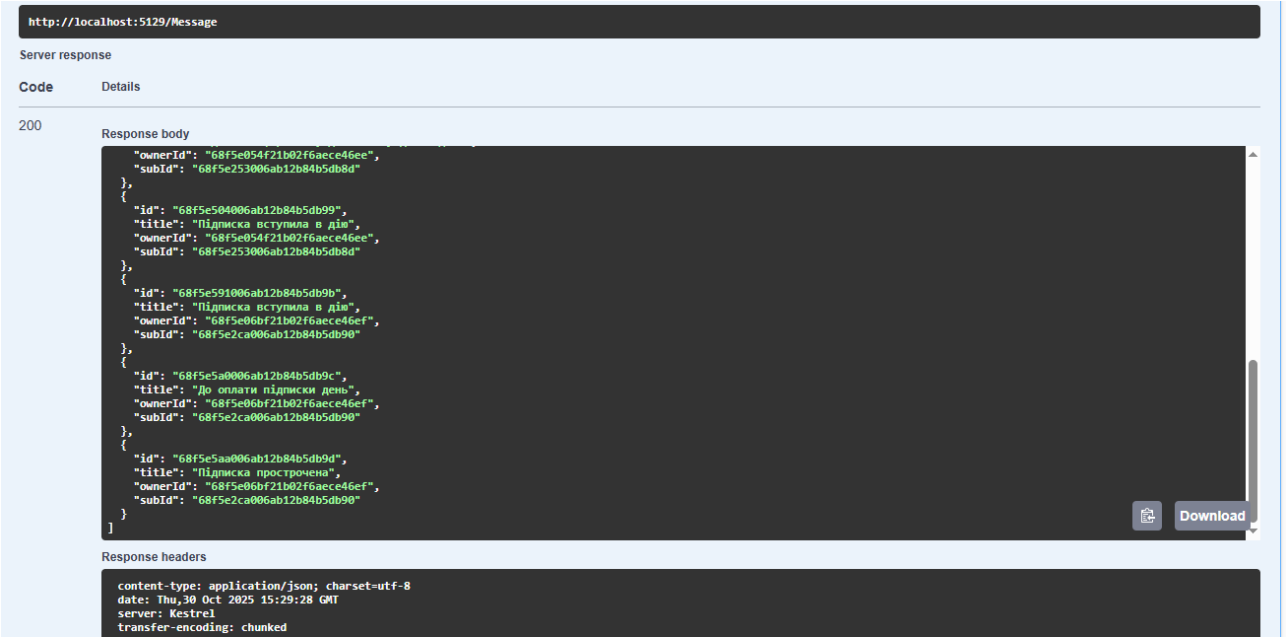


Рисунок 111 – Отримання списку всіх повідомлень



Рисунок 112 – Отримання конкретного повідомлення

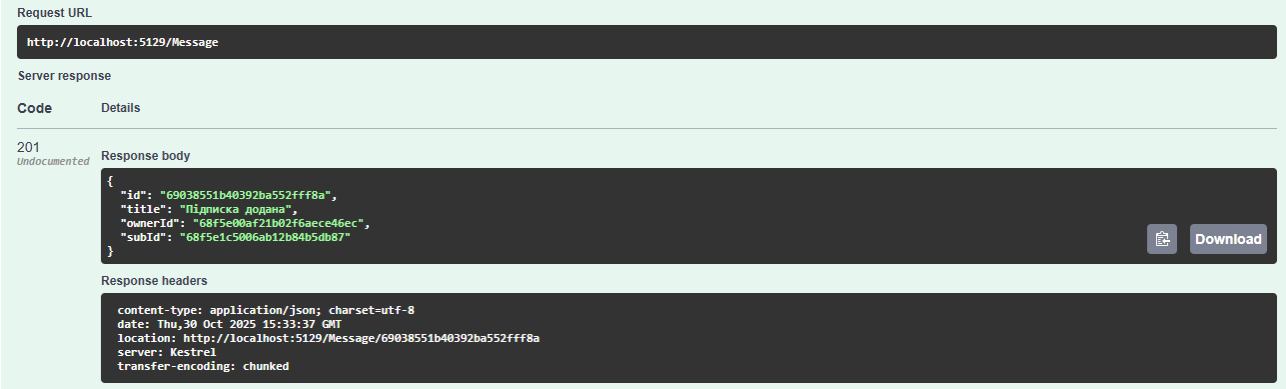


Рисунок 113 – Створення нового повідомлення

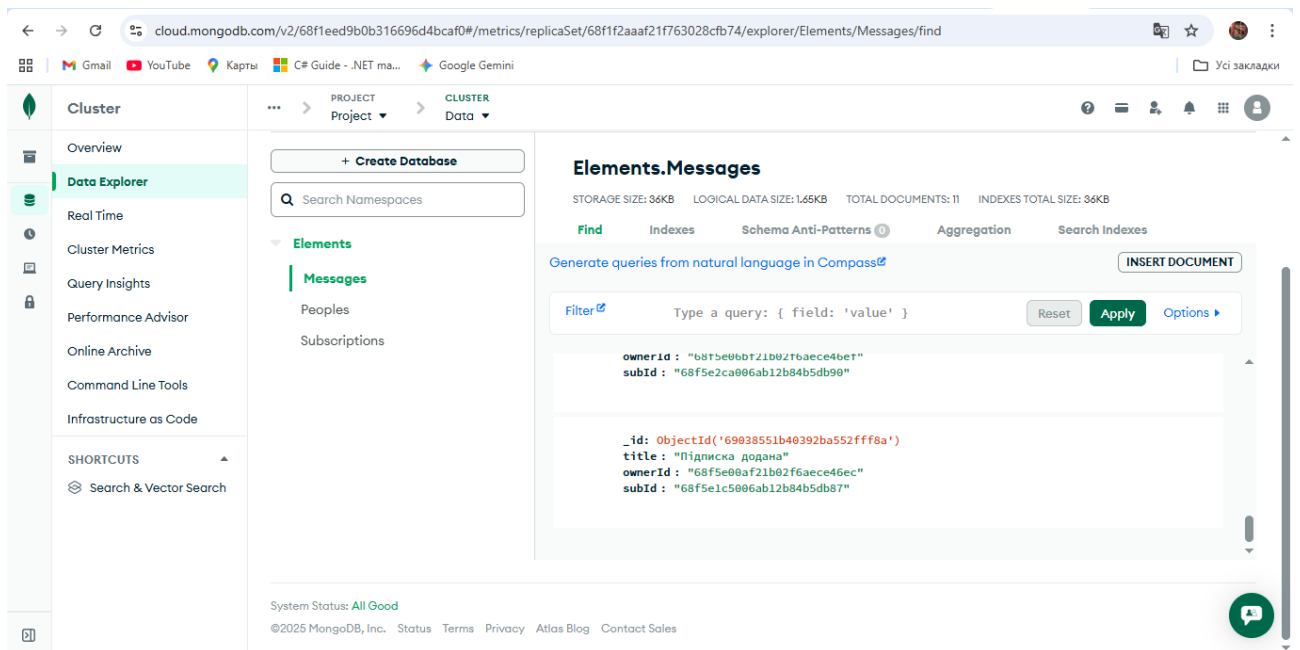


Рисунок 114 – Вміст колекції після додавання

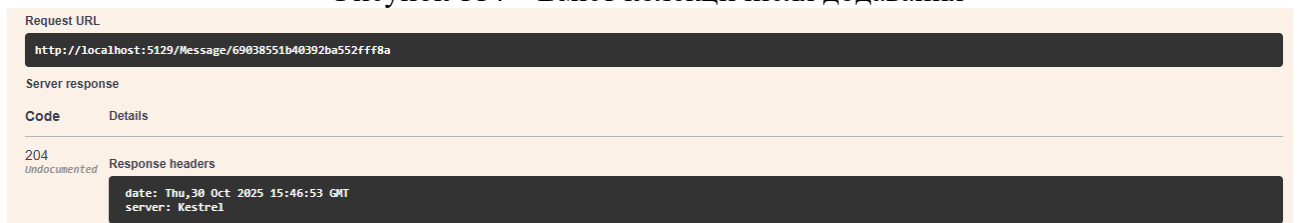


Рисунок 115 – Повне оновлення повідомлення

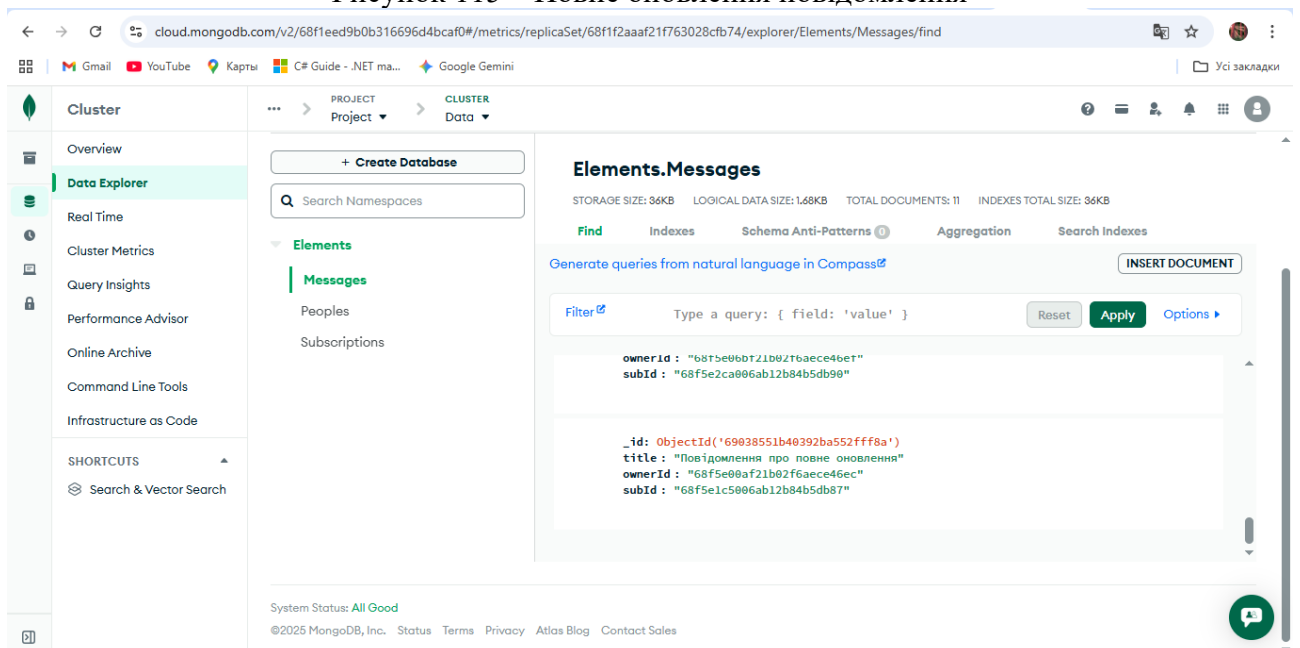


Рисунок 116 – Вміст колекції після повного оновлення

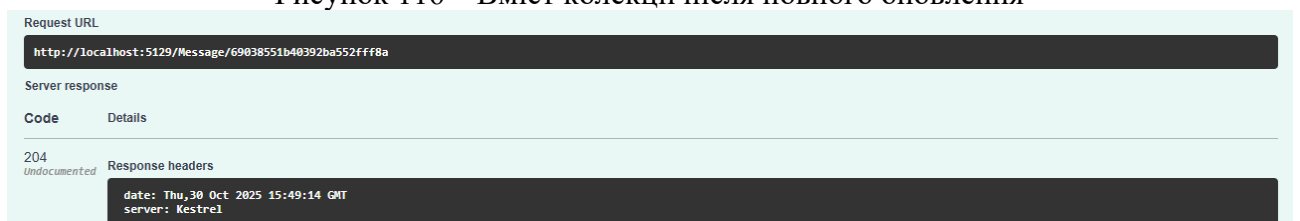


Рисунок 117 – Часткове оновлення повідомлення

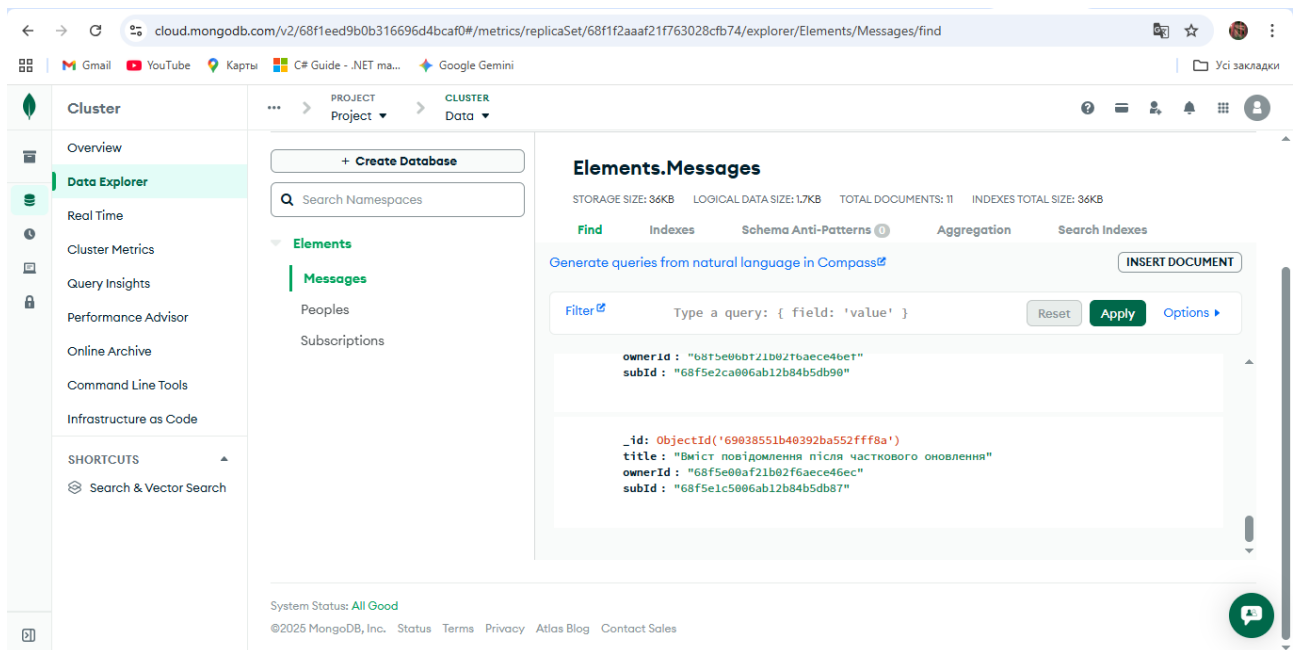


Рисунок 118 – Вміст колекції після часткового оновлення повідомлення



Рисунок 119 – Видалення повідомлення

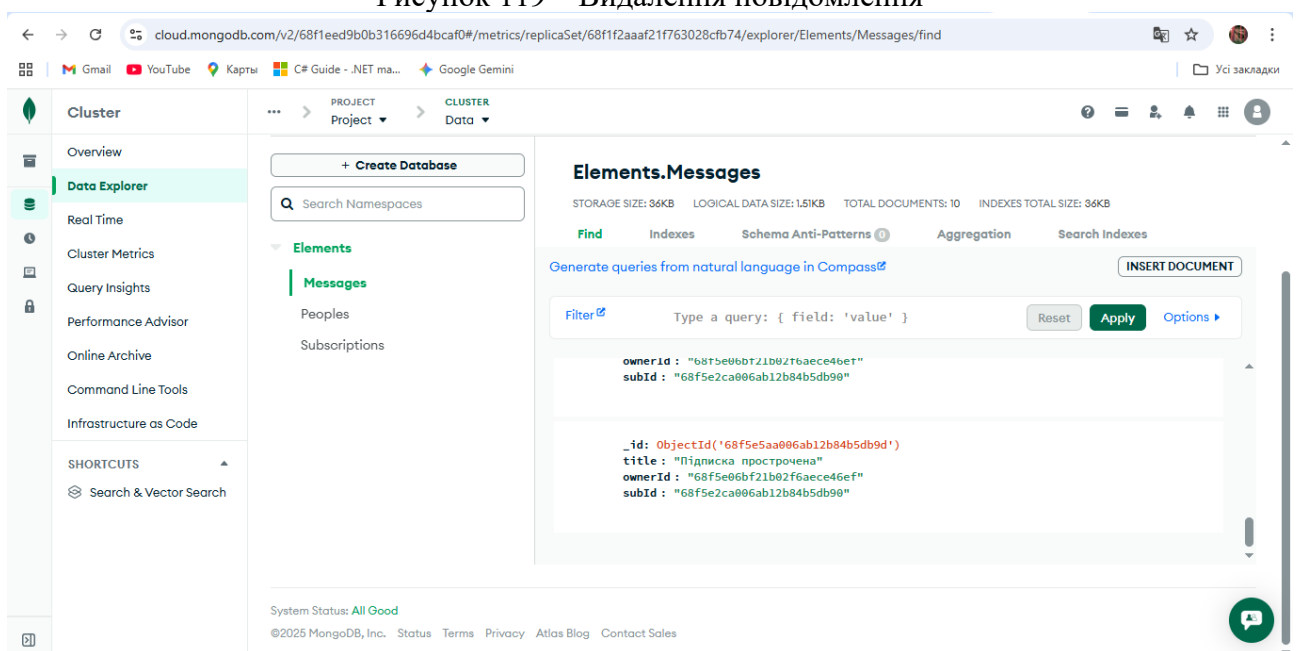


Рисунок 120 – Вміст колекції після оновлення

Завдання 5

В проект із попереднього завдання було додано нові моделі, автотестування, налаштовано використання репозиторіїв, сервісів, автотестування за допомогою DI.

Приклади використання мапінгу в програмі:

PeopleController.cs:

					ЛР.ОК24.ПІ231.03.04-06	Арк.
Змін.	Арк.	№ докум.	Підпис	Дата		68

```

using AutoMapper;
using LW4_task_3.Interfaces;
using LW4_task_3.Models.Entities;
using LW4_task_3.Models.Request;
using LW4_task_3.Models.Response;
using LW4_task_3.Validators;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using System.Text.Json;
namespace LW4_task_3.Controllers
{
    [Route("[controller]")]
    [ApiController]
    public class PeopleController : ControllerBase
    {
        private readonly IPeopleService _peopleService;
        // Створення змінної для подальшого використання за допомогою DI
        private readonly IMapper _mapper;
        public PeopleController(IPeopleService peopleService, IMapper mapper)
        {
            _peopleService = peopleService;
            _mapper = mapper;
        }
        [HttpGet]
        public async Task<ActionResult<IEnumerable<PeopleResponse>>> Get(
            [FromQuery] string? name, [FromQuery] string? email)
        {
            try
            {
                var peoplesItems = await _peopleService.GetPeoplesItemsAsync(name, email);
                // Перетворення DTO в Entity
                var peoples = _mapper.Map<IEnumerable<PeopleResponse>>(peoplesItems);
                return Ok(peoples);
            }
            catch (KeyNotFoundException kex)

```

					ЛР.ОК24.ПІ231.03.04-06	Арк.
Змін.	Арк.	№ докум.	Підпис	Дата		69

```

        {
            return NotFound(kex.Message);
        }
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<PeopleResponse>> GetById(string id)
    {
        try
        {
            ValidElement.ValidId(id);
            var peopleItem = await _peopleService.GetByIdAsync(id);
            // Перетворення DTO в Entity
            var people = _mapper.Map<PeopleResponse>(peopleItem);
            return Ok(people);
        }
        catch (KeyNotFoundException kex)
        {
            return NotFound(kex.Message);
        }
        catch (ArgumentException aex)
        {
            return BadRequest(aex.Message);
        }
    }

    [HttpPost]
    public async Task<IActionResult> Create(PeopleRequest item)
    {
        // Перетворення DTO в Entity
        var element = _mapper.Map<PeopleItem>(item);
        await _peopleService.CreateAsync(element);
        // Перетворення Entity в DTO
        var resp = _mapper.Map<PeopleResponse>(element);
        return CreatedAtAction(nameof(GetById), new { Id = resp.Id }, resp);
    }

    [HttpPut("{id}")]

```

```

public async Task<IActionResult> Update(string id, PeopleRequest item)
{
    try
    {
        ValidElement.ValidId(id);
        // Перетворення DTO в Entity
        var element = _mapper.Map<PeopleItem>(item);
        await _peopleService.UpdateAsync(id, element);
        return NoContent();
    }
    catch (KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
    catch (ArgumentException aex)
    {
        return BadRequest(aex.Message);
    }
}
[HttpPatch("{id}")]
public async Task<IActionResult> UpdatePart(string id, JsonElement element)
{
    try
    {
        ValidElement.ValidId(id);
        var upPeople = await _peopleService.GetByIdAsync(id);
        // Перетворення Entity в DTO
        var temp = _mapper.Map<PeopleRequest>(upPeople);
        if (element.TryGetProperty("name", out var name))
            temp.Name = name.GetString();
        if (element.TryGetProperty("email", out var email))
            temp.Email = email.GetString();
        if (!TryValidateModel(temp))
            return BadRequest(ModelState);

        //Малпінг DTO в Entity

```

```

        _mapper.Map(temp, upPeople);
        await _peopleService.UpdateAsync(id, upPeople);
        return NoContent();
    }
    catch (KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
    catch (ArgumentException aex)
    {
        return BadRequest(aex.Message);
    }
}
}
}

```

Створено профілі для маппінгу кожного типу моделей:

```

namespace LW4_task_3.Mapping
{
    public class MessageProfile: Profile
    {
        public MessageProfile()
        {
            CreateMap<MessageRequest, MessageItem>().ReverseMap();
            CreateMap<MessageItem, MessageResponse>();
        }
    }
}

namespace LW4_task_3.Mapping
{
    public class PeopleProfile: Profile
    {
        public PeopleProfile()
        {
            CreateMap<PeopleRequest, PeopleItem>().ReverseMap();
            CreateMap<PeopleItem, PeopleResponse>();
        }
    }
}

```



```

    }
}
}
namespace LW4_task_3.Mapping
{
    public class SubProfile: Profile
    {
        public SubProfile()
        {
            CreateMap<SubscriptionRequest, SubscriptionItem>()
                .ForMember(d => d.Status, o => o.MapFrom(s => Enum.Parse<SubStatus>(s.S
tatus,true)));
            CreateMap<SubscriptionItem, SubscriptionRequest>()
                .ForMember(d => d.Status, o => o.MapFrom(s => s.Status.ToString()));
            CreateMap<SubscriptionItem, SubscriptionResponse>()
                .ForMember(d => d.Status, o => o.MapFrom(s => s.Status.ToString()));
        }
    }
}

```

Додано до конструкторів контролерів, сервісів прив'язку до інтерфейсів, а не до об'єктів.

Виконано запити для перевірки роботи DI та Mapping

MessageController:

```

private readonly IMessageService _messageService;
private readonly IMapper _mapper;
public MessageController(IMessageService messageService, IMapper mapper)
{
    _messageService = messageService;
    _mapper = mapper;
}

```

SubController:

```

private readonly ISubService _subService;
private readonly IMapper _mapper;
public SubController(ISubService subService, IMapper mapper)
{

```

```

        _subService = subService;
        _mapper = mapper;
    }

```

PeopleController:

```

private readonly IPeopleService _peopleService;
private readonly IMapper _mapper;
public PeopleController(IPeopleService peopleService, IMapper mapper)
{
    _peopleService = peopleService;
    _mapper = mapper;
}

```

MessageService:

```

private readonly IMessageRepository _messageRepository;
private readonly ISubRepository _subRepository;
private readonly IPeopleRepository _peopleRepository;
public MessageService(IMessageRepository messageRepository, ISubRepository subRepository, IPeopleRepository peopleRepository)
{
    _messageRepository = messageRepository;
    _subRepository = subRepository;
    _peopleRepository = peopleRepository;
}

```

SubService:

```

private readonly ISubRepository _subRepository;
private readonly IPeopleRepository _peopleRepository;
public SubService(ISubRepository subRepository, IPeopleRepository peopleRepository)
{
    _subRepository = subRepository;
    _peopleRepository = peopleRepository;
}

```

PeopleService:

```

private readonly IPeopleRepository _peopleRepository;
public PeopleService(IPeopleRepository peopleRepository)
{
}

```

```

        _peopleRepository = peopleRepository;
    }

В Program.cs додано прив'язку між інтерфейсом та класом для реалізації DI:
using FluentValidation;
using FluentValidation.AspNetCore;
using LW4_task_3.Models;
using LW4_task_3.Validators;
using LW4_task_3.Services;
using LW4_task_3.Interfaces;
using LW4_task_3.InterfacesRepository;
using LW4_task_3.Repositories;
using LW4_task_3.Mapping;
var builder = WebApplication.CreateBuilder(args);
// Add services to the container.
builder.Services.AddControllers().AddFluentValidation();
builder.Services.AddScoped<IPeopleRepository, PeopleRepository>();
builder.Services.AddScoped<ISubRepository, SubRepository>();
builder.Services.AddScoped<IMessageRepository, MessageRepository>();
builder.Services.AddScoped<IPeopleService, PeopleService>();
builder.Services.AddScoped<ISubService, SubService>();
builder.Services.AddScoped<IMessageService, MessageService>();
// Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbu
ckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    var xmlFile = $"{System.Reflection.Assembly.GetExecutingAssembly().GetName().Name}.xml";
    var xmlPath = Path.Combine(AppContext.BaseDirectory, xmlFile);
    options.IncludeXmlComments(xmlPath);
});
builder.Services.AddValidatorsFromAssemblyContaining<PeopleValidator>();
builder.Services.AddValidatorsFromAssemblyContaining<SubValidator>();
builder.Services.AddValidatorsFromAssemblyContaining<MessageValidator>();
builder.Services.AddAutoMapper(typeof(PeopleProfile));

```

```

builder.Services.AddAutoMapper(typeof(SubProfile));
builder.Services.AddAutoMapper(typeof(MessageProfile));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseAuthorization();

app.MapControllers();

app.Run();

```

В програмі DI працює за допомогою заміни створення об'єктів у конструкторах класів на прив'язку до інтерфейсів, які підставляються автоматично за допомогою реєстрації в Program.cs. AutoMapper також використовується за допомогою DI, клас має об'єкт типу IMapper за допомогою якого робить мапінг, зареєстрований в класові – профілі.

Здійснено перевірку роботи CRUD – операцій після оновлення проекту.

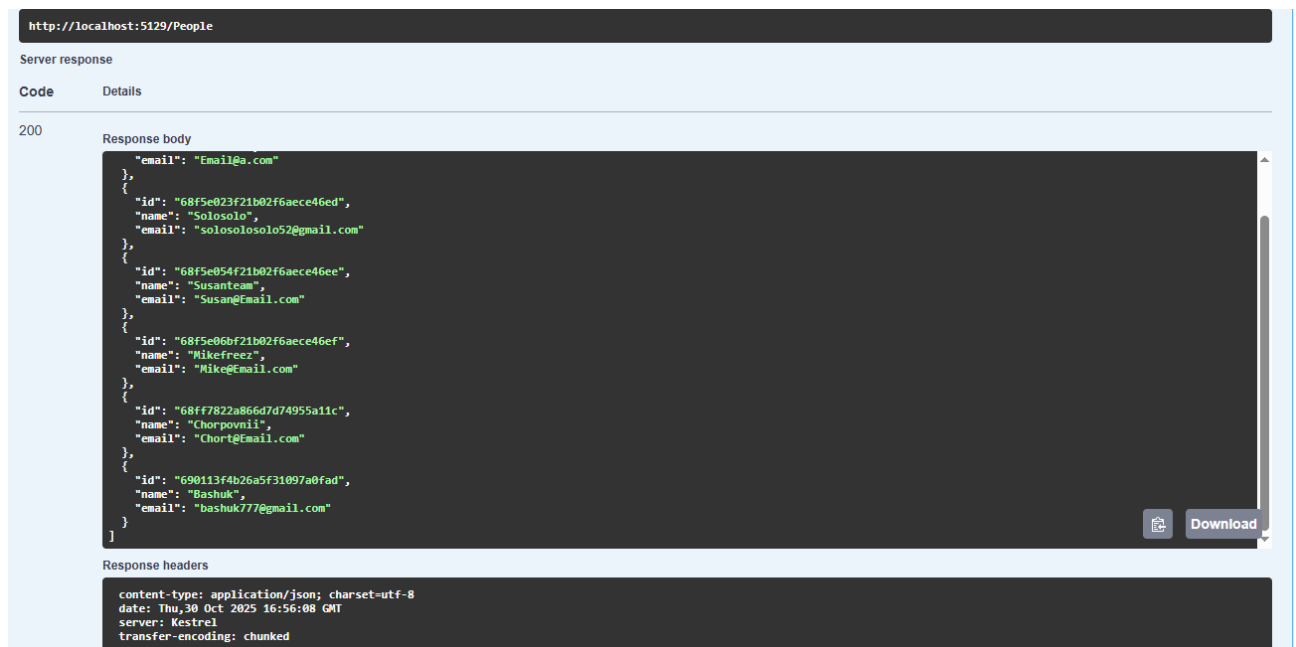


Рисунок 121 – Отримано список всіх користувачів



Рисунок 122 – Отримано користувачів за вказаним фільтром

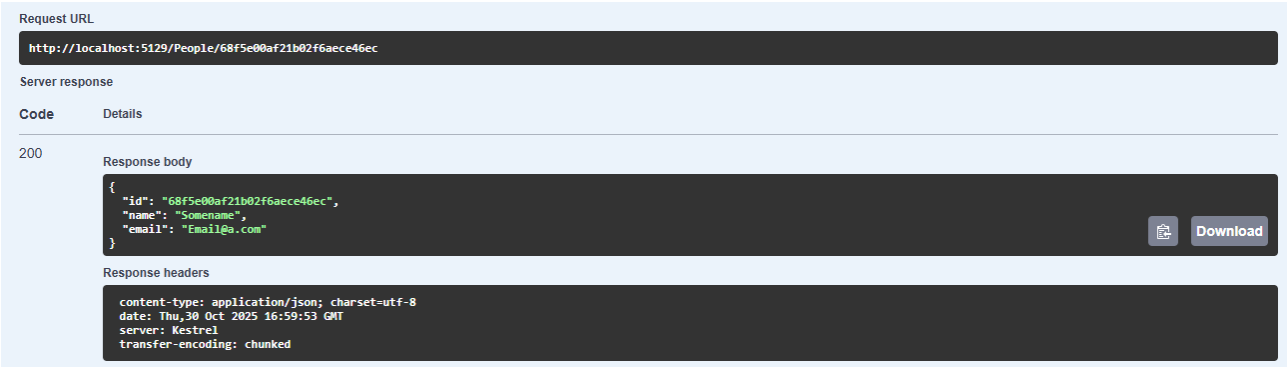


Рисунок 123 – Отримано користувача за Id



Рисунок 124 – Створено нового користувача



Рисунок 125 – Оновлено повністю користувача



Рисунок 126 – Здійснено часткове оновлення



Рисунок 127 – Видалено користувача

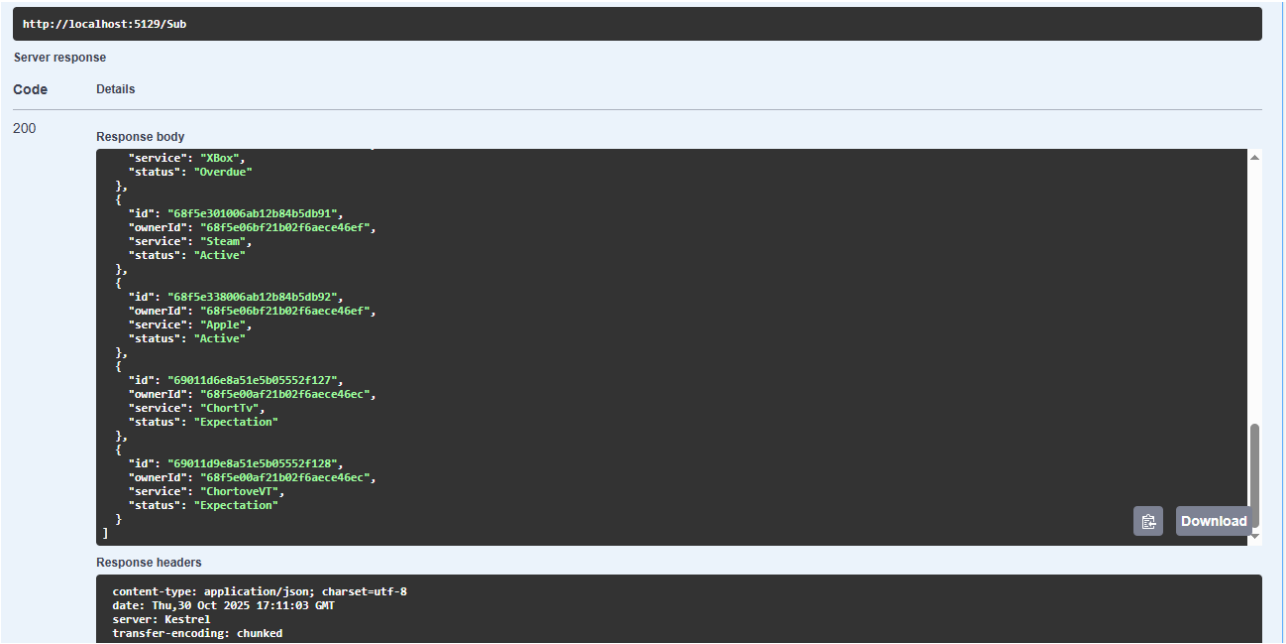


Рисунок 128 – Отримано список всіх підписок

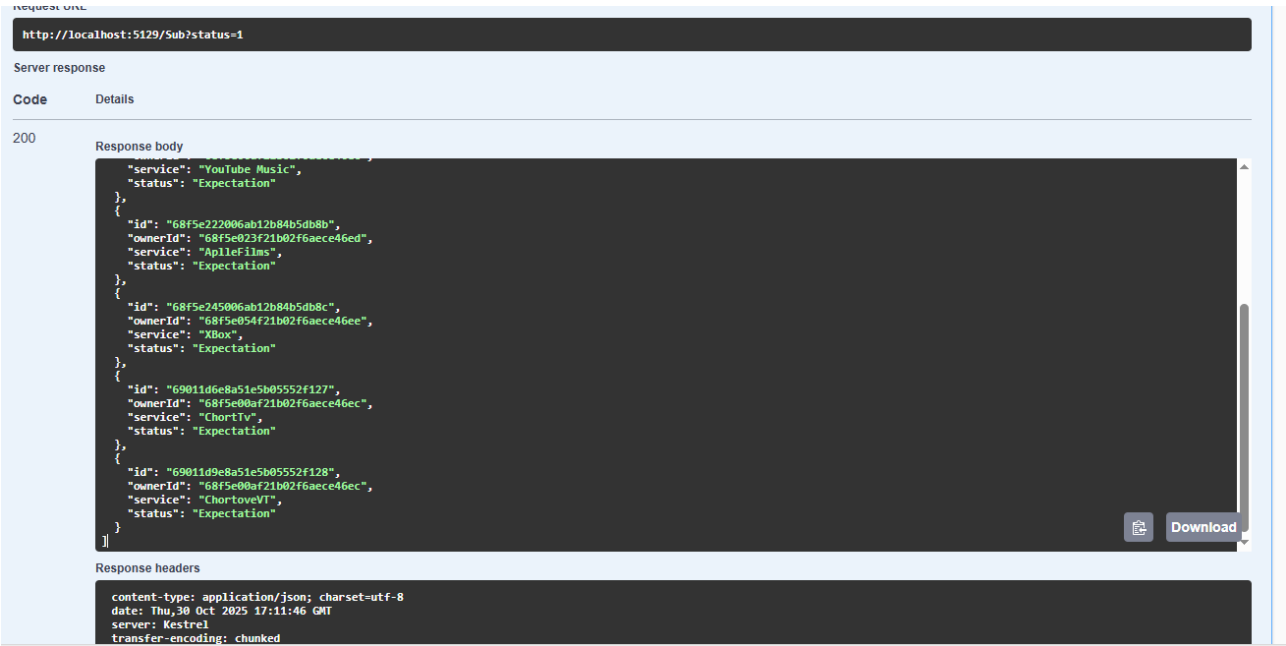


Рисунок 129 – Отримано список підписок, що очікують на активацію



Рисунок 130 – Отримано підписку за Id

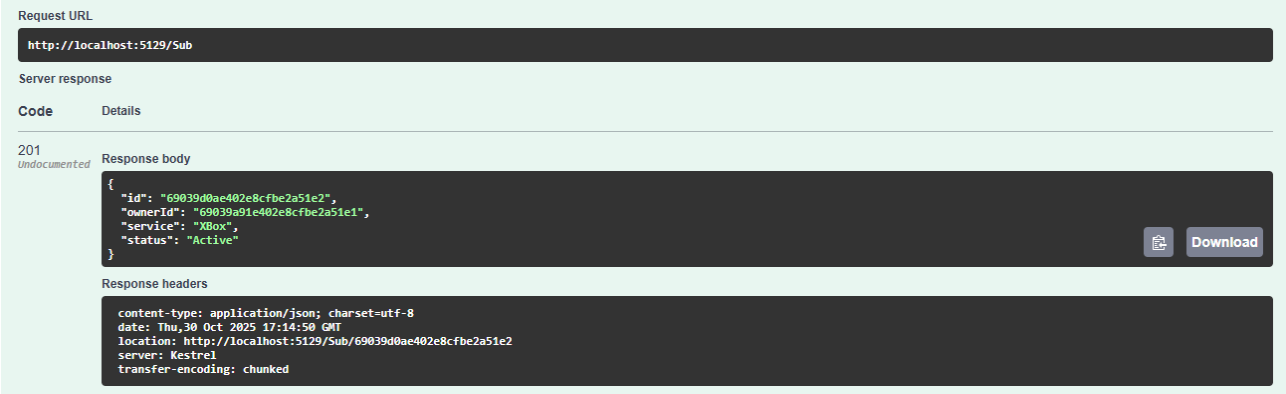


Рисунок 130 – Створено нову підписку для користувача



Рисунок 131 – Здійснено повне оновлення підписки



Рисунок 132 – Здійснено часткове оновлення підписки



Рисунок 133 – Видалено підписку

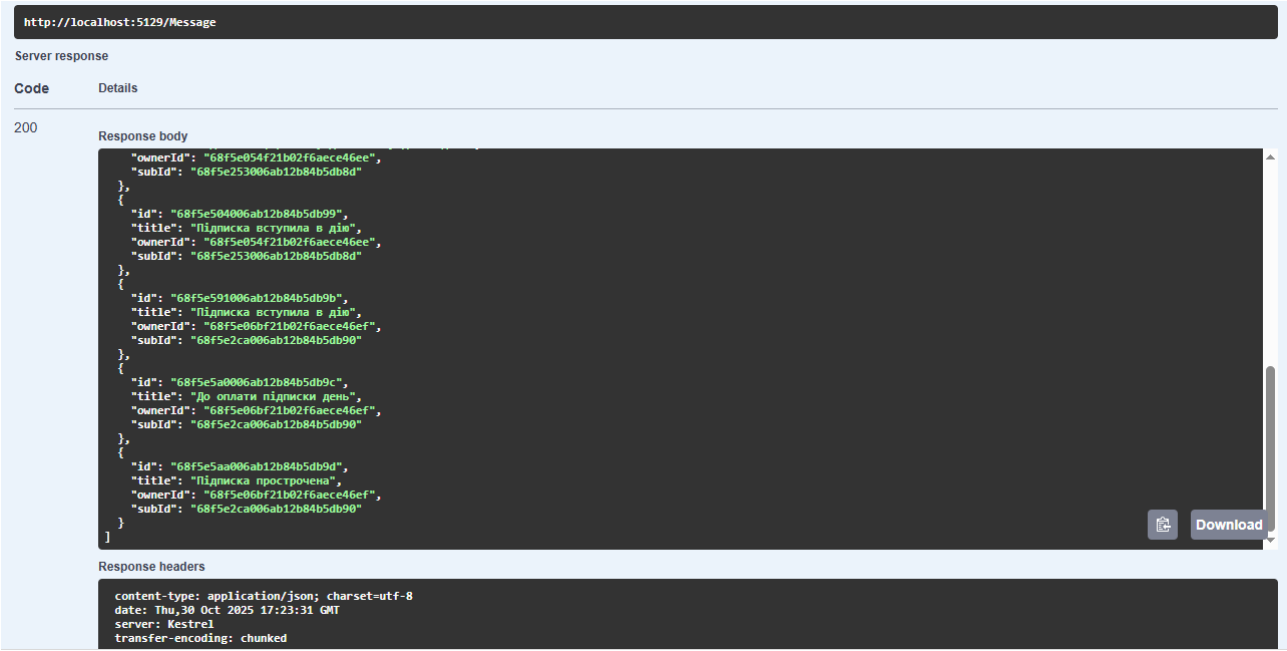


Рисунок 134 – Отримано список всіх повідомлень



Рисунок 135 – Отримано повідомлення за Id

Request URL

http://localhost:5129/Message

Server response

Code

Details

201

Undocumented

Response body

```
{
  "id": "69039f92e402e8cfe2a51e4",
  "title": "Підписка активована",
  "ownerId": "68f5e023f21b02f6aece46ed",
  "subId": "68f5e219006ab12b84b5db8a"
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,30 Oct 2025 17:25:38 GMT
location: http://localhost:5129/Message/69039f92e402e8cfe2a51e4
server: Kestrel
transfer-encoding: chunked
```

Рисунок 136 – Створено нове повідомлення

Request URL

http://localhost:5129/Message/69039f92e402e8cfe2a51e4

Server response

Code

Details

204

Undocumented

Response headers

```
date: Thu,30 Oct 2025 17:26:38 GMT
server: Kestrel
```

Рисунок 137 – Повністю оновлено повідомлення

Request URL

http://localhost:5129/Message/69039f92e402e8cfe2a51e4

Server response

Code

Details

204

Undocumented

Response headers

```
date: Thu,30 Oct 2025 17:27:35 GMT
server: Kestrel
```

Рисунок 138 – Частково оновлено повідомлення

Request URL

http://localhost:5129/Message/68f5e430006ab12b84b5db95

Server response

Code

Details

204

Undocumented

Response headers

```
date: Thu,30 Oct 2025 17:29:10 GMT
server: Kestrel
```

Рисунок 139 – Видалено повідомлення

Завдання 6

Встановлено необхідні пакети та налаштовано appsetting.json:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
```

```
"JwtSettings": {
  "SecretKey": "My_Super_Secret_Key_2w22w3_4w52q2728=93+_some",
  "Issuer": "http://localhost:5129",
  "Audience": "http://localhost:5129",
  "AccessTokenExpirationMinutes": 60,
  "RefreshTokenExpirationDays": 30
}
```

Створено модель JwtSetting та сутність для роботи із БД, і DTO моделі для логіну та реєстрації:

```
namespace LW4_task_3.Models
{
    public class JwtSettings
    {
        public string SecretKey { get; set; }
        public string Issuer { get; set; }
        public string Audience { get; set; }
        public int AccessTokenExpirationMinutes { get; set; }
        public int RefreshTokenExpirationDays { get; set; }
    }
}

using LW4_task_3.Enums;
using Microsoft.AspNetCore.Identity;
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;
namespace LW4_task_3.Models.Entities
{
    public class UserItem
    {
        [BsonId]
        [BsonRepresentation(BsonType.ObjectId)]
        public string? Id { get; set; }
        [BsonElement("username")]
        public string? UserName { get; set; }
    }
}
```

```

        [BsonElement("email")]
        public string? Email { get; set; }
        [BsonElement("passwordHash")]
        public string? Password { get; set; }
        [BsonElement("refreshToken")]
        public string? RefreshToken { get; set; }
        [BsonElement("refreshTokenExpireTime")]
        public DateTime? RefreshTokenExpireTime { get; set; }
        [BsonElement("role")]
        public UserRole Role { get; set; } = UserRole.User;
    }
}

namespace LW4_task_3.Models.Request
{
    public class LoginModel
    {
        public string Email { get; set; }
        public string Password { get; set; }
    }
}

namespace LW4_task_3.Models.Request
{
    public class UserRegistration
    {
        public string? UserName { get; set; }
        public string? Email { get; set; }
        public string? Password { get; set; }
    }
}

Створено сервіс для хешування та сервіс для генерування accesstoken, refreshtoken:
using LW4_task_3.Interface.Interfaces;
using System.Security.Cryptography;
using System.Text;
namespace LW4_task_3.Services
{

```

```

public class PaswordHasher : IPasswordHasher
{
    public string Hash(string password)
    {
        using var sha256 = SHA256.Create();
        var bytes = Encoding.UTF8.GetBytes(password);
        var hash = sha256.ComputeHash(bytes);
        return Convert.ToBase64String(hash);
    }
    public bool Verify(string password, string hash)
    {
        var hashPas = Hash(password);
        return hashPas == hash;
    }
}

using LW4_task_3.Models;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using System.Text;
using System.Security.Claims;
using System.IdentityModel.Tokens.Jwt;
using LW4_task_3.Models.Entities;
using System.Security.Cryptography;
using LW4_task_3.Enums;
namespace LW4_task_3.Services
{
    public class JwtTokenGenerator
    {
        private readonly JwtSettings _jwtSettings;
        public JwtTokenGenerator(IOptions<JwtSettings> options)
        {
            _jwtSettings = options.Value;
        }
        public string GenerateToken(UserItem user)

```

```

    {
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_jwtSettings.SecretKey));

        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
        var claims = new List<Claim>
        {
            new Claim(JwtRegisteredClaimNames.Sub, user.Id),
            new Claim(JwtRegisteredClaimNames.Email, user.Email),
            new Claim(JwtRegisteredClaimNames.Name, user.UserName)
        };
        foreach (UserRole role in Enum.GetValues(typeof(UserRole)))
        {
            if (role == UserRole.None)
                continue;
            if (user.Role.HasFlag(role))
                claims.Add(new Claim(ClaimTypes.Role, role.ToString()));
        }
        claims.Add(new Claim("rolesValue", ((int)user.Role).ToString()));
        var token = new JwtSecurityToken(
            issuer: _jwtSettings.Issuer,
            audience: _jwtSettings.Audience,
            claims: claims,
            expires: DateTime.UtcNow.AddMinutes(_jwtSettings.AccessTokenExpirationMinutes),
            signingCredentials: creds
        );
        return new JwtSecurityTokenHandler().WriteToken(token);
    }
    public string GeneretaRefreshToken()
    {
        var randomNumbers = new byte[32];
        using var random = RandomNumberGenerator.Create();
        random.GetBytes(randomNumbers);
        return Convert.ToBase64String(randomNumbers);
    }
}

```

```

public ClaimsPrincipal? GetClaimsPrincipalFromExpiredToken(string token)
{
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_jwtSettings.SecretKey));

    var tokenValidationParameters = new TokenValidationParameters
    {
        ValidateAudience = true,
        ValidateIssuer = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = _jwtSettings.Issuer,
        ValidAudience = _jwtSettings.Audience,
        IssuerSigningKey = key,
        ValidateLifetime = false
    };

    var tokenHandler = new JwtSecurityTokenHandler();
    try
    {
        var principal = tokenHandler.ValidateToken(token, tokenValidationParameters, out var securityToken);
        if (securityToken is not JwtSecurityToken jwtSecurityToken ||
            !jwtSecurityToken.Header.Alg.Equals(SecurityAlgorithms.HmacSha256, StringComparison.InvariantCultureIgnoreCase))
            return null;

        return principal;
    }
    catch
    {
        return null;
    }
}

```

Створено сервіс та репозиторій для зберігання та роботи із сутностями в MongoDB:

					ЛР.ОК24.ПІ231.03.04-06	Арк.
Змін.	Арк.	№ докум.	Підпис	Дата		86

```

using LW4_task_3.Clients;
using LW4_task_3.Interface.InterfacesRepository;
using LW4_task_3.Models.Entities;
using MongoDB.Driver;
namespace LW4_task_3.Repositories
{
    public class UserRepository : IUserRepository
    {
        IMongoCollection<UserItem> _users;

        public UserRepository()
        {
            _users = MongoClient.Instance.GetCollection<UserItem>("Users");
        }

        public async Task CreateAsync(UserItem element) =>
            await _users.InsertOneAsync(element);

        public async Task DeleteAsync(string id) =>
            await _users.DeleteOneAsync(x => x.Id == id);

        public async Task<List<UserItem>> GetAllAsync() =>
            await _users.Find(x => true).ToListAsync();

        public async Task<UserItem> GetByEmailAsync(string email) =>
            await _users.Find(x => x.Email == email).FirstOrDefaultAsync();

        public async Task<UserItem> GetByIdAsync(string id) =>
            await _users.Find(x => x.Id == id).FirstOrDefaultAsync();

        public async Task<bool> IsExist(string id) =>
            await _users.Find(x => x.Id == id).AnyAsync();

        public async Task UpdateAsync(string id, UserItem element) =>
            await _users.ReplaceOneAsync(x => x.Id == id, element);
    }
}

```

Створено TestCotroller для роботи перевірки роботи автентифікації:

```

using LW4_task_3.Enums;
using LW4_task_3.Interface.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Http;

```

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.SignalR;
using System.Security.Claims;

namespace LW4_task_3.Controllers
{
    [Route("[controller]")]
    [ApiController]
    public class TestController : ControllerBase
    {
        IUserService _userService;

        public TestController(IUserService userService)
        {
            _userService = userService;
        }

        [Authorize]
        [HttpGet("private")]
        public async Task<IActionResult> GetIdEmail()
        {
            var email = User?.Claims.FirstOrDefault(c => c.Type == ClaimTypes.Email)?.Value;

            var user = await _userService.GetByEmailAsync(email);
            return Ok(new
            {
                UserId = user.Id,
                Email = email,
            });
        }
    }
}

```

Створено контролер для автентифікації, логіну та оновлення токену:

```

using AutoMapper;
using LW4_task_3.Interface.Interfaces;
using LW4_task_3.Models.Entities;
using LW4_task_3.Models.Request;

```

					ЛР.ОК24.ПІ231.03.04-06	Арк.
Змін.	Арк.	№ докум.	Підпис	Дата		88


```

using LW4_task_3.Services;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Configuration.UserSecrets;
using System.Security.Claims;
namespace LW4_task_3.Controllers
{
    [Route("[controller]")]
    [ApiController]
    public class AuthController : ControllerBase
    {
        private readonly IUserService _userService;
        private readonly IPasswordHasher _passwordHasher;
        private readonly JwtTokenGenerator _jwtGenerator;
        private readonly IMapper _mapper;
        public AuthController(IUserService userService,
            IPasswordHasher passwordHasher,
            JwtTokenGenerator JwtTokenGenerator, IMapper mapper)
        {
            _userService = userService;
            _passwordHasher = passwordHasher;
            _jwtGenerator = JwtTokenGenerator;
            _mapper = mapper;
        }
        [HttpPost("register")]
        public async Task<IActionResult> Reg(UserRegistration user)
        {
            var userItem = _mapper.Map<UserItem>(user);
            try
            {
                await _userService.CreateAsync(userItem);
                return Ok("Користувача зареєстровано");
            }
            catch (ArgumentException aex)
            {

```

```

        return BadRequest(aex.Message);
    }
}

[HttpPost("login")]
public async Task<IActionResult> LogIn(LoginModel loginModel)
{
    string message = "Невірна пошта чи пароль";
    UserItem user;
    try
    {
        user = await _userService.GetByEmailAsync(loginModel.Email);
    }
    catch (KeyNotFoundException)
    {
        return Unauthorized(message);
    }
    if(!_passwordHasher.Verify(loginModel.Password,user.Password))
        return BadRequest(message);
    var token = _jwtGenerator.GenerateToken(user);
    user.RefreshToken= _jwtGenerator.GeneretaRefreshToken();
    user.RefreshTokenExpireTime = DateTime.UtcNow.AddDays(7);
    await _userService.UpdateAsync(user.Id, user);
    return Ok(new
    {
        Token = token,
        TokenExpireTime = DateTime.UtcNow.AddMinutes(60),
        ResreshToken = user.RefreshToken,
        ResreshTokenExpireTime = user.RefreshTokenExpireTime
    });
}

[HttpPost("refresh")]
public async Task<IActionResult> Refresh([FromHeader(Name = "Authorization")]
string authHead, string refreshToken)
{
    if (string.IsNullOrEmpty(authHead) || !authHead.StartsWith("Bearer "))

```

```

        return BadRequest("Немає або неправильний заголовок");
var token = authHead.Substring("Bearer ".Length);
var princial = _jwtGenerator.GetClaimsPrincipalFromExpiredToken(token);
if (princial is null)
    return Unauthorized("Невірний token");
var id = princial?.Claims.FirstOrDefault(c => c.Type == ClaimTypes.NameIdentifier)?.Value;

    UserItem user;
    try
    {
        user = await _userService.GetByIdAsync(id);
    }
    catch(KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
    if (user.RefreshToken != refreshToken
        || user.RefreshTokenExpireTime <= DateTime.UtcNow)
        return Unauthorized("Невірний RefreshToken");
var newToken = _jwtGenerator.GenerateToken(user);
var newrefreshToken = _jwtGenerator.GeneretaRefreshToken();
user.RefreshToken = newrefreshToken;
user.RefreshTokenExpireTime = DateTime.UtcNow.AddDays(7);
await _userService.UpdateAsync(id, user);
return Ok(new
{
    Token = token,
    TokenExpireTime = DateTime.UtcNow.AddMinutes(60),
    ResreshToken = user.RefreshToken,
    ResreshTokenExpireTime = user.RefreshTokenExpireTime
});
}
}
}
}

```

Створено контролер для роботи із User:

					ЛР.ОК24.ПІ231.03.04-06	Арк.
Змін.	Арк.	№ докум.	Підпис	Дата		91

```

using AutoMapper;
using LW4_task_3.Enums;
using LW4_task_3.Interface.Interfaces;
using LW4_task_3.Models.Entities;
using LW4_task_3.Models.Request;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Protocols.OpenIdConnect;
using System.Data;
using ZstdSharp.Unsafe;
namespace LW4_task_3.Controllers.A_A
{
    [Route("[controller]")]
    [ApiController]
    [Authorize(Roles = nameof(UserRole.Admin))]
    public class UserController : ControllerBase
    {
        private readonly IUserService _userService;
        private readonly IMapper _mapper;
        private readonly IPasswordHasher _passwordHasher;
        public UserController(IUserService userService, IMapper mapper, IPasswordHasher
passwordHasher)
        {
            _userService = userService;
            _mapper = mapper;
            _passwordHasher = passwordHasher;
        }
        [Authorize(Roles = $" {nameof(UserRole.Admin)}, {nameof(UserRole.Manager)}")]
        [HttpGet]
        public async Task<IActionResult> GetAllUser()
        {
            try
            {
                var users = await _userService.GetAllAsync();
            }
        }
    }
}

```

```

        return Ok(users);
    }
    catch (KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
}

[Authorize(Roles = $"{nameof(UserRole.Admin)}, {nameof(UserRole.Manager)}")]
[HttpGet("{id}")]
public async Task<IActionResult> GetOneUser(string id)
{
    try
    {
        var user = await _userService.GetByIdAsync(id);
        return Ok(user);
    }
    catch(KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
}

[HttpPut("{id}")]
public async Task<IActionResult> UpdateUser(string id, UserRegistration userRegi
stration)
{
    try
    {
        var updateUser = await _userService.GetByIdAsync(id);
        var emailUser = await _userService.GetByEmailAsync(userRegistration.Email)
;

        if (emailUser is not null && emailUser.Id != updateUser.Id)
            return BadRequest("Ця пошта вже використовується");

        userRegistration.Password = _passwordHasher.Hash(userRegistration.Passwor
d);

        _mapper.Map(userRegistration, updateUser);
    }
}

```

```

        await _userService.UpdateAsync(id, updateUser);
        return NoContent();
    }
    catch (KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
}

[HttpDelete("{id}")]
public async Task<IActionResult> DeleteUser(string id)
{
    try
    {
        await _userService.DeleteAsync(id);
        return NoContent();
    }
    catch (KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
}

[HttpPost("setRole")]
public async Task<IActionResult> SetRole(string id, UserRole role)
{
    try
    {
        var user = await _userService.GetByIdAsync(id);
        user.Role = role;
        await _userService.UpdateAsync(id, user);
        return Ok(new { UserName = user.UserName, Email = user.Email, Role = (int)
user.Role });
    }
    catch (KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
}

```

```

    }
}
[HttpPost("addRole")]
public async Task<IActionResult> AddRole(string id, UserRole role)
{
    try
    {
        var user = await _userService.GetByIdAsync(id);
        user.Role |= role;
        await _userService.UpdateAsync(id, user);
        return Ok(new { Username = user.Username, Email = user.Email, Role = (int)
user.Role });
    }
    catch (KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
}
[HttpPost("removeRole")]
public async Task<IActionResult> RemoveRole(string id, UserRole role)
{
    try
    {
        var user = await _userService.GetByIdAsync(id);
        user.Role &= ~role;
        await _userService.UpdateAsync(id, user);
        return Ok(new { Username = user.Username, Email = user.Email, Role = (int)
user.Role });
    }
    catch (KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
}
}

```

Протестовано роботу програми з різними ролями:

Рисунок 140 – Реєстрація нового користувача

Рисунок 141 – Невірно введений пароль

Рисунок 142 – Вхід із роллю Admin

Рисунок 143 – Запит на TestController

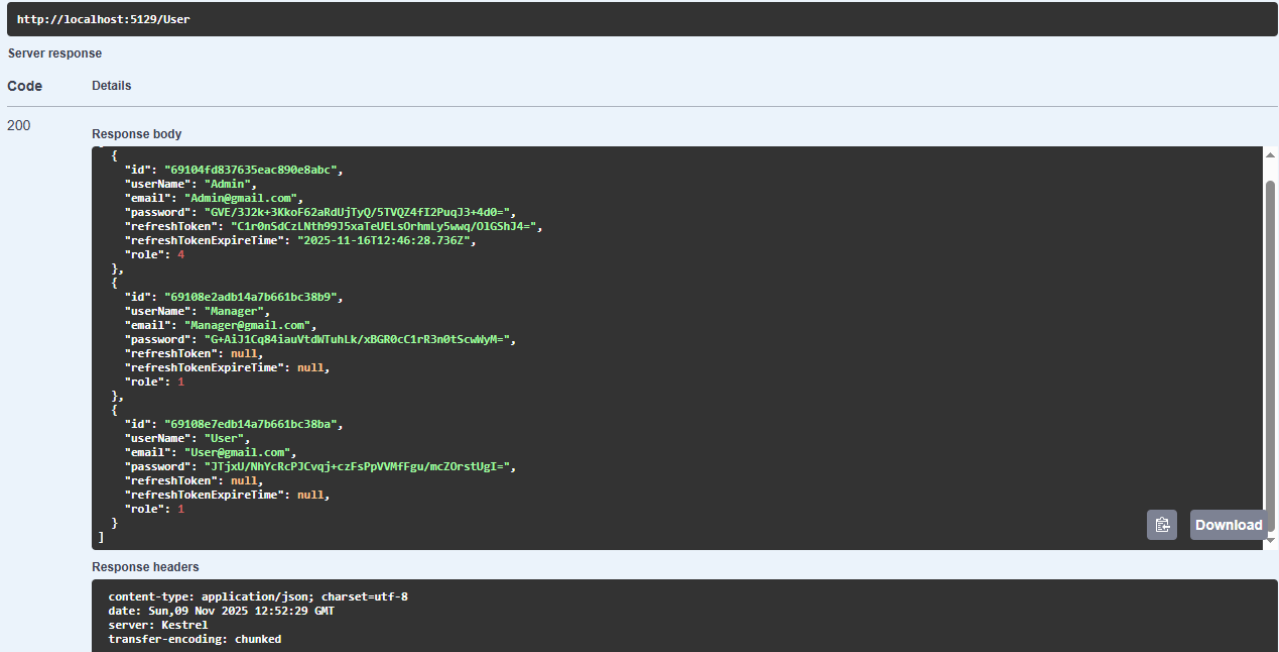


Рисунок 144 – Переглянути список всіх User-ів



Рисунок 145 – Отримано інформацію про конкретного User-а



Рисунок 146 – Вкористано /setRole для зміни ролі менеджера

Request URL	
<code>http://localhost:5129/User/addRole?id=69108e2adb14a7b661bc38b9&role=2</code>	
Server response	
Code	Details
200	Response body <pre>{ "userName": "Manager", "email": "Manager@gmail.com", "role": 3 }</pre>  Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sun,09 Nov 2025 12:56:50 GMT server: Kestrel transfer-encoding: chunked</pre>

Рисунок 147 – Додано роль менеджера через /addRole

Request URL	
<code>http://localhost:5129/User/removeRole?id=69108e2adb14a7b661bc38b9&role=1</code>	
Server response	
Code	Details
200	Response body <pre>{ "userName": "Manager", "email": "Manager@gmail.com", "role": 2 }</pre>  Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sun,09 Nov 2025 12:57:57 GMT server: Kestrel transfer-encoding: chunked</pre>

Рисунок 148 – Видалено роль User через /removeRole

Request URL	
<code>http://localhost:5129/User/69108e2adb14a7b661bc38b9</code>	
Server response	
Code	Details
204 <i>Undocumented</i>	Response headers <pre>date: Sun,09 Nov 2025 13:00:18 GMT server: Kestrel</pre>

Рисунок 149 – Оновлено ім'я для Manager

Request URL	
<code>http://localhost:5129/User/69108e2adb14a7b661bc38b9</code>	
Server response	
Code	Details
200	Response body <pre>{ "id": "69108e2adb14a7b661bc38b9", "userName": "NewManager", "email": "Manager@gmail.com", "password": "GzAi1fG84iauVtdmTuhLk/xBGR0cC1rR3n0tScwMy#=", "refreshToken": null, "refreshTokenExpiration": null, "role": 2 }</pre>  Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sun,09 Nov 2025 13:01:14 GMT server: Kestrel transfer-encoding: chunked</pre>

Рисунок 150 – Результат оновлення

Request URL	
<code>http://localhost:5129/User/69108e2adb14a7b661bc38b9</code>	
Server response	
Code	Details
204 <i>Undocumented</i>	Response headers <pre>date: Sun,09 Nov 2025 13:02:04 GMT server: Kestrel</pre>

Рисунок 151 – Видалено оновленого користувача

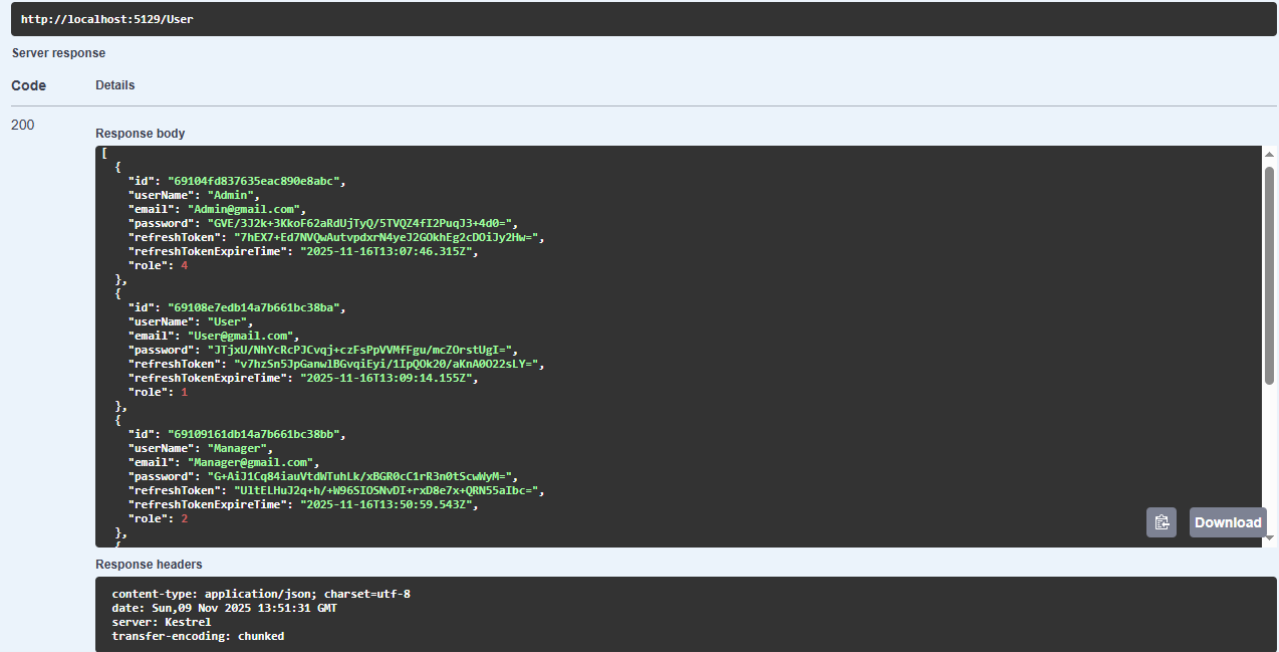


Рисунок 157 – Отримано список всіх User-ів

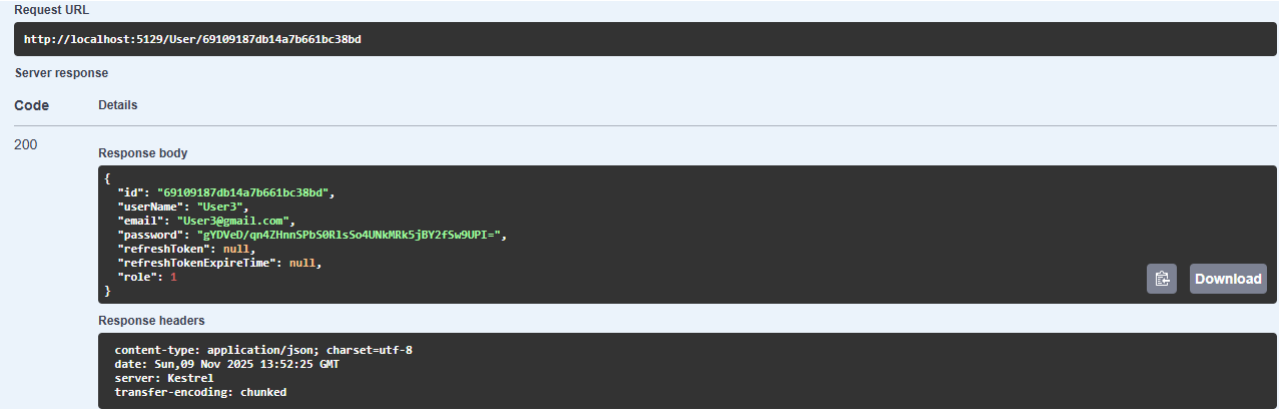


Рисунок 158 – Отримано конкретного User-а



Рисунок 159 – Спроба видалити User-а із роллю Manager

Request URL

http://localhost:5129/People/68f5e054f21b02f6aece46ee

Server response

Code	Details
200	Response body <pre>{ "id": "68f5e054f21b02f6aece46ee", "name": "Susanteam", "email": "Susan@email.com" }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sun, 09 Nov 2025 14:04:02 GMT server: Kestrel transfer-encoding: chunked</pre>

Рисунок 164 – Отримано конкретного користувача підписки

Request URL

http://localhost:5129/People/68f5e054f21b02f6aece46ee

Server response

Code	Details
403 <i>Undocumented</i>	Error: Forbidden Response headers <pre>content-length: 0 date: Sun, 09 Nov 2025 14:04:46 GMT server: Kestrel</pre>

Рисунок 165 – Споба видалення користувача підписки

Висновок: на даній лабораторній роботі я реалізував CRUD – операції у C# ASP.Net Web – API, Minimal API та навчився працювати із ними, також здобув практичні навички по створенню RESTful API для взаємодії із клієнтом. Також засвоїв як реалізувати та працювати з тришаровою архітектурою, як використовувати DI. Також я засвоїв роботу із автентифікацією та авторизацією